TÉCNICO
UNIVERSIDADE
DE LISBOA



Cooperative control of a tethered drone and surface vessel for sustained presence at sea

Tomás Grossinho Esteves

Thesis to obtain the Master of Science Degree in

Electrical and Computer Engineering

Supervisor(s): Prof. David Alexandre Cabecinhas
Prof. Rita Maria Mendes de Almeida Correia da Cunha

Examination Committee

Prof. Meysam Basiri

January 2026

Declaration

I declare that this document is an original work of my own authorship and that it fulfills all the requirements of the Code of Conduct and Good Practices of the Universidade de Lisboa.

Contents

Acronyms ix

1 Introduction **1**

1.1 Motivation 1

1.2 Problem Statement 2

1.3 System Overview 3

1.4 Expected Contributions 4

1.5 Thesis Structure 4

2 State of the Art and Related Work **7**

2.1 Modelling and System Identification 7

2.1.1 White-Box, Grey-Box, and Black-Box Modelling 7

2.1.2 Tether Dynamics Modelling 8

2.1.3 Optimisation-Based Estimation 8

2.1.4 Related Work on Tethered Vehicle Modelling 9

2.2 Control Strategies for Autonomous Vehicles 9

2.2.1 Classical and Modern Control 9

2.2.2 Predictive and Optimisation-Based Control 10

2.2.3 Active Winch and Tension Control 10

2.2.4 Related Work on Tethered Vehicle Control 11

2.3 Cooperative Coordination and Systems 11

2.3.1 Centralised vs. Distributed Coordination 11

2.3.2 Unmanned Aerial Vehicle (UAV)–Unmanned Surface Vessel (USV) Cooperative Systems 11

2.3.3 Related Work on Maritime Cooperative Robotics 12

2.4 Path and Trajectory Planning 12

2.4.1 Offline and Online Planning 12

2.4.2 Related Work on Trajectory Planning 13

3 Proposed Methodology **15**

3.1 System Modelling 15

3.1.1 Reference Coordinate Frames and Notation 15

3.1.2 WAM-V Model . . . 16
3.1.3 x500 UAV Model . . . 20
3.2 System Control . . . 25
3.2.1 Cooperative Control Design . . . 25
3.2.2 Non-linear Model Predictive Control . . . 27
3.2.3 USV Non-linear Model Predictive Control (NMPC) . . . 29
3.2.4 UAV NMPC . . . 33

4 Implementation **41**
4.1 Simulation Environment . . . 41
4.1.1 World . . . 41
4.1.2 Wave Adaptive Modular Vessel . . . 42
4.1.3 x500 UAV . . . 44
4.1.4 Tether . . . 46
4.1.5 Tether Winch . . . 48
4.2 Models Validation . . . 50
4.2.1 USV Model . . . 50
4.2.2 UAV Model . . . 53
4.2.3 Tether Model . . . 57
4.3 Control Implementation . . . 60
4.3.1 Optimization Framework . . . 60
4.3.2 USV Control . . . 60
4.3.3 UAV Control . . . 62
4.4 Manoeuvres and Missions . . . 65
4.4.1 Manoeuvres . . . 65
4.4.2 Missions . . . 67
4.5 Software Architecture and Mission Control . . . 68
4.5.1 Architecture . . . 68
4.5.2 Mission Control Centre . . . 68

5 Results **71**
5.1 Trajectory Tracking . . . 71
5.1.1 Nominal Tracking . . . 71
5.1.2 Tether Constraint . . . 71
5.1.3 Actuator Limits . . . 71
5.2 Missions . . . 71
5.2.1 Scout . . . 71
5.2.2 Inspection . . . 71
5.2.3 Patrol . . . 71
5.3 Disturbances . . . 71

5.3.1 Waves 71
5.3.2 Wind 71

6 Conclusions **72**
6.1 Achievements 72
6.2 Future Work 72

Bibliography **73**

Acronyms

AHC	active heave compensation
DOF	degree of freedom
EMSA	European Maritime Safety Agency
ENU	East-North-Up
GPS	Global Positioning System
HPIPM	High-Performance Interior Point Method
LQR	linear quadratic regulator
LTI	linear time-invariant
MPC	Model Predictive Control
NMPC	Non-linear Model Predictive Control
NRMSE	normalised root-mean-square error
PD	proportional–derivative
PEM	prediction error method
PID	proportional–integral–derivative
PRM	Probabilistic Roadmap
QP	quadratic programming
RMSE	root-mean-square error
ROS	Robot Operating System
RRT	Rapidly-exploring Random Tree
RTI	real-time iteration
RTK	Real-Time Kinematic
SITL	Software-in-the-Loop
SQP	sequential quadratic programming
UAV	Unmanned Aerial Vehicle
USV	Unmanned Surface Vessel
VRX	Virtual RobotX
WAM-V	Wave Adaptive Modular Vessel

Chapter 1

Introduction

This chapter introduces the thesis, presenting the context, motivation, and objectives of the research, followed by an overview of the system architecture, the expected scientific and engineering contributions, and the overall structure of the dissertation.

1.1 Motivation

Saving lives at sea, protecting marine ecosystems, and securing maritime infrastructure all rely on one essential capability: understanding what is happening, where, and when. In vast and dynamic maritime environments, this requires maintaining reliable, high-resolution situational awareness over long periods. The European Maritime Safety Agency (EMSA) recognises this challenge and highlights the need for continuous monitoring and locally persistent operations to enable effective maritime surveillance and response [1].

Different maritime assets meet this challenge in complementary ways. Surface vessels give a physical presence at sea, with long endurance and large payload capacity [2, 3]. However, their range is limited by line of sight, and operations are expensive [4]. Aerial platforms, such as manned aircraft and UAVs, allow fast deployment and elevated sensing [5]. However, aircraft are costly, and UAVs suffer from low endurance and unreliable communication [6].

Together, these assets form a connected system rather than separate solutions. Each one compensates for what the other lacks. USVs provide long-term presence at sea at much lower cost than crewed vessels, allowing wide and affordable patrols. UAVs extend this reach by sensing beyond the horizon from the air. Working as a team, they maintain continuous and local maritime awareness. This motivates the study of coordinated USV–UAV systems, where cooperation creates mission capabilities that neither platform can achieve alone in complex and changing environments.

A natural way to strengthen this cooperation is to add a physical tether between both platforms. The tether provides continuous power and reliable communication between the USV and UAV [7]. By placing the energy supply and communication systems on the USV, it overcomes two main limits of maritime UAVs: short endurance and weak long-range communication. This approach has recently transitioned

from academic concepts to real-world industrial systems; for instance, the marine robotics company Exail partnered with Elistair to integrate a tethered UAV directly onto the DriX USV catamaran for persistent maritime surveillance [8]. Similarly, ACUA Ocean and Teledyne FLIR Defense demonstrated an integrated tethered aerial-surface system for the UK Ministry of Defence to support long-endurance maritime security and target detection [9].

Figure 1.1 illustrates the configuration of the tethered USV–UAV system considered in this work.

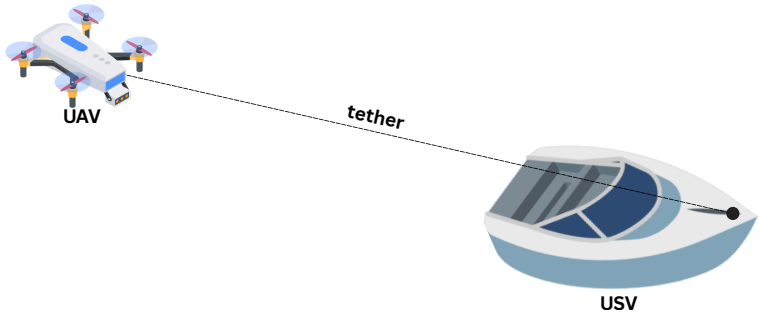


Figure 1.1: Schematic of a tethered USV–UAV system.

The main challenge of this work is to design a control framework that enables two autonomous vehicles, connected by a tether, to operate as a single coordinated system. The controller must manage not only the motion of both vehicles but also account for the tether constraints and dynamics, ensuring safe operation and robustness to wind, waves, and obstacles.

The USV and UAV operate under very different dynamics. The USV moves slowly, dominated by hydrodynamic drag and inertia, while the UAV reacts quickly, with motion evolving on much shorter time scales. Coordinated control is challenging because the controller must handle these different response rates and actuation dynamics consistently.

The maritime environment adds further complexity. Wind, waves, and water motion constantly disturb both platforms, making robustness to disturbances essential. Nearby structures, vessels, or obstacles can restrict their movement and risk tether entanglement. Effective cooperation relies on continuous information exchange and synchronised decision-making between both agents.

Most existing tethered UAV systems focus on power delivery and communication, with operation often remaining manual or semi-autonomous. In research prototypes, the aerial and surface vehicles are usually decoupled, the tether is simplified or linearised, or the surface platform is treated as fixed. These assumptions make control design easier but reduce realism. Few cooperative control approaches explicitly model the tether while coordinating both agents under realistic maritime conditions.

1.2 Problem Statement

This work addresses the problem of controlling a cooperative system composed of a USV (the Wave Adaptive Modular Vessel (WAM-V)) and a physically connected UAV (the x500) operating in a coordinated manner. The objective of such a system is to autonomously execute maritime missions by

following predefined waypoints or continuous reference trajectories, while respecting the physical and operational constraints of both platforms and of the tether that connects them.

To achieve this objective, the control framework must be capable of handling different phases of operation and mission requirements. This includes the definition of multiple control modes that allow the system to perform manoeuvres such as takeoff and landing, as well as different forms of coordinated motion during navigation. Both NMPCs actively prevent the tether from exceeding the maximum admissible separation distance, utilising soft constraints and slacks, while exchanging predicted trajectories to coordinate the motion of the UAV and the dynamic USV deck.

1.3 System Overview

Interaction with the system is defined at the mission level. The operator selects a mission, such as patrolling a given area, following a moving target, or visiting a set of waypoints that represent regions of interest. This mission information is received by a behaviour management layer, which selects the active control mode and handles mission logic, including switching between different relative configurations of the UAV with respect to the USV, as well as executing specific manoeuvres such as takeoff and landing.

Based on the selected mission and active control mode, the system generates coordinated motion for both the USV and the UAV. This process combines planning and control to compute trajectories and actuation commands that satisfy vehicle and tether constraints while pursuing the mission objectives. The WAM-V USV acts as the leader planar platform carrying an active winch system on its deck. The winch dynamically spools the tether in or out to manage slack and tether tension. The x500 UAV follows a relative orbit or observation trajectory above the USV, using its elevated perspective to record visual data.

Figure 1.2 presents a high-level block diagram of the proposed system, showing the interaction between the coordinator, the decentralised NMPC solvers, the vehicles, and the co-simulation interfaces.

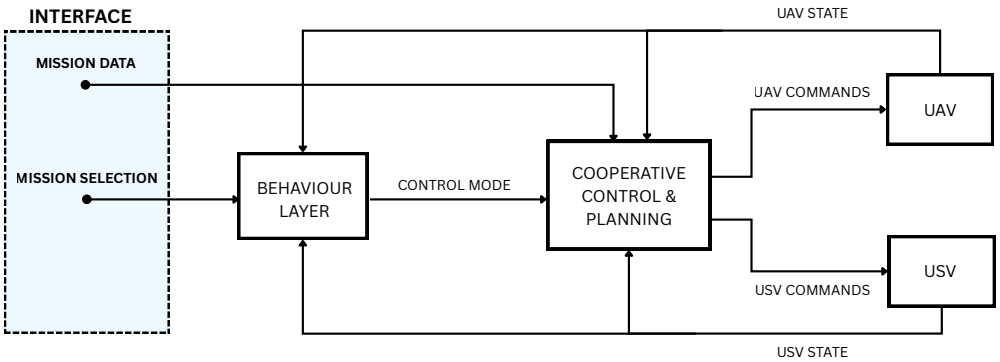


Figure 1.2: High-level block diagram of the proposed cooperative control system.

1.4 Expected Contributions

This thesis contributes to the design, implementation, and evaluation of cooperative control solutions for tethered UAV–USV systems, to enable autonomous and persistent maritime missions. The main contributions of this work are summarised as follows:

- A unified, C++ Robot Operating System (ROS) 2 control pipeline utilising Acados and High-Performance Interior Point Method (HPIPM) to generate and execute real-time non-linear model predictive control solvers for both surface tracking and aerial guidance.
- The modelling and experimental identification of the UAV inner-loop attitude response lag ($\tau = 0.15\,\text{s}$) using PX4 doublet flight data.
- The design and implementation of a virtual winch controller operating in both geometric and tension-feedback proportional–derivative (PD) modes to prevent tether slackness and tension spikes.
- A realistic co-simulation setup in Gazebo Harmonic, integrated with the PX4 Software-in-the-Loop (Software-in-the-Loop (SITL)) firmware and the MoorDyn physics engine for multi-body lumped-mass tether representation.
- The verification of NMPC constraint enforcement (states, inputs, and tether separation limits) and evaluation of solver latency.
- The validation of the entire cooperative control stack under an integrated search and pursuit mission targeting a USV teleoperated in real time.

1.5 Thesis Structure

The remainder of this dissertation is organised as follows:

- **Chapter 2 (Literature Review and State of the Art)** reviews existing research in cooperative marine-aerial robotics, tethered UAVs, and predictive control algorithms.
- **Chapter 3 (System Modelling)** derives the kinematic and dynamic models of both vehicles, as well as the catenary tether force model.
- **Chapter 4 (Cooperative Control Design)** details the decentralised NMPC mathematical formulation, cost functions, and physical constraints for both platforms.
- **Chapter 5 (Simulation Environment and Software Implementation)** introduces the physical characteristics of the virtual WAM-V and x500 platforms, the active winch spooling controller, the MoorDyn tether integration, the simulation setup, and the software modularity.
- **Chapter 6 (Results and Discussion)** evaluates the model validation, parameter identification, tracking precision, constraint enforcement, solver latency, and the target pursuit mission.

- **Chapter 7 (Conclusions and Future Work)** summarises the findings of the thesis and outlines directions for future research.

Chapter 2

State of the Art and Related Work

This chapter is structured into four main sections, each representing a key element of cooperative system control. Each section presents the main techniques commonly used in control theory, together with subsections reviewing related work on topics closely related to this research.

The first section addresses *modelling and identification*, which provides the mathematical and experimental tools required to describe system dynamics and estimate relevant parameters, including tether physics. The second focuses on *control strategies*, ensuring stability, accurate tracking, and robustness to disturbances at the individual agent level, including predictive control and active winch architectures. The third section concerns *cooperative coordination*, which enables communication and shared decision-making so that multiple agents can operate as a unified system. Finally, *path and trajectory planning* deals with the generation of feasible and safe motion plans that connect mission objectives with low-level control actions.

2.1 Modelling and System Identification

Understanding how a system evolves is a prerequisite for any control design. Modelling provides this understanding by offering a mathematical description of the system dynamics, capturing the dominant behaviours while deliberately neglecting secondary effects that are not essential for analysis or control purposes [10]. The level of detail of such a model is inherently linked to the available physical insight, the amount and quality of experimental data, and the specific objectives for which the model is intended.

2.1.1 White-Box, Grey-Box, and Black-Box Modelling

Depending on the available physical knowledge and data, modelling methodologies are classified into three main groups:

- **White-Box Modelling:** Based on first principles and physical laws that govern system behaviour. By expressing the dynamics through analytical formulations, often derived from Newtonian mechanics, hydrodynamics, or aerodynamics, this approach provides clear insight into stability, con-

trollability, and energy transfer. To preserve analytical tractability, simplifying assumptions such as linearisation or the neglect of higher-order effects are commonly introduced.

- **Grey-Box Modelling:** Lies between purely analytical and purely data-driven approaches. The structure of the model is defined using physical insight, while parameters that are difficult to determine analytically, such as hydrodynamic or aerodynamic coefficients, are estimated from experimental data. This combination preserves a clear physical interpretation while improving accuracy through calibration.
- **Black-Box Modelling:** Relies entirely on measured input–output data to describe system behaviour. Rather than using explicit physical laws, this approach identifies mathematical relationships that best fit the observed data. Typical examples include linear parametric models, state-space representations obtained from data, and learning-based methods.

2.1.2 Tether Dynamics Modelling

Modelling flexible tethers is critical since they introduce highly non-linear forces and kinematic constraints on the vehicles:

- **Quasistatic Catenary Models:** Describe the steady-state profile of a heavy tether suspended between two points. These models use catenary equations to resolve tether sag and tension analytically, assuming the tether is in static equilibrium at each step. While computationally efficient, they neglect dynamic effects such as transverse wave propagation and drag forces, making them unsuitable for rapid aerial manoeuvres.
- **Lumped-Mass Models:** Discretise the tether into a finite chain of nodes, or "lumped masses", connected by spring-damper segments. Each segment models axial elasticity, damping, and hydrodynamic drag, while the nodes resolve gravitational and buoyancy forces. This formulation, implemented in solvers such as MoorDyn, offers a high-fidelity representation of transient tether whip, slackness, and hydrodynamic drag under dynamic boundary conditions.
- **Finite-Segment and Multi-Body Models:** Formulate the tether as a chain of rigid cylinders connected by spherical joints. This approach computes bending and torsional moments accurately, but requires solving high-index differential-algebraic equations, which increases the computational burden.

2.1.3 Optimisation-Based Estimation

System identification offers an alternative path by constructing models directly from measured input–output data when physical knowledge is incomplete. Optimisation-based estimation focuses on identifying model parameters offline by minimising the mismatch between measured and predicted system outputs. This is typically achieved through cost functions based on least-squares formulations, including weighted least-squares and prediction-error methods (prediction error method (PEM)) [11]. For

non-linear or constrained problems, advanced optimisation schemes such as trust-region methods and constrained least-squares are often employed, allowing grey-box models to be calibrated while enforcing physical bounds and regularisation.

2.1.4 Related Work on Tethered Vehicle Modelling

In systems connected by a tether, the model of the overall system must account at least for the tether geometry to constrain the relative motion of the agents and avoid physically infeasible configurations that could lead to tether failure.

In [12], a buoy–UAV system is presented in which the tether is modelled explicitly as a geometric and dynamic constraint between the buoy and the aerial vehicle. The tether tension is computed from the coupled dynamics and incorporated as an external force acting along the tether direction in the UAV model. This formulation relies on simplifying assumptions, namely that the tether is inextensible, massless, free of hydrodynamic effects, and remains taut during coupled operation.

An alternative modelling perspective is proposed in [13], where tether-induced forces are not included explicitly in the system dynamics. Instead, their effect is lumped into an external disturbance acting on the UAV. This disturbance is then handled at the control level through active disturbance rejection, avoiding the need for explicit tether modelling while still compensating for its influence in real time.

In [14], a cooperative UAV–USV system is modelled for the purpose of manipulating a floating object. Both vehicles are connected to the object through individual tethers, and the full system is described using simplified white-box models. The formulation assumes inextensible and massless tethers that remain permanently taut.

In [15], a tether is not included, and both the UAV and the USV are described using simplified white-box models. The USV is represented by a planar marine vehicle model commonly used for path-following applications, while the UAV is modelled independently using a reduced-order dynamic model.

2.2 Control Strategies for Autonomous Vehicles

Control theory provides the mathematical framework for designing systems that regulate their behaviour through feedback, ensuring stability, accuracy, and robustness in the presence of uncertainty and disturbances.

2.2.1 Classical and Modern Control

Classical control is based on the representation of systems through transfer functions in the Laplace or frequency domain, under the assumption of linear time-invariant (linear time-invariant (LTI)) dynamics. Within this framework, feedback laws are typically implemented through proportional–integral–derivative (proportional–integral–derivative (PID)) controllers and frequency-domain compensators designed to shape the closed-loop response.

Modern control theory generalises these principles to systems that are multivariable, time-varying, or subject to strong coupling between states and inputs. It is formulated in the *state-space* domain, where the internal dynamics of a system are described by sets of first-order differential equations. This representation enables systematic analysis of controllability, observability, and stability. Within this framework, optimal control theory defines feedback laws that minimise a performance index, such as the linear quadratic regulator (linear quadratic regulator (LQR)) or robust control methods such as H_∞ [16, 17].

2.2.2 Predictive and Optimisation-Based Control

Predictive and optimisation-based control methods determine control actions by solving an optimisation problem that predicts the system behaviour over a finite horizon. At each sampling instant, the controller minimises a cost function subject to system dynamics and operational constraints, typically penalising deviations from reference trajectories and excessive control effort. Only the first control action is applied, and the process repeats at the next time step, forming the receding-horizon principle that defines Model Predictive Control (MPC) [18].

Extensions to non-linear systems led to the development of NMPC, which replaces linear models with non-linear formulations and uses numerical optimisation techniques to compute control inputs in real time [19, 20]. To enable real-time embedded execution on high-rate systems (e.g. UAV flight control), modern applications rely on symbolic framework code generators (such as CasADi) that export native C solvers. These solvers utilise highly efficient quadratic programming (QP) kernels such as HPIPM and execution frameworks like the sequential quadratic programming real-time iteration (sequential quadratic programming (SQP)–real-time iteration (RTI)) scheme, which performs a single optimisation iteration per time step to minimise solver latency.

2.2.3 Active Winch and Tension Control

In tethered UAV operations, winch design is a core element:

- **Passive Tension Tethers:** Rely on spring-loaded spools or constant-torque slip rings. While simple and requiring no electrical control, they cannot adapt to fast USV manoeuvres, leading to large tension spikes or excessive slack.
- **Active Spooling Winches:** Utilise motorised actuators regulated by microcontrollers. The winch dynamically adjusts the tether length based on geometric tracking (distance between platforms) or force-feedback algorithms.
- **Tension-Feedback and Active Heave Compensation (active heave compensation (AHC)):** Read force telemetry from a load cell. A closed-loop proportional–derivative (PD) controller regulates the winch spooling speed to maintain a target tension. In maritime environments, this functions as AHC, isolating the flying UAV from wave-induced USV heave.

2.2.4 Related Work on Tethered Vehicle Control

In tethered UAV systems, the tether introduces additional forces and coupling effects that influence control design. In [12], this coupling is addressed through a non-linear backstepping controller derived directly from the white-box dynamics of the UAV–buoy system. The work in [13] adopts an alternative approach based on active disturbance rejection, where the tether influence is not explicitly modelled but treated as an external disturbance estimated in real time by a reduced-order observer.

In [14], cooperative manipulation with two tethered agents is controlled using a MPC formulation based on simplified white-box models of the UAV, the USV, and the floating object. A similar predictive strategy is presented in [21], where both the UAV and the USV are coordinated using NMPC, relying on known dynamic models.

In [15], the USV employs a line-of-sight guidance law combined with PID control for heading and speed regulation. The UAV motion is controlled using an NMPC formulation, which generates reference accelerations to track the desired relative trajectory. No joint or centralised control law is employed, and each vehicle is controlled using its own dedicated controller.

2.3 Cooperative Coordination and Systems

Cooperative coordination refers to the control and organisation of multiple agents working towards achieving goals through communication and joint decision-making.

2.3.1 Centralised vs. Distributed Coordination

Depending on how information is exchanged and the control authority is structured, cooperative architectures are broadly categorised as centralised or distributed:

- **Centralised Coordination:** A single decision node aggregates global information and computes control commands for all agents. This architecture facilitates global optimisation and ensures consistency of mission-level objectives, since the planner has access to full system states and constraints. However, centralised control scales poorly with team size and is vulnerable to communication delays or single-point failures.
- **Distributed Coordination:** Eliminates the need for a central planner by allowing each agent to make decisions based on locally available information and limited communication. Agents exchange state or intent data to achieve group-level objectives. Distributed coordination provides scalability and robustness to communication losses and agent failures, though it faces challenges related to synchronisation and network stability.

2.3.2 UAV–USV Cooperative Systems

Cooperative deployments of UAVs and USVs combine the speed and altitude of aerial sensing with the weight-bearing capability and range of marine hulls:

- **Autonomous Landing on Moving Surfaces:** Focuses on landing a UAV on a moving deck (helideck) in waves. This requires precise relative state estimation (visual markers, Real-Time Kinematic (RTK)–Global Positioning System (GPS)) and predictive control to synchronise the UAV's landing velocity with the sea-state pitch/roll profile of the USV deck.
- **Mobile Recharging Stations:** The USV acts as a mothership. UAVs fly reconnaissance sweeps, return to land on the USV deck to swap or recharge batteries, and take off again, enabling long-term persistence at sea.
- **Cooperative Target Tracking:** Cooperative path-following algorithms steer the USV planar trajectory to intercept a target, while the UAV orbits above to provide continuous visual tracking and image feedback.

2.3.3 Related Work on Maritime Cooperative Robotics

In [21], cooperation is based on a tracking strategy in which the UAV is controlled to follow and land on a moving USV. Only the UAV is actively controlled, while the USV moves independently and provides its state as a reference. The interaction is therefore unidirectional, with the UAV adapting its motion to the USV.

In contrast, [14] proposes a centralised model predictive control approach in which the UAV and the USV are controlled jointly. A single optimisation problem computes the control inputs for both vehicles, explicitly accounting for the coupling introduced by the tether. This approach focuses on cooperative object manipulation rather than on mission-level coordination between the vehicles.

In [15], coordination is achieved through a leader–follower structure. The USV acts as the leader and follows a predefined path, while the UAV adjusts its motion to maintain a desired relative configuration with respect to the USV. Coordination is therefore based on reference sharing and relative motion constraints, without joint optimisation or centralised decision-making.

2.4 Path and Trajectory Planning

Path and trajectory planning address the problem of generating feasible, collision-free, and dynamically consistent motions for autonomous systems operating in structured or unstructured environments, translating abstract goals into trackable trajectories.

2.4.1 Offline and Online Planning

Methods are broadly divided into offline and online approaches, depending on when the plan is computed and how environmental information is handled:

- **Offline Planning:** Performed before execution, assuming full or near-complete knowledge of the operating environment. Classical approaches include graph-search methods (Dijkstra's, A*), sampling-based methods (Rapidly-exploring Random Tree (RRT), Probabilistic Roadmap (PRM)),

and trajectory optimisation. While generating globally optimal paths, they are computationally demanding and cannot adapt to dynamic changes.

- **Online Planning:** Addresses the generation and adaptation of trajectories in real time under dynamic or uncertain conditions. Reactive techniques such as potential field methods and dynamic window approaches provide fast obstacle avoidance but can suffer from local minima. Optimisation-based MPC frameworks integrate vehicle dynamics into the planning loop, generating dynamically consistent collision-free paths online.

2.4.2 Related Work on Trajectory Planning

A centralised predictive planning approach is presented in [14], where a single MPC formulation jointly optimises the motion of both the UAV and the USV. In this case, planning is achieved online through joint trajectory optimisation that explicitly accounts for the coupling introduced by tethers.

Planning is addressed from a different perspective in [15], where cooperative behaviour is achieved through coordinated path following. A predefined path is assigned to the USV, while the UAV generates its motion online as part of the control process to maintain coordination. In this framework, planning is limited to the online generation of the UAV motion, and no cooperative path planning between the two vehicles is performed.

Overall, existing works typically adopt one of two approaches to planning in UAV–USV systems: either a predefined path is assigned to one vehicle while the other generates its motion online to follow or support it, or both vehicles generate their motion online as part of a predictive control framework.

Chapter 3

Proposed Methodology

This chapter presents the theoretical and algorithmic methodology developed in this work for the autonomous cooperative control of the tethered USV–UAV system. It is structured into two main parts: Section 3.1 derives the control-oriented kinematic and dynamic models of both vehicles, their actuator mappings, and the physical coupling introduced by the tether; Section 3.2 then formulates the cooperative non-linear model predictive control (NMPC) architecture that exploits these models to compute optimal, collision-free, and constraint-respecting trajectories.

3.1 System Modelling

Real-time control needs models that are simple enough to compute quickly, yet detailed enough to capture how each vehicle actually moves and interacts with the other. The kinematics, dynamics, and physical coupling between the two vehicles let the controller predict how the system evolves and compute feasible, optimal trajectories.

The reference frames and notation are fixed first, followed by the kinematic and dynamic models of the USV and the UAV.

3.1.1 Reference Coordinate Frames and Notation

Deriving the equations of motion requires specifying the frames against which positions, orientations, and velocities are measured. Since the USV and the UAV move both with respect to the world and with respect to each other, a common frame of reference is required to describe their motion. Figure 3.1 illustrates these frames.

Inertial Reference Frame $\{\mathcal{I}\}$ This frame establishes the global coordinate system following the East-North-Up (East-North-Up (ENU)) convention, aligned with the standard ROS convention. Its origin $\mathcal{O}_I$ is fixed at an arbitrary point on the water surface within the operating area, following the usual flat-Earth convention. The axis $\mathbf{x}^{\mathcal{I}}$ points East, $\mathbf{y}^{\mathcal{I}}$ points North, and $\mathbf{z}^{\mathcal{I}}$ points upward, opposite to gravity.

USV Body-Fixed Reference Frame $\{\mathcal{B}_v\}$ This frame is rigidly attached to the USV, with origin $\mathcal{O}_v$ at its centre of gravity. This choice decouples the linear and rotational momentum equations. The forward-pointing axis $\mathbf{x}^{\mathcal{B}_v}$ defines surge, $\mathbf{y}^{\mathcal{B}_v}$ points to port (sway), and $\mathbf{z}^{\mathcal{B}_v}$ points upward (heave).

UAV Body-Fixed Reference Frame $\{\mathcal{B}_a\}$ Similarly, this frame attaches to the UAV, with origin $\mathcal{O}_a$ at its centre of gravity, decoupling the translational and rotational dynamics in the same way. The longitudinal axis $\mathbf{x}^{\mathcal{B}_a}$ points forward, the lateral axis $\mathbf{y}^{\mathcal{B}_a}$ points to the left, and the vertical axis $\mathbf{z}^{\mathcal{B}_a}$ points upward, aligning with the collective thrust direction.

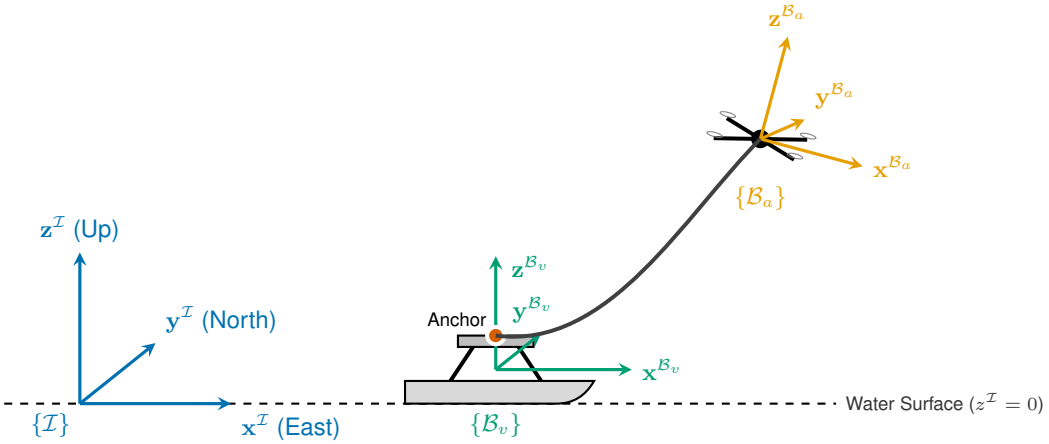


Figure 3.1: Reference coordinate frames $\{\mathcal{I}\}$, $\{\mathcal{B}_v\}$, and $\{\mathcal{B}_a\}$.

3.1.2 WAM-V Model

The mathematical model of the WAM-V catamaran, covering its horizontal manoeuvring dynamics and thruster allocation, forms a control-oriented state-space model suitable for trajectory tracking and predictive control.

Although a rigid body moving freely in the marine environment inherently has six degrees of freedom, the WAM-V is a USV that moves on the surface, which allows the motion to be simplified to the horizontal plane. Under this assumption, the model is reduced to three degrees of freedom.

Predicting the USV's motion, as needed for trajectory tracking and predictive control, requires knowing how its state changes over time for a given input. This relationship is captured by the non-linear state-space model

$$\dot{\mathbf{x}}_v = f(\mathbf{x}_v, \mathbf{u}_v), \tag{3.1}$$

The state vector $\mathbf{x}_v \in \mathbb{R}^6$ is given by

$$\mathbf{x}_v = [x, y, \psi, u, v, r]^\top, \tag{3.2}$$

where:

- x, y – position of the origin $\mathcal{O}_v$ of $\{\mathcal{B}_v\}$ relative to $\{\mathcal{I}\}$, expressed in $\{\mathcal{I}\}$;
- ψ – heading of $\{\mathcal{B}_v\}$ relative to $\{\mathcal{I}\}$;

- u, v – surge and sway linear velocities of $\{\mathcal{B}_v\}$ relative to $\{\mathcal{I}\}$, expressed in $\{\mathcal{B}_v\}$;
- r – angular velocity of $\{\mathcal{B}_v\}$ relative to $\{\mathcal{I}\}$, expressed in $\{\mathcal{B}_v\}$;

The control input vector $\mathbf{u}_v \in \mathbb{R}^3$ is given by

$$\mathbf{u}_v = [X, Y, N]^\top, \tag{3.3}$$

where:

- X – surge force generated by the propulsion system, applied to $\{\mathcal{B}_v\}$, expressed in $\{\mathcal{B}_v\}$;
- Y – sway force generated by the propulsion system, applied to $\{\mathcal{B}_v\}$, expressed in $\{\mathcal{B}_v\}$;
- N – yaw moment generated by the propulsion system, applied to $\{\mathcal{B}_v\}$, expressed in $\{\mathcal{B}_v\}$;

The vector field $f(\cdot)$ splits into a kinematic block and a dynamic block, derived in turn in the following subsections.

Kinematic Model

The kinematics describe how the USV's inertial position and heading evolve as a function of its body-fixed velocities, forming the first half of (3.1).

Let $\mathbf{p} = x\,\mathbf{x}^{\mathcal{I}} + y\,\mathbf{y}^{\mathcal{I}}$ denote the inertial position of the origin $\mathcal{O}_v$ of $\{\mathcal{B}_v\}$. Since $\mathbf{x}^{\mathcal{I}}, \mathbf{y}^{\mathcal{I}}$ are constant, differentiating yields

$$\dot{\mathbf{p}} = \dot{x}\,\mathbf{x}^{\mathcal{I}} + \dot{y}\,\mathbf{y}^{\mathcal{I}}, \tag{3.4}$$

i.e. $\dot{x}, \dot{y}$ are, by definition, the components of $\dot{\mathbf{p}}$ in the inertial frame.

The surge and sway velocities u, v are defined in the body-fixed frame, so that

$$\dot{\mathbf{p}} = u\,\mathbf{x}^{\mathcal{B}} + v\,\mathbf{y}^{\mathcal{B}}, \tag{3.5}$$

where $\mathbf{x}^{\mathcal{B}}, \mathbf{y}^{\mathcal{B}}$ denote the basis vectors of $\{\mathcal{B}_v\}$, i.e. $\mathbf{x}^{\mathcal{B}_v}, \mathbf{y}^{\mathcal{B}_v}$.

Since $\{\mathcal{B}_v\}$ is rotated relative to $\{\mathcal{I}\}$ by the heading angle ψ, the inertial unit vectors are related to the body-fixed ones by the rotation matrix $\mathbf{R}(\psi) \in SO(2)$,

$$\begin{bmatrix} \mathbf{x}^{\mathcal{I}} \\ \mathbf{y}^{\mathcal{I}} \end{bmatrix} = \mathbf{R}(\psi) \begin{bmatrix} \mathbf{x}^{\mathcal{B}} \\ \mathbf{y}^{\mathcal{B}} \end{bmatrix}, \tag{3.6}$$

following the standard planar rotation matrix,

$$\mathbf{R}(\psi) = \begin{bmatrix} \cos\psi & -\sin\psi \\ \sin\psi & \cos\psi \end{bmatrix}, \tag{3.7}$$

which is equivalent to

$$\mathbf{x}^{\mathcal{B}} = \cos\psi\,\mathbf{x}^{\mathcal{I}} + \sin\psi\,\mathbf{y}^{\mathcal{I}}, \qquad \mathbf{y}^{\mathcal{B}} = -\sin\psi\,\mathbf{x}^{\mathcal{I}} + \cos\psi\,\mathbf{y}^{\mathcal{I}}. \tag{3.8}$$

Substituting (3.8) into (3.5) and grouping terms gives

$$\dot{\mathbf{p}} = \underbrace{(u\cos\psi - v\sin\psi)}_{\dot{x}}\,\mathbf{x}^{\mathcal{I}} + \underbrace{(u\sin\psi + v\cos\psi)}_{\dot{y}}\,\mathbf{y}^{\mathcal{I}}. \tag{3.9}$$

Comparing with (3.4) then directly gives

$$\dot{x} = u\cos\psi - v\sin\psi, \qquad \dot{y} = u\sin\psi + v\cos\psi, \tag{3.10}$$

which is precisely

$$\begin{bmatrix}\dot{x}\\ \dot{y}\end{bmatrix} = \mathbf{R}(\psi)\begin{bmatrix}u\\ v\end{bmatrix}. \tag{3.11}$$

The heading rate follows directly from the definition of the angular velocity r:

$$\dot{\psi} = r. \tag{3.12}$$

Dynamic Model

The dynamics describe how the USV's body-fixed velocities evolve as a function of the applied forces and moments, forming the second half of (3.1).

Unlike $\mathbf{x}^{\mathcal{I}}, \mathbf{y}^{\mathcal{I}}$, the body-fixed unit vectors $\mathbf{x}^{\mathcal{B}}, \mathbf{y}^{\mathcal{B}}$ rotate with the vehicle. Differentiating (3.8) and using $\dot{\psi} = r$ gives

$$\dot{\mathbf{x}}^{\mathcal{B}} = r\,\mathbf{y}^{\mathcal{B}}, \qquad \dot{\mathbf{y}}^{\mathcal{B}} = -r\,\mathbf{x}^{\mathcal{B}}. \tag{3.13}$$

The inertial velocity, expressed in the body-fixed frame, is $\dot{\mathbf{p}} = u\,\mathbf{x}^{\mathcal{B}} + v\,\mathbf{y}^{\mathcal{B}}$. Differentiating once more and substituting (3.13) yields the inertial acceleration

$$\ddot{\mathbf{p}} = (\dot{u} - vr)\,\mathbf{x}^{\mathcal{B}} + (\dot{v} + ur)\,\mathbf{y}^{\mathcal{B}}, \tag{3.14}$$

where the cross terms $-vr$ and ur are Coriolis/centripetal terms arising because $\{\mathcal{B}_v\}$ is a rotating, non-inertial frame.

Let F_x, F_y denote the resultant surge and sway forces applied at $\mathcal{O}_v$, so that $\mathbf{F} = F_x\,\mathbf{x}^{\mathcal{B}} + F_y\,\mathbf{y}^{\mathcal{B}}$. By Newton's second law, $\mathbf{F} = m\,\ddot{\mathbf{p}}$, and comparing with (3.14),

$$F_x = m(\dot{u} - vr), \qquad F_y = m(\dot{v} + ur). \tag{3.15}$$

Since the USV's motion is planar, Euler's rotational equation about $\mathbf{z}^{\mathcal{B}}$ takes the form

$$M_z = I_z\,\dot{r}, \tag{3.16}$$

where M_z is the resultant yaw moment about $\mathcal{O}_v$ and I_z the yaw moment of inertia.

The resultant force and moment F_x, F_y, M_z combine two contributions: the thrust generated by the

propulsion system, X, Y, N, and the hydrodynamic damping opposing the USV's motion. This damping follows the standard linear-plus-quadratic model for USVs [22]. The quadratic surge term is further split into forward and backward coefficients, to capture the asymmetric drag between the two directions:

$$F_x = X - \big(X_u + X_{uu}^{\pm}|u|\big)u, \qquad X_{uu}^{\pm} = \begin{cases} X_{uu}^{+}, & u \geq 0 \\ X_{uu}^{-}, & u < 0 \end{cases} \tag{3.17}$$

$$F_y = Y - \big(Y_v + Y_{vv}|v|\big)v, \qquad M_z = N - \big(N_r + N_{rr}|r|\big)r. \tag{3.18}$$

Substituting (3.17)–(3.18) into (3.15) and (3.16), and solving for the accelerations, gives the dynamic block of (3.1):

$$\dot{u} = \frac{1}{m}\Big[X - \big(X_u + X_{uu}^{\pm}|u|\big)u\Big] + vr, \tag{3.19}$$

$$\dot{v} = \frac{1}{m}\Big[Y - \big(Y_v + Y_{vv}|v|\big)v\Big] - ur, \tag{3.20}$$

$$\dot{r} = \frac{1}{I_z}\Big[N - \big(N_r + N_{rr}|r|\big)r\Big]. \tag{3.21}$$

Thruster Allocation

The thruster allocation maps the physical actuator commands, the left and right thrust magnitudes T_L, T_R and azimuth angles α_L, α_R, to the resultant force and moment X, Y, N required by the dynamics of Section 3.1.2, according to the geometry shown in Figure 3.2.

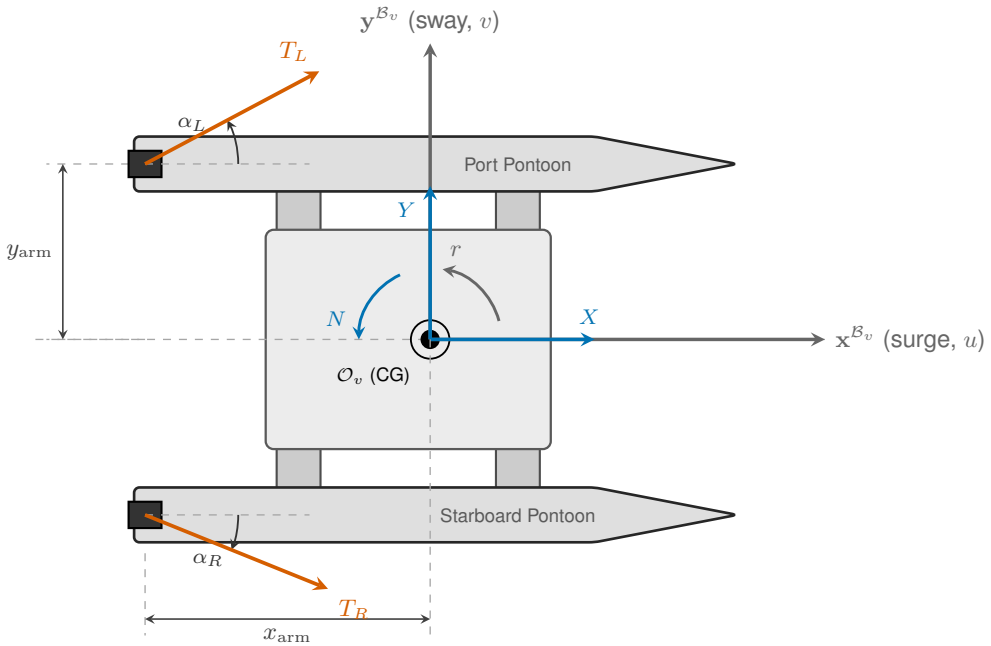


Figure 3.2: Planar USV schematic and thruster allocation geometry.

Each azimuth angle is measured relative to $\mathbf{x}^{\mathcal{B}}$, so that each thruster produces a planar force

$$\mathbf{f}_L = T_L \cos\alpha_L\, \mathbf{x}^{\mathcal{B}} + T_L \sin\alpha_L\, \mathbf{y}^{\mathcal{B}}, \qquad \mathbf{f}_R = T_R \cos\alpha_R\, \mathbf{x}^{\mathcal{B}} + T_R \sin\alpha_R\, \mathbf{y}^{\mathcal{B}}. \tag{3.22}$$

Summing the components of both thrusters along $\mathbf{x}^{\mathcal{B}}$ and $\mathbf{y}^{\mathcal{B}}$ gives the resultant force applied at $\mathcal{O}_v$:

$$X = T_L \cos\alpha_L + T_R \cos\alpha_R, \qquad Y = T_L \sin\alpha_L + T_R \sin\alpha_R. \tag{3.23}$$

Both thrusters are mounted behind $\mathcal{O}_v$, at the same longitudinal distance x_{arm}, and symmetrically on either side of the centerline, at a lateral distance y_{arm}. Expressed in $\{\mathcal{B}_v\}$, the left (port) thruster is located at $(-x_{\mathrm{arm}}, y_{\mathrm{arm}})$ and the right (starboard) thruster at $(-x_{\mathrm{arm}}, -y_{\mathrm{arm}})$.

The yaw moment produced by a planar force (f_x, f_y) applied at (r_x, r_y) is given by the two-dimensional cross product $r_x f_y - r_y f_x$. Applying this to each thruster and summing gives the resultant yaw moment about $\mathcal{O}_v$:

$$N = -x_{\mathrm{arm}}\big(T_L \sin\alpha_L + T_R \sin\alpha_R\big) - y_{\mathrm{arm}}\big(T_L \cos\alpha_L - T_R \cos\alpha_R\big). \tag{3.24}$$

Equations (3.23) and (3.24) complete the mapping from the four physical actuator commands $T_L, T_R, \alpha_L, \alpha_R$ to the control input $\mathbf{u}_v = [X, Y, N]^\top$ used by the dynamic model.

3.1.3 x500 UAV Model

The mathematical model of the x500 UAV, covering its translational dynamics in three-dimensional space, forms a control-oriented state-space model suitable for trajectory tracking and predictive control. The vehicle's attitude is stabilised by an inner-loop attitude controller, to which the attitude used in the translational dynamics is commanded.

Predicting the vehicle's motion, as needed for trajectory tracking and predictive control, requires knowing how its state changes over time for a given input. This relationship is captured by the non-linear state-space model

$$\dot{\mathbf{x}}_a = f(\mathbf{x}_a, \mathbf{u}_a), \tag{3.25}$$

The state vector $\mathbf{x}_a \in \mathbb{R}^6$ is given by

$$\mathbf{x}_a = [x, y, z, \dot{x}, \dot{y}, \dot{z}]^\top, \tag{3.26}$$

where:

- x, y, z – position of the origin $\mathcal{O}_a$ of $\{\mathcal{B}_a\}$ relative to $\{\mathcal{I}\}$, expressed in $\{\mathcal{I}\}$;
- $\dot{x}, \dot{y}, \dot{z}$ – linear velocity of $\mathcal{O}_a$ relative to $\{\mathcal{I}\}$, expressed in $\{\mathcal{I}\}$;

The control input vector $\mathbf{u}_a \in \mathbb{R}^4$ is given by

$$\mathbf{u}_a = [T, \phi, \theta, \psi]^\top, \tag{3.27}$$

where:

- T – total thrust force, applied to $\{\mathcal{B}_a\}$, expressed in $\{\mathcal{B}_a\}$;
- ϕ, θ, ψ – roll, pitch, and yaw angles of $\{\mathcal{B}_a\}$ relative to $\{\mathcal{I}\}$;

The vector field $f(\cdot)$ splits into a kinematic block, relating the inertial position and velocity, and a dynamic block, mapping the thrust, attitude, aerodynamic drag, and tether force to the inertial translational acceleration.

Kinematic Model

The kinematics describe how the vehicle's inertial position evolves as a function of its inertial velocity, forming the first half of (3.25).

Let $\mathbf{p} = x\,\mathbf{x}^{\mathcal{I}} + y\,\mathbf{y}^{\mathcal{I}} + z\,\mathbf{z}^{\mathcal{I}}$ denote the inertial position of the origin $\mathcal{O}_a$ of $\{\mathcal{B}_a\}$. Since $\mathbf{x}^{\mathcal{I}}, \mathbf{y}^{\mathcal{I}}, \mathbf{z}^{\mathcal{I}}$ are constant, differentiating yields

$$\dot{\mathbf{p}} = \dot{x}\,\mathbf{x}^{\mathcal{I}} + \dot{y}\,\mathbf{y}^{\mathcal{I}} + \dot{z}\,\mathbf{z}^{\mathcal{I}}, \tag{3.28}$$

i.e. $\dot{x}, \dot{y}, \dot{z}$ are, by definition, the components of $\dot{\mathbf{p}}$ in the inertial frame, which are precisely the linear velocity components in $\mathbf{x}_a$. The kinematic block of (3.25) is therefore the trivial identity

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{z} \end{bmatrix} = \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{z} \end{bmatrix}. \tag{3.29}$$

This is in contrast to a USV such as the WAM-V (Section 3.1.2), where the linear velocities are instead expressed in $\{\mathcal{B}_v\}$, requiring an additional change of basis via $\mathbf{R}(\psi)$ to relate them to $\dot{\mathbf{p}}$. Here, this rotation is required instead to express the thrust force, applied along $\mathbf{z}^{\mathcal{B}_a}$, in $\{\mathcal{I}\}$. It is therefore deferred to the dynamic block, derived next.

Dynamic Model

The dynamics describe how the vehicle's inertial velocity evolves as a function of the applied thrust, attitude, aerodynamic drag, and tether force, forming the second half of (3.25).

The thrust generated by the propulsion system acts along the vertical axis of $\{\mathcal{B}_a\}$, so that the resultant thrust force is

$$\mathbf{F}_{\text{thr}} = T\,\mathbf{z}^{\mathcal{B}}, \tag{3.30}$$

where $\mathbf{z}^{\mathcal{B}}$ denotes the vertical basis vector of $\{\mathcal{B}_a\}$, i.e. $\mathbf{z}^{\mathcal{B}_a}$.

Since (3.29) relates the inertial position and velocity directly, without an intermediate change of basis, Newton's second law is more conveniently applied directly in $\{\mathcal{I}\}$, requiring $\mathbf{F}_{\text{thr}}$ to instead be expressed in the inertial frame.

Since $\{\mathcal{B}_a\}$ is rotated relative to $\{\mathcal{I}\}$ by the roll, pitch, and yaw angles ϕ, θ, ψ, the inertial unit vectors are related to the body-fixed ones by the rotation matrix $\mathbf{R}(\phi, \theta, \psi) \in SO(3)$,

$$\begin{bmatrix} \mathbf{x}^{\mathcal{I}} \\ \mathbf{y}^{\mathcal{I}} \\ \mathbf{z}^{\mathcal{I}} \end{bmatrix} = \mathbf{R}(\phi, \theta, \psi) \begin{bmatrix} \mathbf{x}^{\mathcal{B}} \\ \mathbf{y}^{\mathcal{B}} \\ \mathbf{z}^{\mathcal{B}} \end{bmatrix}, \tag{3.31}$$

following the ZYX (yaw-pitch-roll) Euler angle convention,

$$\mathbf{R}(\phi,\theta,\psi) = \begin{bmatrix} c_\theta c_\psi & s_\phi s_\theta c_\psi - c_\phi s_\psi & c_\phi s_\theta c_\psi + s_\phi s_\psi \\ c_\theta s_\psi & s_\phi s_\theta s_\psi + c_\phi c_\psi & c_\phi s_\theta s_\psi - s_\phi c_\psi \\ -s_\theta & s_\phi c_\theta & c_\phi c_\theta \end{bmatrix}, \tag{3.32}$$

where $s_\bullet = \sin(\bullet)$ and $c_\bullet = \cos(\bullet)$.

Projecting $\mathbf{F}_{\text{thr}}$ onto $\{\mathcal{I}\}$ then gives

$$\mathbf{F}^{\mathcal{I}}_{\text{thr}} = \mathbf{R}(\phi,\theta,\psi) \begin{bmatrix} 0 \\ 0 \\ T \end{bmatrix}. \tag{3.33}$$

Figure 3.3 illustrates this projection in the longitudinal plane, showing how the body tilt θ splits the thrust force T into vertical and horizontal inertial components.

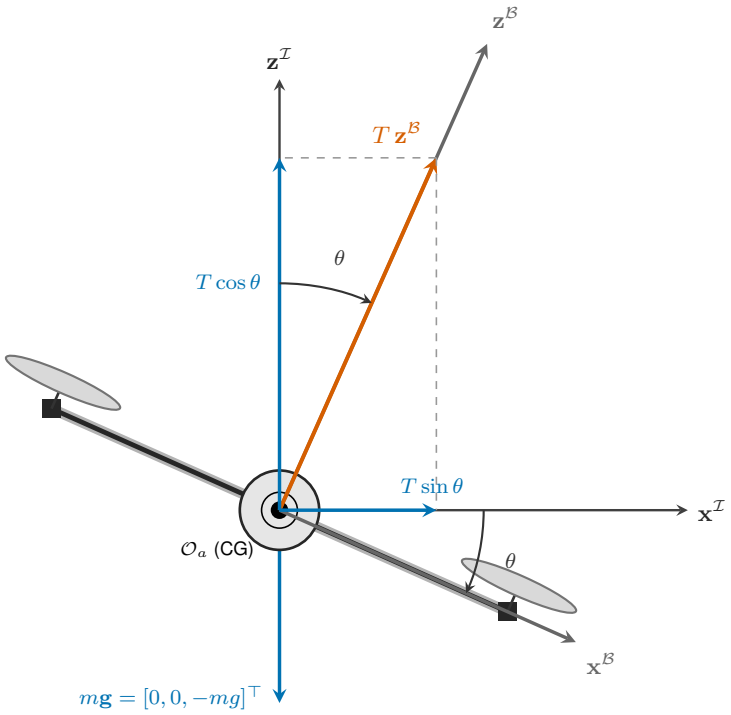


Figure 3.3: UAV thrust vectoring in the longitudinal plane.

Gravity, unlike the thrust, is already naturally expressed in $\{\mathcal{I}\}$:

$$\mathbf{F}^{\mathcal{I}}_{\text{grav}} = \begin{bmatrix} 0 \\ 0 \\ -mg \end{bmatrix}. \tag{3.34}$$

During translational flight, the vehicle is also subject to aerodynamic drag opposing its planar motion, arising primarily from the relative airflow across the spinning rotor disks (rotor drag). This force is natu-

rally expressed as a linear damping of the velocity in $\{B_a\}$, with coefficient d_r along the body axes x^B and y^B. For small roll and pitch angles, the horizontal axes of the body remain approximately horizontal in $\{I\}$, so the drag can be written directly in the inertial frame as

$$\mathbf{F}^{\mathcal{I}}_{\text{drag}} = \begin{bmatrix} -d_r\,\dot{x} \\ -d_r\,\dot{y} \\ 0 \end{bmatrix}, \tag{3.35}$$

where the small coupling between horizontal and vertical velocities caused by the tilt of the vehicle is neglected.

In addition to thrust, gravity, and aerodynamic drag, the vehicle is subject to a force transmitted through a tether attached at the origin $\mathcal{O}_a$ of $\{\mathcal{B}_a\}$, denoted $\mathbf{F}^{\mathcal{I}}_{\text{tether}}$ and derived separately in Section 3.1.3.

Since $\{\mathcal{I}\}$ is non-rotating, Newton's second law applies directly to the inertial acceleration, without the transport terms that arise when applying it in a rotating frame, such as $\{\mathcal{B}_v\}$ in the WAM-V model (Section 3.1.2):

$$m\,\ddot{\mathbf{p}} = \mathbf{F}^{\mathcal{I}}_{\text{thr}} + \mathbf{F}^{\mathcal{I}}_{\text{grav}} + \mathbf{F}^{\mathcal{I}}_{\text{drag}} + \mathbf{F}^{\mathcal{I}}_{\text{tether}}. \tag{3.36}$$

Substituting (3.33), (3.34), and (3.35) into (3.36), and solving for the acceleration, gives the dynamic block of (3.25):

$$\begin{bmatrix} \ddot{x} \\ \ddot{y} \\ \ddot{z} \end{bmatrix} = \frac{T}{m}\,\mathbf{R}(\phi,\theta,\psi) \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ -g \end{bmatrix} - \frac{d_r}{m} \begin{bmatrix} \dot{x} \\ \dot{y} \\ 0 \end{bmatrix} + \frac{1}{m}\mathbf{F}^{\mathcal{I}}_{\text{tether}}. \tag{3.37}$$

Tether Force Model

In addition to thrust and gravity, the vehicle is subject to a force transmitted through a tether connecting the origin $\mathcal{O}_a$ of $\{\mathcal{B}_a\}$ to a fixed attachment point on the WAM-V, whose position $\mathbf{p}_v$ is treated here only as a known geometric anchor, without modelling the (negligible) reaction force exerted on the USV.

The relevant geometric quantity is the chord, the straight-line vector between the two attachment points,

$$\mathbf{r} = \mathbf{p}_v - \mathbf{p}, \qquad d = \|\mathbf{r}\|, \qquad \hat{\mathbf{d}}_0 = \frac{\mathbf{r}}{d}, \tag{3.38}$$

where d is the length of the chord, i.e. the straight-line distance between the UAV and the USV, and $\hat{\mathbf{d}}_0$ is its direction, which the tether would have if taut.

Denoting by $L \geq d$ the tether length at a given instant, its slack is conveniently quantified by the dimensionless ratio

$$\varepsilon = \frac{L-d}{L}, \tag{3.39}$$

with $\varepsilon = 0$ corresponding to a taut tether, and ε increasing as more slack is introduced.

Under its own weight, a tether of linear mass density μ suspended between two attachment points takes the shape of a catenary, contained in the vertical plane defined by the two endpoints. Due to the

geometric self-similarity of the catenary curve at a given slack ratio ε and chord inclination, the tension magnitude scales proportionally with the length d of the chord (or with the unstretched length L), while the tension direction is governed by the angular deflection γ by which the tension vector at $\mathcal{O}_a$ deviates from $\hat{\mathbf{d}}_0$ towards the vertical.

The exact relations governing the deflection angle $\gamma_{\text{exact}}(\varepsilon, \alpha)$ and the normalised tension coefficient $c_d^{\text{exact}}(\varepsilon, \alpha) = T_{\text{tether}}/d$ do not admit a closed-form analytical solution, requiring numerical root-finding algorithms at every control cycle.

The tether control scheme considered in this work maintains ε close to a constant operating point ε_0. Under this operational regime, the tension magnitude is approximated as scaling linearly with the length of the chord d:

$$T_{\text{tether}}(d) = c_d(\varepsilon_0)\, d, \tag{3.40}$$

where the nominal coefficient at ε_0 is obtained by approximating the tension at the UAV by the weight of the tether, $T_{\text{tether}}(L) \approx \mu\, g\, L$, with $L = d/(1-\varepsilon_0)$:

$$c_d(\varepsilon_0) = \frac{\mu\, g}{1-\varepsilon_0}. \tag{3.41}$$

This neglects the dependence of the tension on the inclination of the chord, which for the operating slack changes it by about 12% at most. The deflection angle also depends on the inclination of the chord, given by the depression angle α of $\hat{\mathbf{d}}_0$ below the local horizontal, defined in (3.44). Only the component of the weight of the tether normal to the chord, $\mu\, g\, L \cos\alpha$, bends it away from the straight line, so that, for the small slack of the operating point, the deflection approximately scales with $\cos\alpha$. It is maximal for a horizontal chord and vanishes for a vertical one, where the tether hangs straight. The deflection angle is therefore given by its nominal value at ε_0 for a horizontal chord, $\gamma_0 = \gamma(\varepsilon_0, 0)$, scaled by $\cos\alpha$:

$$\gamma(\alpha) = \gamma_0 \cos\alpha, \tag{3.42}$$

which replaces the transcendental relation with a closed-form expression evaluated at ε_0.

With γ given by (3.42), the tension direction $\hat{\mathbf{d}}$ can now be constructed. This requires first introducing the direction $\hat{\mathbf{e}}_h$, orthogonal to $\mathbf{z}^{\mathcal{I}}$ within the catenary plane,

$$\mathbf{h} = \hat{\mathbf{d}}_0 - (\hat{\mathbf{d}}_0 \cdot \mathbf{z}^{\mathcal{I}})\, \mathbf{z}^{\mathcal{I}}, \qquad \hat{\mathbf{e}}_h = \frac{\mathbf{h}}{\sqrt{\|\mathbf{h}\|^2 + \delta^2}}, \tag{3.43}$$

where $\delta > 0$ is a small constant that keeps $\hat{\mathbf{e}}_h$ defined and differentiable when the chord is vertical and $\mathbf{h} = \mathbf{0}$, and has a negligible effect otherwise. The depression angle α of $\hat{\mathbf{d}}_0$ below $\hat{\mathbf{e}}_h$ follows from

$$\hat{\mathbf{d}}_0 = \cos\alpha\, \hat{\mathbf{e}}_h - \sin\alpha\, \mathbf{z}^{\mathcal{I}}. \tag{3.44}$$

Figure 3.4 illustrates this construction within the catenary plane, showing the local horizontal $\hat{\mathbf{e}}_h$, the depression angle α, and the resulting tension direction $\hat{\mathbf{d}}$, tilted further from $\hat{\mathbf{d}}_0$ by the angle γ.

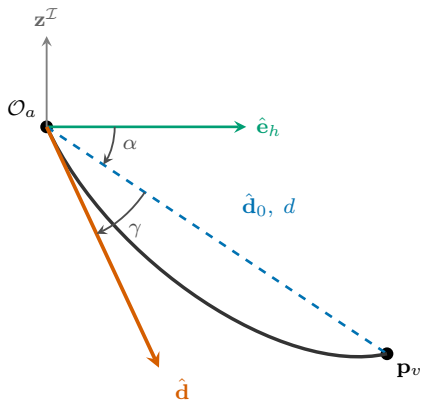


Figure 3.4: Tether catenary plane.

The tension direction at $\mathcal{O}_a$ then follows by tilting $\hat{\mathbf{d}}_0$ further towards the vertical by the angle $\gamma(\alpha)$, within the same plane,

$$\hat{\mathbf{d}} = \cos(\alpha+\gamma)\,\hat{\mathbf{e}}_h - \sin(\alpha+\gamma)\,\mathbf{z}^{\mathcal{I}}, \tag{3.45}$$

and the resulting tether force, applied at $\mathcal{O}_a$ and expressed in $\{\mathcal{I}\}$, is

$$\mathbf{F}^{\mathcal{I}}_{\text{tether}} = T_{\text{tether}}(d)\,\hat{\mathbf{d}} = c_d(\varepsilon_0)\,d\,\hat{\mathbf{d}}, \tag{3.46}$$

with d evaluated at each instant from (3.38). Equation (3.46) is the term $\mathbf{F}^{\mathcal{I}}_{\text{tether}}$ introduced in (3.36). For a vertical chord, $\mathbf{h} = \mathbf{0}$ and so $\hat{\mathbf{e}}_h = \mathbf{0}$, and since $\cos(\alpha+\gamma) = 0$ the direction reduces to $-\mathbf{z}^{\mathcal{I}}$.

3.2 System Control

Having established, in Chapter 3.1, the kinematic and dynamic models governing the USV and UAV, the natural next step is to determine how these models can be exploited to compute the control inputs that drive each vehicle along a desired trajectory. Because the two vehicles are physically coupled through the tether, their motion cannot be planned independently without accounting for this interaction, motivating a cooperative control strategy.

The overall control approach is therefore established first, followed by the design of the controllers for each vehicle.

3.2.1 Cooperative Control Design

Before formulating either controller, the strategies available are weighed against the system's dynamics and its objectives, so that the method chosen suits both vehicles.

Control Method

Keeping the tether within safe operating conditions depends on how the distance between the two vehicles evolves over time. A controller that only reacts to the current state can correct a violation only

after it happens. Anticipating where the system is headed allows corrective action before reaching the safety limits.

A predictive controller does this by repeatedly solving, at each control instant, an optimisation problem over a finite look-ahead window. Because the optimisation runs over the predicted trajectory rather than the current state alone, requirements such as keeping the tether within safe conditions can be stated as explicit constraints in the problem itself, enforcing them explicitly over the prediction horizon.

The predictions rely on the model of the system dynamics, so their accuracy is tied to how well that model represents the real behaviour. The USV and UAV models derived in Chapter 3.1 include non-linear terms. The predictive controller uses these non-linear models directly, making NMPC the method used for both vehicles.

Cooperative Architecture

The USV and the UAV operate on different timescales. The USV moves in the horizontal plane with slow, inertia-dominated motion. The UAV's dynamics evolve much faster and demand a finer sampling time. A single NMPC built around the combined state of both vehicles would force one prediction horizon and one sampling time onto both, compromising one vehicle's timescale to accommodate the other.

Combining both vehicles into one NMPC would also enlarge the state, input, and constraint sets a single solver must handle, making a problem that already runs under real-time constraints harder to solve within the available computation time. Keeping the two problems separate matches each optimisation's size to its own vehicle and keeps the computation more manageable in real time.

Each vehicle is therefore controlled by its own NMPC. This separation isolates the two controllers from each other's failures: if one solver fails to find a feasible solution, the other vehicle's NMPC continues operating on its own well-posed problem.

The tether's safety limits depend on the relative position between the USV and the UAV, so each vehicle's NMPC accounts for the other vehicle's position. At every control instant, each controller receives the other vehicle's most recently computed predicted trajectory, and uses it to compute the predicted inter-vehicle distance at the nodes of its own horizon covered by that trajectory. Because the two controllers operate at different sampling intervals, the received predicted trajectory is temporally interpolated and resampled at the receiver's own prediction nodes.

Without this exchange, a vehicle would have to treat the other as stationary or extrapolate its motion from the last known state. Either assumption misrepresents the tether coupling and can drive the system into a constraint violation.

Because the UAV operates with a finer sampling time and a possibly shorter prediction horizon than the USV, its predicted trajectory may cover only part of the USV's horizon. The USV therefore enforces the tether constraint only at the nodes covered by the received trajectory, and leaves it inactive at the remaining ones, where no information about the UAV is available. This avoids constraining the USV with an assumed UAV position that has no basis in the prediction. Since the problem is solved again at every instant, the covered window slides forward with time, and any potential violation is accounted for as soon as it enters it. In the opposite direction no such restriction is needed, since the USV's horizon

fully covers that of the UAV.

Figure 3.5 shows this architecture, with two independent NMPC loops, each driving its own vehicle, exchanging predicted horizons to enforce the tether constraint.

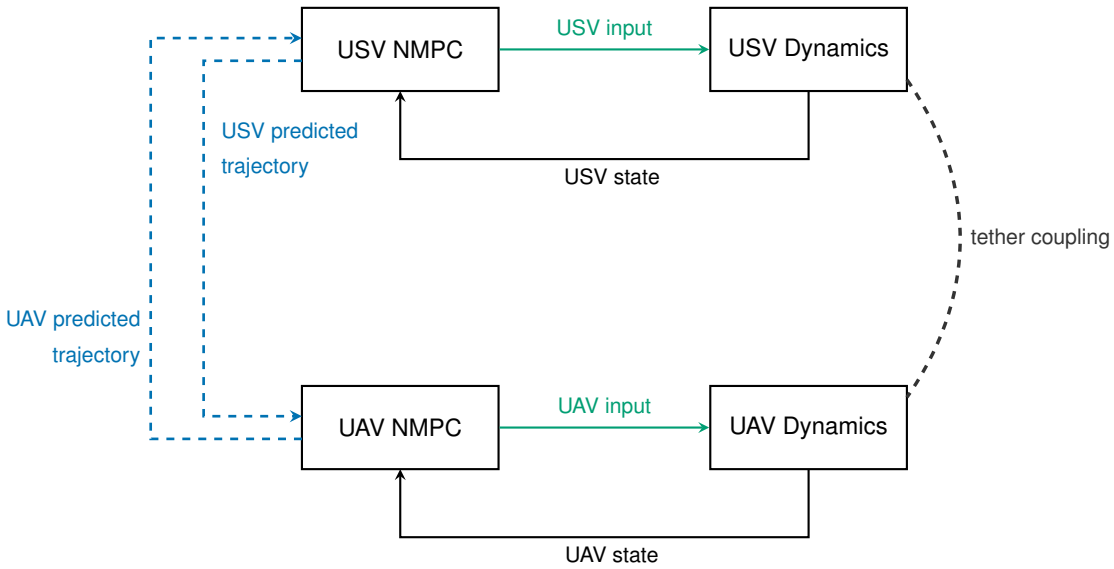


Figure 3.5: Cooperative control architecture diagram.

3.2.2 Non-linear Model Predictive Control

The vehicle models of Chapter 3.1 are continuous-time systems of the form

$$\dot{\mathbf{x}} = \mathbf{f}\left(\mathbf{x}, \mathbf{u}\right), \tag{3.47}$$

where $\mathbf{x} \in \mathbb{R}^{n_x}$ is the state vector and $\mathbf{u} \in \mathbb{R}^{n_u}$ is the control input vector.

However, the controller acts at discrete sampling instants $t_k = kT_s$, with the input held constant over each interval $[t_k, t_{k+1})$. Under this assumption, the state at the next instant follows from integrating the dynamics over one sampling interval,

$$\mathbf{x}(t_{k+1}) = \mathbf{x}(t_k) + \int_{t_k}^{t_{k+1}} \mathbf{f}\left(\mathbf{x}(\tau), \mathbf{u}_k\right) d\tau. \tag{3.48}$$

Since this integral generally has no closed-form solution for non-linear dynamics, it is approximated by applying a numerical integration scheme to $\mathbf{f}$ over one sampling interval. This defines a one-step discrete map $\mathbf{\Phi}$,

$$\mathbf{x}_{k+1} = \mathbf{\Phi}\left(\mathbf{x}_k, \mathbf{u}_k\right), \tag{3.49}$$

where $\mathbf{x}_k \approx \mathbf{x}(t_k)$ is the system state at instant k, so that $\mathbf{\Phi}$ returns the approximate state one sampling interval ahead, given the current state and input.

At every instant k, NMPC uses this map to predict the future behaviour of the system over a finite look-ahead window of N steps, and computes the input that steers it toward a reference. Denote by $\hat{\mathbf{x}}_{k+i|k}$ and $\hat{\mathbf{u}}_{k+i|k}$ the state and input predicted i steps ahead at instant k, for $i = 0, \ldots, N$ and $i = 0, \ldots, N-1$,

respectively. The predicted trajectory starts from the estimated current state,

$$\hat{\mathbf{x}}_{k|k} = \mathbf{x}_k, \tag{3.50}$$

and evolves according to the system model,

$$\hat{\mathbf{x}}_{k+i+1|k} = \mathbf{\Phi}\left(\hat{\mathbf{x}}_{k+i|k}, \hat{\mathbf{u}}_{k+i|k}\right), \quad i = 0, \ldots, N-1. \tag{3.51}$$

Each predicted step is then scored against a reference trajectory. A stage cost $\ell(\hat{\mathbf{x}}_{k+i|k}, \hat{\mathbf{u}}_{k+i|k})$, typically quadratic, penalises the tracking error and the control effort at every step of the horizon, and a terminal cost $\ell_f(\hat{\mathbf{x}}_{k+N|k})$ penalises the tracking error at its final step. Summing both over the horizon gives the total predicted cost,

$$J = \sum_{i=0}^{N-1} \ell\left(\hat{\mathbf{x}}_{k+i|k}, \hat{\mathbf{u}}_{k+i|k}\right) + \ell_f\left(\hat{\mathbf{x}}_{k+N|k}\right). \tag{3.52}$$

Minimising J alone ignores the physical limits of the system, so the prediction is also constrained. The predicted states and inputs must remain within the admissible sets $\mathcal{X}$ and $\mathcal{U}$ at every step of the horizon,

$$\hat{\mathbf{x}}_{k+i|k} \in \mathcal{X}, \quad i = 0, \ldots, N, \tag{3.53}$$

$$\hat{\mathbf{u}}_{k+i|k} \in \mathcal{U}, \quad i = 0, \ldots, N-1, \tag{3.54}$$

where $\mathcal{X}$ and $\mathcal{U}$ collect the admissible states and inputs. These sets may be defined by simple bounds or by general non-linear inequality constraints of the form $h(\hat{\mathbf{x}}_{k+i|k}, \hat{\mathbf{u}}_{k+i|k}) \leq 0$.

Enforcing these sets as hard constraints can make the problem infeasible. If a disturbance pushes the true state outside the admissible set, no predicted trajectory satisfies the constraints, and the solver has no solution to return even though one is required at every sampling instant. To avoid this, a constraint $h(\hat{\mathbf{x}}_{k+i|k}, \hat{\mathbf{u}}_{k+i|k}) \leq 0$ can be softened by a slack variable $\sigma \geq 0$,

$$h(\hat{\mathbf{x}}_{k+i|k}, \hat{\mathbf{u}}_{k+i|k}) \leq \sigma, \tag{3.55}$$

which allows the constraint to be violated by an amount σ. The slack becomes an additional decision variable at each step where the constraint is softened, and a penalty

$$\rho(\sigma) = \mu_1 \sigma + \mu_2 \sigma^2, \qquad \mu_1, \mu_2 \geq 0, \tag{3.56}$$

is added to the cost J for each slack. Since $\rho(0) = 0$, the solver prefers $\sigma = 0$ whenever the original constraint can be met, and large weights μ_1, μ_2 keep any violation close to zero.

Assembling the cost, the prediction model, and the constraints defines the optimal control problem

solved at every sampling instant k,

$$\min_{\hat{\mathbf{x}}_{k+i|k},\, \hat{\mathbf{u}}_{k+i|k},\, \hat{\sigma}_{k+i|k}} \quad J + \sum_{i=0}^{N-1} \rho\left(\hat{\sigma}_{k+i|k}\right) \tag{3.57}$$

$$\begin{aligned}
\text{s.t.} \quad & \hat{\mathbf{x}}_{k|k} = \mathbf{x}_k, \\
& \hat{\mathbf{x}}_{k+i+1|k} = \boldsymbol{\Phi}\left(\hat{\mathbf{x}}_{k+i|k}, \hat{\mathbf{u}}_{k+i|k}\right), \quad i = 0, \ldots, N-1, \\
& \hat{\mathbf{x}}_{k+i|k} \in \mathcal{X}, \quad i = 0, \ldots, N, \\
& \hat{\mathbf{u}}_{k+i|k} \in \mathcal{U}, \quad i = 0, \ldots, N-1, \\
& h\left(\hat{\mathbf{x}}_{k+i|k}, \hat{\mathbf{u}}_{k+i|k}\right) \leq \hat{\sigma}_{k+i|k}, \quad \hat{\sigma}_{k+i|k} \geq 0, \quad i = 0, \ldots, N-1.
\end{aligned}$$

Solving this problem returns a full predicted input sequence, $\hat{\mathbf{u}}_{k|k}, \ldots, \hat{\mathbf{u}}_{k+N-1|k}$, but only its first element, $\hat{\mathbf{u}}_{k|k}$, is applied to the system. At the next instant, $k+1$, the state is estimated again and the problem is solved again from $\hat{\mathbf{x}}_{k+1|k+1} = \mathbf{x}_{k+1}$, with the horizon shifted one step forward. This strategy, known as the receding horizon principle, computes every input from the latest state estimate, so the controller can correct for disturbances and model errors that the previous prediction did not anticipate.

3.2.3 USV NMPC

The USV NMPC computes, at every sampling instant, the thruster commands that best track a reference trajectory over a finite prediction horizon, subject to the USV's dynamics, the physical limits of its actuators, and the tether workspace. It predicts the USV's motion with the model derived in Chapter 3.1, and solves a constrained optimisation problem at each step to obtain the commands applied to the thrusters.

Predictive Model

The USV model of Chapter 3.1 is driven by the resultant thrust force and moment $\boldsymbol{\tau} = [X, Y, N]^\top$, which are obtained from the physical actuator commands $T_L, T_R, \alpha_L, \alpha_R$ through the allocation

$$X = T_L \cos\alpha_L + T_R \cos\alpha_R, \qquad Y = T_L \sin\alpha_L + T_R \sin\alpha_R, \tag{3.58}$$

$$N = -x_{arm}\big(T_L \sin\alpha_L + T_R \sin\alpha_R\big) - y_{arm}\big(T_L \cos\alpha_L - T_R \cos\alpha_R\big). \tag{3.59}$$

Since the controller ultimately has to command $T_L, T_R, \alpha_L, \alpha_R$, the NMPC is formulated directly in terms of these variables, rather than choosing $\boldsymbol{\tau}$ and allocating it separately. This way, the limits of the physical actuators can be imposed directly in the optimisation problem.

Each thruster is limited both in the thrust and azimuth it can deliver,

$$T_{L,min} \leq T_L \leq T_{L,max}, \quad T_{R,min} \leq T_R \leq T_{R,max}, \quad \alpha_{L,min} \leq \alpha_L \leq \alpha_{L,max}, \quad \alpha_{R,min} \leq \alpha_R \leq \alpha_{R,max}, \tag{3.60}$$

and in how fast it can change them,

$$|\dot{T}_L| \le \dot{T}_{L,max}, \quad |\dot{T}_R| \le \dot{T}_{R,max}, \quad |\dot{\alpha}_L| \le \dot{\alpha}_{L,max}, \quad |\dot{\alpha}_R| \le \dot{\alpha}_{R,max}. \tag{3.61}$$

The rate limits act on the time derivatives of the commands, so imposing them directly as bounds requires these derivatives to become the control input,

$$\mathbf{u}_v = [\dot{T}_L, \dot{T}_R, \dot{\alpha}_L, \dot{\alpha}_R]^\top \in \mathbb{R}^4, \tag{3.62}$$

which in turn makes $T_L, T_R, \alpha_L, \alpha_R$ part of the state vector,

$$\mathbf{x}_v = [x, y, \psi, u, v, r,\ T_L, T_R, \alpha_L, \alpha_R]^\top \in \mathbb{R}^{10}. \tag{3.63}$$

With this choice, the range limits act on the state, and the rate limits act directly on the input.

With $\mathbf{x}_v$ and $\mathbf{u}_v$ defined, the prediction model $\dot{\mathbf{x}}_v = \mathbf{f}_v(\mathbf{x}_v, \mathbf{u}_v)$ is assembled from the USV kinematics and dynamics.

The kinematics are given by (3.11) and (3.12),

$$\dot{x} = u\cos\psi - v\sin\psi, \qquad \dot{y} = u\sin\psi + v\cos\psi, \qquad \dot{\psi} = r. \tag{3.64}$$

The dynamics follow (3.19)–(3.21), with X, Y, N given by (3.58)–(3.59) in terms of the thruster state,

$$\dot{u} = \frac{1}{m}\Big[X - \big(X_u + X^{\pm}_{uu}|u|\big)u\Big] + vr, \tag{3.65}$$

$$\dot{v} = \frac{1}{m}\Big[Y - \big(Y_v + Y_{vv}|v|\big)v\Big] - ur, \tag{3.66}$$

$$\dot{r} = \frac{1}{I_z}\Big[N - \big(N_r + N_{rr}|r|\big)r\Big]. \tag{3.67}$$

Finally, $\dot{T}_L, \dot{T}_R, \dot{\alpha}_L, \dot{\alpha}_R$ appear directly as the components of $\mathbf{u}_v$,

$$\dot{T}_L = \dot{T}_L, \qquad \dot{T}_R = \dot{T}_R, \qquad \dot{\alpha}_L = \dot{\alpha}_L, \qquad \dot{\alpha}_R = \dot{\alpha}_R. \tag{3.68}$$

Discretisation over a sampling interval T_s, following Section 3.2.2, gives the one-step map $\mathbf{\Phi}_v$ such that $\hat{\mathbf{x}}_{v,k+i+1|k} = \mathbf{\Phi}_v\big(\hat{\mathbf{x}}_{v,k+i|k}, \hat{\mathbf{u}}_{v,k+i|k}\big)$ approximates the solution of $\dot{\mathbf{x}}_v = \mathbf{f}_v(\mathbf{x}_v, \mathbf{u}_v)$ over one sampling interval.

Cost and Constraints

Let $\mathbf{p} = [x, y]^\top$ denote the position, ψ the heading, and $\boldsymbol{\nu} = [u, v, r]^\top$ the body-fixed velocities, with $\mathbf{p}_{ref}$, ψ_{ref}, and $\boldsymbol{\nu}_{ref}$ the corresponding references from a given trajectory.

Tracking the reference position is penalised quadratically as

$$\|\mathbf{p} - \mathbf{p}_{ref}\|^2_{Q_p}, \tag{3.69}$$

where $\|\boldsymbol{\xi}\|^2_W \coloneqq \boldsymbol{\xi}^\top W \boldsymbol{\xi}$ for a positive semi-definite weighting matrix W, which sets the relative importance of each component of $\boldsymbol{\xi}$ in the cost.

Tracking the heading reference is penalised as

$$q_\psi\big(1 - \cos(\psi - \psi_{ref})\big), \tag{3.70}$$

where $q_\psi \geq 0$ is a scalar weight setting the relative importance of heading tracking.

Both ψ and ψ_{ref} are angles represented within an interval of width 2π, such as $(-\pi, \pi]$. When they lie near opposite ends of this interval, their raw difference $\psi - \psi_{ref}$ approaches $\pm 2\pi$, even though the two headings are then almost identical. As $\cos(\cdot)$ has period 2π, its value approaches one whenever $\psi - \psi_{ref}$ is near zero or $\pm 2\pi$, so the cost (3.70) is near zero in both cases, correctly reflecting that the two headings are almost identical.

Tracking the reference velocity is penalised in the same way as the position,

$$\|\boldsymbol{\nu} - \boldsymbol{\nu}_{ref}\|^2_{Q_\nu}. \tag{3.71}$$

The thruster commands $T_L, T_R, \alpha_L, \alpha_R$ are regularised directly as

$$\big\|[T_L, T_R, \alpha_L, \alpha_R]^\top\big\|^2_{Q_\tau}, \tag{3.72}$$

discouraging thrust and azimuth commands larger than necessary to track the reference.

The last term penalises $\mathbf{u}_v$, the rate of change of the thruster commands,

$$\big\|[\dot{T}_L, \dot{T}_R, \dot{\alpha}_L, \dot{\alpha}_R]^\top\big\|^2_{R}, \tag{3.73}$$

discouraging abrupt changes in thrust and azimuth between consecutive instants.

Summing (3.69)–(3.73) gives the stage cost

$$\begin{aligned}\ell(\mathbf{x}_v, \mathbf{u}_v) = \|\mathbf{p} - \mathbf{p}_{ref}\|^2_{Q_p} + q_\psi\big(1 - \cos(\psi - \psi_{ref})\big) + \|\boldsymbol{\nu} - \boldsymbol{\nu}_{ref}\|^2_{Q_\nu} \\ + \big\|[T_L, T_R, \alpha_L, \alpha_R]^\top\big\|^2_{Q_\tau} + \big\|[\dot{T}_L, \dot{T}_R, \dot{\alpha}_L, \dot{\alpha}_R]^\top\big\|^2_{R}.\end{aligned} \tag{3.74}$$

The terminal cost ℓ_f penalises the last predicted state, so that the optimiser does not disregard where the USV ends up at the end of the horizon. It equals the stage cost (3.74) without the input rate term, since no control input is chosen at the final node:

$$\ell_f(\mathbf{x}_v) = \|\mathbf{p} - \mathbf{p}_{ref}\|^2_{Q_p} + q_\psi\big(1 - \cos(\psi - \psi_{ref})\big) + \|\boldsymbol{\nu} - \boldsymbol{\nu}_{ref}\|^2_{Q_\nu} + \big\|[T_L, T_R, \alpha_L, \alpha_R]^\top\big\|^2_{Q_\tau}. \tag{3.75}$$

The thruster limits introduced in the predictive model enter the optimisation problem as constraints.

Since thrust and azimuth are components of the state, their range limits define the admissible state set

$$\mathcal{X} = \left\{ \mathbf{x}_v : \begin{array}{ll} T_{L,min} \le T_L \le T_{L,max}, & T_{R,min} \le T_R \le T_{R,max}, \\ \alpha_{L,min} \le \alpha_L \le \alpha_{L,max}, & \alpha_{R,min} \le \alpha_R \le \alpha_{R,max} \end{array} \right\}, \tag{3.76}$$

while the limits on their rates of change, which act on the input, define the admissible input set

$$\mathcal{U} = \left\{ \mathbf{u}_v : \begin{array}{ll} |\dot{T}_L| \le \dot{T}_{L,max}, & |\dot{T}_R| \le \dot{T}_{R,max}, \\ |\dot{\alpha}_L| \le \dot{\alpha}_{L,max}, & |\dot{\alpha}_R| \le \dot{\alpha}_{R,max} \end{array} \right\}. \tag{3.77}$$

The surge speed is bounded from below, so that the USV does not sail astern: without this bound, when the reference changes direction the tracking cost can be reduced faster by turning the stern towards it and backing than by turning the bow, which is not how the vessel is meant to be operated. A small negative bound $u_{min} \le 0$ still allows the slow backing needed to hold a position or to brake. Since the speed is a state that a disturbance can push below the bound, it is enforced as a soft constraint through a slack variable $\sigma_u \ge 0$,

$$u_{min} - u \le \sigma_u, \tag{3.78}$$

penalised by $\rho_u(\sigma_u)$ of the form (3.56), with its own weights $\mu_{u,1}$ and $\mu_{u,2}$.

The last constraint couples the USV to the UAV through the tether, and is the only one that depends on the other vehicle. The tether limits the distance between its attachment point on the USV deck and the UAV to a maximum admissible separation d_{max}. The attachment point is vertically aligned with the origin of the USV's body frame, at a fixed height z_t in the inertial frame, so its position is (x, y, z_t). Along the horizon, the USV controller uses the UAV's most recently computed predicted trajectory $\hat{\mathbf{p}}_a = [\hat{x}_a, \hat{y}_a, \hat{z}_a]^\top$, temporally resampled at the USV's prediction nodes covered by it, to compute the squared distance between the attachment point and the UAV,

$$d^2 = (x - \hat{x}_a)^2 + (y - \hat{y}_a)^2 + (\hat{z}_a - z_t)^2. \tag{3.79}$$

The requirement $d^2 \le d_{max}^2$ is enforced as a soft constraint through a slack variable $\sigma \ge 0$,

$$\underbrace{d^2 - d_{max}^2}_{h(\mathbf{x}_v)} \le \sigma, \tag{3.80}$$

which is penalised by $\rho(\sigma)$ as in (3.56), with large weights μ_1 and μ_2 so that violations remain small whenever geometrically feasible.

Optimal Control Problem

At every sampling instant k, the kinematic and dynamic components of the initial state come from the USV's current estimate, while the thruster components use the last command actually applied:

$$\mathbf{x}_{v,k} = \left[x_k, y_k, \psi_k, u_k, v_k, r_k,\ T_{L,k-1}, T_{R,k-1}, \alpha_{L,k-1}, \alpha_{R,k-1} \right]^\top. \tag{3.81}$$

Starting from the last applied command ensures that the rate limits in (3.61) are enforced relative to the actual thruster values, so the first input chosen is always attainable by the thrusters.

The references and the UAV's predicted trajectory are given data at each instant. Since the UAV's predicted trajectory can be shorter than the USV's horizon, let $N_c \leq N$ denote the last node of the USV's horizon covered by it. The tether constraint, and its slack, are enforced only for $i = 0, \ldots, N_c$.

Assembling the cost, the model, and the constraints gives the optimal control problem the USV solves at every sampling instant:

$$
\begin{aligned}
\min_{\hat{\mathbf{x}}_v,\, \hat{\mathbf{u}}_v,\, \hat{\sigma},\, \hat{\sigma}_u} \quad & \sum_{i=0}^{N-1} \Big[\ell\big(\hat{\mathbf{x}}_{v,k+i|k}, \hat{\mathbf{u}}_{v,k+i|k}\big) + \rho_u\big(\hat{\sigma}_{u,k+i|k}\big) \Big] + \ell_f\big(\hat{\mathbf{x}}_{v,k+N|k}\big) + \sum_{i=0}^{N_c} \rho\big(\hat{\sigma}_{k+i|k}\big) \\
\text{s.t.} \quad & \hat{\mathbf{x}}_{v,k|k} = \mathbf{x}_{v,k}, \\
& \hat{\mathbf{x}}_{v,k+i+1|k} = \boldsymbol{\Phi}_v\big(\hat{\mathbf{x}}_{v,k+i|k}, \hat{\mathbf{u}}_{v,k+i|k}\big), \quad i = 0, \ldots, N-1, \\
& \hat{\mathbf{x}}_{v,k+i|k} \in \mathcal{X}, \quad i = 0, \ldots, N, \\
& \hat{\mathbf{u}}_{v,k+i|k} \in \mathcal{U}, \quad i = 0, \ldots, N-1, \\
& u_{min} - \hat{u}_{k+i|k} \leq \hat{\sigma}_{u,k+i|k}, \quad \hat{\sigma}_{u,k+i|k} \geq 0, \quad i = 1, \ldots, N-1, \\
& h\big(\hat{\mathbf{x}}_{v,k+i|k}\big) \leq \hat{\sigma}_{k+i|k}, \quad \hat{\sigma}_{k+i|k} \geq 0, \quad i = 0, \ldots, N_c.
\end{aligned} \tag{3.82}
$$

The USV's low-level actuators receive the integrated thruster commands $T_{L,k}, T_{R,k}, \alpha_{L,k}, \alpha_{R,k}$, obtained by integrating the first optimal input rate $\hat{\mathbf{u}}_{v,k|k}$ over the sampling interval T_s. These become the thruster components of the initial state at the subsequent sampling instant.

3.2.4 UAV NMPC

The UAV NMPC computes, at every sampling instant, the thrust and attitude commands that best track a reference trajectory over a finite prediction horizon, subject to the vehicle's dynamics, its actuation limits, and the tether workspace. It predicts the UAV's motion with the model derived in Section 3.1, and solves a constrained optimisation problem at each step to obtain the commands applied to the UAV.

Predictive Model

The UAV model defined in Section 3.1 assumed that the commanded attitude is tracked instantaneously by the low-level flight controller. In reality, the autopilot tracks the commanded attitude with a delay, and it does so by rotating the vehicle so that the direction of the thrust approaches the commanded one, while the yaw is controlled separately. The commanded attitude is given by the setpoints $\phi_{\mathrm{cmd}}, \theta_{\mathrm{cmd}}, \psi_{\mathrm{cmd}}$, and, following (3.33), the commanded direction of the thrust is $\mathbf{R}(\phi_{\mathrm{cmd}}, \theta_{\mathrm{cmd}}, \psi_{\mathrm{cmd}})[0, 0, 1]^\top$. The setpoints reach the autopilot after a transport delay, τ_d for roll and pitch and $\tau_{d,\psi}$ for yaw, and the closed-loop response of the autopilot exhibits a first-order lag characterised by a time constant $\tau > 0$ for

the direction of the thrust and $\tau_\psi > 0$ for the yaw:

$$\begin{aligned}\dot{\mathbf{z}}^{\mathcal{B}_a} &= \frac{1}{\tau}\Big(\mathbf{R}\big(\phi_{\mathrm{cmd}}(t-\tau_d), \theta_{\mathrm{cmd}}(t-\tau_d), \psi_{\mathrm{cmd}}(t-\tau_d)\big)[0,0,1]^\top - \mathbf{z}^{\mathcal{B}_a}\Big), \\ \dot{\psi} &= \frac{1}{\tau_\psi}\big(\psi_{\mathrm{cmd}}(t-\tau_{d,\psi}) - \psi\big),\end{aligned} \tag{3.83}$$

where $\mathbf{z}^{\mathcal{B}_a}$, the direction of the thrust, is expressed in $\{\mathcal{I}\}$, and ψ is the actual yaw of the UAV.

Since the yaw only rotates the direction of the thrust about the vertical, this direction is described by two tilt angles ϕ, θ, measured relative to the commanded yaw,

$$\begin{aligned}\mathbf{z}^{\mathcal{B}_a} &= \mathbf{R}(\phi, \theta, \psi_{\mathrm{cmd}})[0,0,1]^\top = \mathbf{R}(0,0,\psi_{\mathrm{cmd}})\,\mathbf{n}(\phi,\theta), \\ \mathbf{n}(\phi,\theta) &= \mathbf{R}(\phi,\theta,0)[0,0,1]^\top = [\sin\theta\cos\phi,\ -\sin\phi,\ \cos\theta\cos\phi]^\top,\end{aligned} \tag{3.84}$$

which coincide with the roll and pitch angles of the UAV when its yaw equals the commanded yaw. The setpoints $\phi_{\mathrm{cmd}}, \theta_{\mathrm{cmd}}$ are, in the same way, the tilt angles of the commanded direction of the thrust.

Furthermore, similarly to the thrusters of the USV, the attitude commands and the thrust cannot change arbitrarily fast, and their rates of change are subject to physical slew-rate limits:

$$|\dot{\phi}_{\mathrm{cmd}}| \le \dot{\phi}_{max}, \quad |\dot{\theta}_{\mathrm{cmd}}| \le \dot{\theta}_{max}, \quad |\dot{\psi}_{\mathrm{cmd}}| \le \dot{\psi}_{max}, \quad |\dot{T}| \le \dot{T}_{max}. \tag{3.85}$$

Imposing these limits directly as bounds in the optimisation problem requires the rates of the commanded angles and of the thrust to be part of the control input vector,

$$\mathbf{u}_a = [\dot{\phi}_{\mathrm{cmd}}, \dot{\theta}_{\mathrm{cmd}}, \dot{\psi}_{\mathrm{cmd}}, \dot{T}]^\top \in \mathbb{R}^4, \tag{3.86}$$

which in turn incorporates the commanded attitude and the thrust into the state vector,

$$\mathbf{x}_a = [x, y, z, \dot{x}, \dot{y}, \dot{z},\ \phi, \theta, \psi,\ \phi_{\mathrm{cmd}}, \theta_{\mathrm{cmd}}, \psi_{\mathrm{cmd}},\ T]^\top \in \mathbb{R}^{13}. \tag{3.87}$$

With this state augmentation, range limits act on the state and rate limits act directly on the input.

With $\mathbf{x}_a$ and $\mathbf{u}_a$ defined, the prediction model $\dot{\mathbf{x}}_a = \mathbf{f}_a(\mathbf{x}_a, \mathbf{u}_a)$ is assembled from kinematics, translational dynamics, attitude lag, and command integrators.

The kinematics are the trivial identity already used in (3.29),

$$\dot{x} = \dot{x}, \qquad \dot{y} = \dot{y}, \qquad \dot{z} = \dot{z}. \tag{3.88}$$

The translational dynamics follow (3.37), with the thrust term expanded using the direction of the thrust (3.84), including the linear horizontal rotor drag opposing translational motion, the tether force $\mathbf{F}^{\mathcal{I}}_{\mathrm{tether}} = [F^{\mathcal{I}}_{\mathrm{tether},x}, F^{\mathcal{I}}_{\mathrm{tether},y}, F^{\mathcal{I}}_{\mathrm{tether},z}]^\top$ given by (3.38)–(3.46), and an estimated external force $\hat{\mathbf{d}} = [\hat{d}_x, \hat{d}_y, \hat{d}_z]^\top$ that accounts for the forces the model does not describe, such as wind, and is held constant

along the horizon (its estimation is given with the optimal control problem):

$$\ddot{x} = \frac{T}{m}\big(\cos\psi_{\mathrm{cmd}}\sin\theta\cos\phi + \sin\psi_{\mathrm{cmd}}\sin\phi\big) - \frac{d_r}{m}\dot{x} + \frac{1}{m}\big(F^{\mathcal{I}}_{\mathrm{tether},x} + \hat{d}_x\big), \tag{3.89}$$

$$\ddot{y} = \frac{T}{m}\big(\sin\psi_{\mathrm{cmd}}\sin\theta\cos\phi - \cos\psi_{\mathrm{cmd}}\sin\phi\big) - \frac{d_r}{m}\dot{y} + \frac{1}{m}\big(F^{\mathcal{I}}_{\mathrm{tether},y} + \hat{d}_y\big), \tag{3.90}$$

$$\ddot{z} = \frac{T}{m}\cos\theta\cos\phi - g + \frac{1}{m}\big(F^{\mathcal{I}}_{\mathrm{tether},z} + \hat{d}_z\big). \tag{3.91}$$

The attitude dynamics follow from the response (3.83). Since the delayed setpoints are not part of the state, they are approximated to first order with their rates, which are inputs of the model, $\phi_{\mathrm{cmd}}(t-\tau_d) \approx \phi_{\mathrm{cmd}} - \tau_d\,\dot{\phi}_{\mathrm{cmd}}$, and likewise for θ_{cmd} and ψ_{cmd}. Expressing the response of the direction of the thrust in the frame rotated by ψ_{cmd}, in which this direction is $\mathbf{n}(\phi,\theta)$, gives

$$\dot{\mathbf{n}} = \frac{1}{\tau}\Big(\mathbf{R}\big(0,0,-\tau_d\dot{\psi}_{\mathrm{cmd}}\big)\,\mathbf{n}\big(\phi_{\mathrm{cmd}} - \tau_d\dot{\phi}_{\mathrm{cmd}},\ \theta_{\mathrm{cmd}} - \tau_d\dot{\theta}_{\mathrm{cmd}}\big) - \mathbf{n}\Big) - \dot{\psi}_{\mathrm{cmd}}\,[0,0,1]^\top \times \mathbf{n}, \tag{3.92}$$

where the rotation inside the brackets accounts for the change of the commanded yaw during the delay, and the last term for the rotation of the frame itself. The partial derivatives of $\mathbf{n}$ with respect to ϕ and θ are orthogonal, with norms 1 and $\cos\phi$, so the rates of the tilt angles are the projections of $\dot{\mathbf{n}}$ onto them. Together with the response of the yaw, the attitude dynamics are

$$\dot{\phi} = \Big(\frac{\partial\mathbf{n}}{\partial\phi}\Big)^\top\dot{\mathbf{n}}, \qquad \dot{\theta} = \frac{1}{\cos^2\phi}\Big(\frac{\partial\mathbf{n}}{\partial\theta}\Big)^\top\dot{\mathbf{n}}, \qquad \dot{\psi} = \frac{1}{\tau_\psi}\big(\psi_{\mathrm{cmd}} - \tau_{d,\psi}\dot{\psi}_{\mathrm{cmd}} - \psi\big), \tag{3.93}$$

and they are completed by the command rate integrators:

$$\dot{\phi}_{\mathrm{cmd}} = \dot{\phi}_{\mathrm{cmd}}, \qquad \dot{\theta}_{\mathrm{cmd}} = \dot{\theta}_{\mathrm{cmd}}, \qquad \dot{\psi}_{\mathrm{cmd}} = \dot{\psi}_{\mathrm{cmd}}, \qquad \dot{T} = \dot{T}. \tag{3.94}$$

Discretisation over a sampling interval T_s, following Section 3.2.2, gives the one-step map $\mathbf{\Phi}_a$ such that $\hat{\mathbf{x}}_{a,k+i+1|k} = \mathbf{\Phi}_a\big(\hat{\mathbf{x}}_{a,k+i|k}, \hat{\mathbf{u}}_{a,k+i|k}\big)$ approximates the solution of $\dot{\mathbf{x}}_a = \mathbf{f}_a(\mathbf{x}_a,\mathbf{u}_a)$ over one sampling interval.

Cost and Constraints

Let $\mathbf{p} = [x,y,z]^\top$ and $\mathbf{v} = [\dot{x},\dot{y},\dot{z}]^\top$ denote the position and velocity, with $\mathbf{p}_{ref}$ and $\mathbf{v}_{ref}$ the corresponding references.

Tracking the reference position and velocity is penalised quadratically as

$$\|\mathbf{p} - \mathbf{p}_{ref}\|^2_{Q_p} + \|\mathbf{v} - \mathbf{v}_{ref}\|^2_{Q_v}. \tag{3.95}$$

The next term penalises ϕ,θ directly, rather than comparing them against any reference,

$$\big\|[\phi,\theta]^\top\big\|^2_{Q_{tilt}}, \tag{3.96}$$

discouraging attitude larger than necessary to track the reference.

The next term penalises the yaw error, but rather than tracking a fixed yaw angle, ψ is aligned with the direction to a reference point $\mathbf{p}_{b,ref} = [x_{b,ref}, y_{b,ref}]^\top$, itself possibly varying along the horizon,

$$\mathbf{b} = \mathbf{p}_{b,ref} - \mathbf{p}_{xy}, \tag{3.97}$$

where $\mathbf{p}_{xy} = [x, y]^\top$. Since ψ is an angle, the periodicity issue discussed for (3.70) also applies here, and it is handled by expressing the error as an alignment between ψ and the bearing vector $\mathbf{b}$, rather than as a difference between two angles,

$$q_\psi \left(1 - \frac{\cos\psi\, b_x + \sin\psi\, b_y}{\sqrt{\|\mathbf{b}\|^2 + \epsilon}}\right), \tag{3.98}$$

where $q_\psi \geq 0$ is a scalar weight setting the relative importance of yaw tracking, and $\epsilon > 0$ is a regularisation constant. This term is minimised when ψ points along $\mathbf{b}$. Far from the reference point, where $\|\mathbf{b}\|^2 \gg \epsilon$, the term approaches q_ψ times one minus the cosine of the angle between ψ and $\mathbf{b}$, while within a horizontal distance of the order of $\sqrt{\epsilon}$ its weight fades. The term thus remains well defined above the reference point, where the bearing is undefined, and does not drive the UAV to rotate when it passes close to it.

The thrust is penalised relative to the hover thrust, since it cannot be penalised toward zero without opposing gravity,

$$Q_T\,(T - mg)^2, \tag{3.99}$$

where $Q_T \geq 0$ is a scalar weight.

The last term penalises the input $\mathbf{u}_a$, the rates of change of the attitude and thrust commands,

$$\|\mathbf{u}_a\|_R^2, \tag{3.100}$$

discouraging abrupt changes of the commands between consecutive instants.

Summing the tracking, tilt, yaw, thrust, and input penalties in (3.95), (3.96), (3.98), (3.99), and (3.100) gives the stage cost

$$\begin{aligned}\ell(\mathbf{x}_a, \mathbf{u}_a) = {} & \|\mathbf{p} - \mathbf{p}_{ref}\|_{Q_p}^2 + \|\mathbf{v} - \mathbf{v}_{ref}\|_{Q_v}^2 + \left\|[\phi, \theta]^\top\right\|_{Q_{tilt}}^2 \\ & + q_\psi \left(1 - \frac{\cos\psi\, b_x + \sin\psi\, b_y}{\|\mathbf{b}\|}\right) + Q_T\,(T - mg)^2 + \|\mathbf{u}_a\|_R^2.\end{aligned} \tag{3.101}$$

The terminal cost ℓ_f penalises the last predicted state, so that the optimiser does not disregard where the UAV ends up at the end of the horizon. It equals the stage cost (3.101) without the input term, since no input is chosen at the final node:

$$\begin{aligned}\ell_f(\mathbf{x}_a) = {} & \|\mathbf{p} - \mathbf{p}_{ref}\|_{Q_p}^2 + \|\mathbf{v} - \mathbf{v}_{ref}\|_{Q_v}^2 + \left\|[\phi, \theta]^\top\right\|_{Q_{tilt}}^2 \\ & + q_\psi \left(1 - \frac{\cos\psi\, b_x + \sin\psi\, b_y}{\|\mathbf{b}\|}\right) + Q_T\,(T - mg)^2.\end{aligned} \tag{3.102}$$

The UAV's operating envelope bounds the altitude, translational velocity, and attitude. To guarantee safe clearance above the water surface and USV deck, the UAV is restricted to $z \geq z_{min}$. Furthermore, because horizontal and vertical velocities are limited by distinct physical factors, the UAV's speed and tilt are bounded separately:

$$z \geq z_{min}, \quad \left\| [\dot{x}, \dot{y}]^\top \right\| \leq v_{xy,max}, \quad |\dot{z}| \leq v_{z,max}, \quad |\phi| \leq \phi_{max}, \quad |\theta| \leq \theta_{max}. \tag{3.103}$$

Unlike the thruster commands of the USV, the velocity and attitude are subject to external disturbances such as wind, which can push the true state past these bounds regardless of what the NMPC has commanded. The bounds of (3.103) are therefore softened as in (3.55), each with its own slack variable $\sigma_j \geq 0$, $j = 1, \ldots, 5$:

$$\underbrace{z_{min} - z}_{h_1(\mathbf{x}_a)} \leq \sigma_1, \tag{3.104}$$

$$\underbrace{\left\| [\dot{x}, \dot{y}]^\top \right\| - v_{xy,max}}_{h_2(\mathbf{x}_a)} \leq \sigma_2, \tag{3.105}$$

$$\underbrace{|\dot{z}| - v_{z,max}}_{h_3(\mathbf{x}_a)} \leq \sigma_3, \tag{3.106}$$

$$\underbrace{|\phi| - \phi_{max}}_{h_4(\mathbf{x}_a)} \leq \sigma_4, \tag{3.107}$$

$$\underbrace{|\theta| - \theta_{max}}_{h_5(\mathbf{x}_a)} \leq \sigma_5. \tag{3.108}$$

Each slack is penalised by $\rho_j(\sigma_j)$ as in (3.56), with weights $\mu_{1,j}$ and $\mu_{2,j}$.

The thrust range acts on the state, since the thrust is a component of $\mathbf{x}_a$, defining the admissible state set

$$\mathcal{X} = \{\mathbf{x}_a : \; T_{min} \leq T \leq T_{max}\}, \tag{3.109}$$

while the command rate limits of the predictive model (3.85) act on the input, defining the admissible input set

$$\mathcal{U} = \left\{ \mathbf{u}_a : \begin{array}{ll} |\dot{\phi}_{\mathrm{cmd}}| \leq \dot{\phi}_{max}, & |\dot{\theta}_{\mathrm{cmd}}| \leq \dot{\theta}_{max}, \\ |\dot{\psi}_{\mathrm{cmd}}| \leq \dot{\psi}_{max}, & |\dot{T}| \leq \dot{T}_{max} \end{array} \right\}. \tag{3.110}$$

The remaining constraints couple the UAV to the USV through the tether, and are the only ones that depend on the other vehicle. First, as for the USV, the tether limits the distance between the UAV and the tether attachment point, described in Section 3.2.3, to the maximum admissible separation d_{max}. The attachment point on the USV is located at $(\hat{x}_v, \hat{y}_v, z_t)$, where the horizontal position $\hat{\mathbf{p}}_v = [\hat{x}_v, \hat{y}_v]^\top$ at each node is taken from the USV's most recently computed predicted trajectory, temporally resampled at the UAV's prediction nodes, and z_t is the height of the attachment point. Since the USV's horizon covers that of the UAV, the constraint is enforced at every node. Denoting by $\mathbf{r} = [x - \hat{x}_v, y - \hat{y}_v, z - z_t]^\top$ the relative position vector between the UAV and the attachment point, the squared separation distance

is

$$d^2 = \|\mathbf{r}\|^2 = (x - \hat{x}_v)^2 + (y - \hat{y}_v)^2 + (z - z_t)^2. \tag{3.111}$$

The requirement $d^2 \leq d_{max}^2$ is enforced as a soft constraint through a slack variable $\sigma_6 \geq 0$,

$$\underbrace{d^2 - d_{max}^2}_{h_6(\mathbf{x}_a)} \leq \sigma_6, \tag{3.112}$$

which is penalised by $\rho_6(\sigma_6)$ as in (3.56), with large weights $\mu_{1,6}$ and $\mu_{2,6}$ so that violations remain small.

Similarly, to prevent tension spikes during tether deployment, the rate at which the inter-vehicle distance increases must not exceed the maximum pay-out rate of the winch mechanism, $\dot{d} \leq \dot{d}_{max}$. Differentiating $d = \|\mathbf{r}\|$ with respect to time yields the projection of the relative velocity $\dot{\mathbf{r}} = [\dot{x} - \dot{\hat{x}}_v,\ \dot{y} - \dot{\hat{y}}_v,\ \dot{z}]^\top$ onto the tether line of sight:

$$\dot{d} = \frac{\mathrm{d}}{\mathrm{d}t}\|\mathbf{r}\| = \frac{\mathbf{r}^\top \dot{\mathbf{r}}}{d}. \tag{3.113}$$

This formulation restricts only radial motion that moves away from the USV ($\dot{d} > 0$), leaving tangential and approaching motions unconstrained. Enforcing $\dot{d} \leq \dot{d}_{max}$ with a small regularisation $\epsilon > 0$ to avoid numerical singularity when $d \to 0$ gives the softened constraint:

$$\underbrace{\frac{\mathbf{r}^\top \dot{\mathbf{r}}}{\sqrt{d^2 + \epsilon}} - \dot{d}_{max}}_{h_7(\mathbf{x}_a)} \leq \sigma_7, \tag{3.114}$$

which is penalised by $\rho_7(\sigma_7)$ as in (3.56).

Together, the minimum altitude z_{min} and the maximum tether separation d_{max} delimit the workspace of the UAV, illustrated in Figure 3.6.

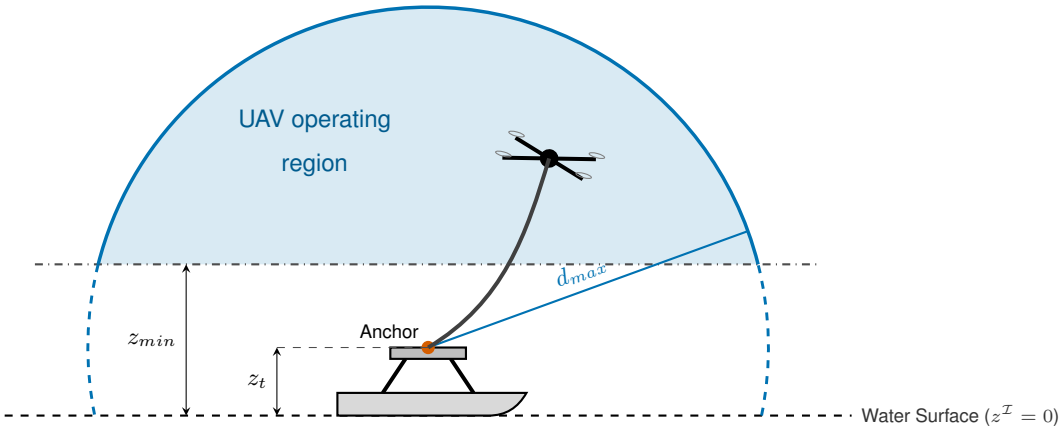


Figure 3.6: Operating region of the UAV.

All seven of these state constraints have the form $h_j(\mathbf{x}_a) \leq \sigma_j$ and are softened, so that the only hard constraint on the state is the thrust range (3.109). With the slacks stacked as $\boldsymbol{\sigma} = [\sigma_1, \ldots, \sigma_7]^\top$,

the combined penalty is

$$P(\boldsymbol{\sigma}) = \sum_{j=1}^{7} \rho_j(\sigma_j). \tag{3.115}$$

Optimal Control Problem

At every sampling instant k, the translational and physical attitude components of the initial state come from the UAV's current state estimate, while the commanded attitude and thrust components use the last commands actually applied:

$$\mathbf{x}_{a,k} = \left[x_k, y_k, z_k, \dot{x}_k, \dot{y}_k, \dot{z}_k,\ \phi_k, \theta_k, \psi_k,\ \phi_{\mathrm{cmd},k-1}, \theta_{\mathrm{cmd},k-1}, \psi_{\mathrm{cmd},k-1},\ T_{k-1}\right]^\top. \tag{3.116}$$

The tilt angles ϕ_k, θ_k are obtained from the estimated direction of the thrust, expressed in the frame rotated by the last commanded yaw as in (3.84), $\mathbf{R}(0,0,\psi_{\mathrm{cmd},k-1})^\top \mathbf{z}^{\mathcal{B}_a} = \mathbf{n}(\phi_k, \theta_k)$. Since the state estimate is not synchronised with the sampling instants, the position is also propagated with the estimated velocity from the instant t_e of the most recent estimate to the sampling instant t_k, $\mathbf{p}_k = \mathbf{p}(t_e) + \mathbf{v}(t_e)\,(t_k - t_e)$.

A force that the model does not describe makes the prediction drift from the true motion, and without a model of it the controller settles with a steady offset. Following the disturbance-model approach to offset-free predictive control [23, 24], the external force $\hat{\mathbf{d}}$ in (3.89)–(3.91) is estimated at every sampling instant from the one-step prediction error of the velocity. The velocity $\hat{\mathbf{v}}_{k|k-1}$ predicted for the current instant by the solution of the previous instant is compared with the estimated velocity $\mathbf{v}_k$, and the force left unexplained over the last interval corrects the estimate,

$$\hat{\mathbf{d}}_k = \mathrm{sat}_{\bar{d}}\Big(\hat{\mathbf{d}}_{k-1} + \beta\,\frac{m}{T_s}\big(\mathbf{v}_k - \hat{\mathbf{v}}_{k|k-1}\big)\Big), \tag{3.117}$$

where $0 < \beta < 1$ sets the speed of the estimate, trading the rejection of the disturbance against the transmission of measurement noise, and $\mathrm{sat}_{\bar{d}}$ bounds each component to $\pm\bar{d}$. For a constant force the update vanishes only when the prediction matches the motion, so the steady offset is removed.

The references and the USV's most recently computed predicted trajectory are given data at each instant.

Assembling the cost, the model, and the constraints gives the optimal control problem the UAV solves

at every sampling instant, with the slacks $\hat{\sigma}_{j,k+i|k}$ entering as additional decision variables:

$$
\begin{aligned}
\min_{\hat{\mathbf{x}}_a,\,\hat{\mathbf{u}}_a,\,\hat{\boldsymbol{\sigma}}} \quad & \sum_{i=0}^{N-1} \ell\big(\hat{\mathbf{x}}_{a,k+i|k}, \hat{\mathbf{u}}_{a,k+i|k}\big) + \ell_f\big(\hat{\mathbf{x}}_{a,k+N|k}\big) + \sum_{i=0}^{N} P\big(\hat{\boldsymbol{\sigma}}_{k+i|k}\big) \\
\text{s.t.} \quad & \hat{\mathbf{x}}_{a,k|k} = \mathbf{x}_{a,k}, \\
& \hat{\mathbf{x}}_{a,k+i+1|k} = \boldsymbol{\Phi}_a\big(\hat{\mathbf{x}}_{a,k+i|k}, \hat{\mathbf{u}}_{a,k+i|k}\big), \quad i = 0, \dots, N-1, \\
& \hat{\mathbf{x}}_{a,k+i|k} \in \mathcal{X}, \quad i = 0, \dots, N, \\
& \hat{\mathbf{u}}_{a,k+i|k} \in \mathcal{U}, \quad i = 0, \dots, N-1, \\
& h_j\big(\hat{\mathbf{x}}_{a,k+i|k}\big) \le \hat{\sigma}_{j,k+i|k}, \quad \hat{\sigma}_{j,k+i|k} \ge 0, \quad j = 1, \dots, 7, \quad i = 0, \dots, N.
\end{aligned}
\tag{3.118}
$$

The UAV's low-level autopilot receives the attitude setpoints $\phi_{\mathrm{cmd},k}, \theta_{\mathrm{cmd},k}, \psi_{\mathrm{cmd},k}$, and the thrust command T_k, obtained by integrating the first optimal rates $\hat{\dot{\phi}}_{\mathrm{cmd},k|k}, \hat{\dot{\theta}}_{\mathrm{cmd},k|k}, \hat{\dot{\psi}}_{\mathrm{cmd},k|k}, \hat{\dot{T}}_{k|k}$ of the first optimal input $\hat{\mathbf{u}}_{a,k|k}$ over T_s. These become the commanded attitude and thrust components of the initial state at the subsequent sampling instant.

Chapter 4

Implementation

Chapter 3 defines the models and the controllers of the tethered USV–UAV system, but their behaviour and performance can only be assessed once they are implemented and executed. This depends on how the controllers interact with the vehicles, the tether, and the marine environment, which cannot be studied analytically.

For this reason, a realistic simulation environment, integrated with ROS 2, was developed to support the design of the controllers. It represents the vehicles and the tether, and provides controlled and repeatable conditions under which the controllers are tested. The controllers were then implemented to run in real time and adapted to the vehicles used in simulation. Finally, manoeuvres and missions were defined to evaluate the performance of the controllers in representative operations.

4.1 Simulation Environment

Simulation is the natural first step in the development of the controllers, since it allows them to be tested and adjusted before being used on the real vehicles. The simulations in this work are run in Gazebo [25], a robotics simulator. A simulator is a program that creates a virtual environment, places robots in it, and calculates how they would behave in reality. At each step, it receives the commands sent to the robots, computes the forces acting on them, updates their positions accordingly, and returns the measurements that their sensors would produce. To the controller, the simulated robot behaves like a real one.

This allows controllers to be developed through many fast iterations, and different scenarios to be repeated under the same conditions, which makes the comparison between controllers and parameter choices fair. Sensor noise and estimation error can also be included in simulation, but they can be left out during development, so that the behaviour of the controllers is analysed on its own.

4.1.1 World

The space where the simulation takes place is called the world. It contains the environment around the robots, such as the terrain, obstacles, and light, and defines the rules that apply inside it, such as

the way time advances, the physical effects that are calculated, and the sensors that can be simulated. The robots are not part of the world. They are added to it afterwards, and what happens to them results from the interaction between their own models and the world in which they are placed.

A world is defined in a text file that lists its elements and the plugins that enable its behaviour. Plugins are software modules that add functionality to the simulator, such as the calculation of buoyancy or the simulation of sensors.

The world used in this work represents an open water area, shown in Figure 4.1. It consists of a sea surface and a seabed below it, and the buoyancy is calculated from the density of the water, which allows the hull of the USV to float. The world also enables the simulation of the sensors that the vehicles can use to estimate their state, such as inertial, magnetic, pressure, and satellite positioning sensors.

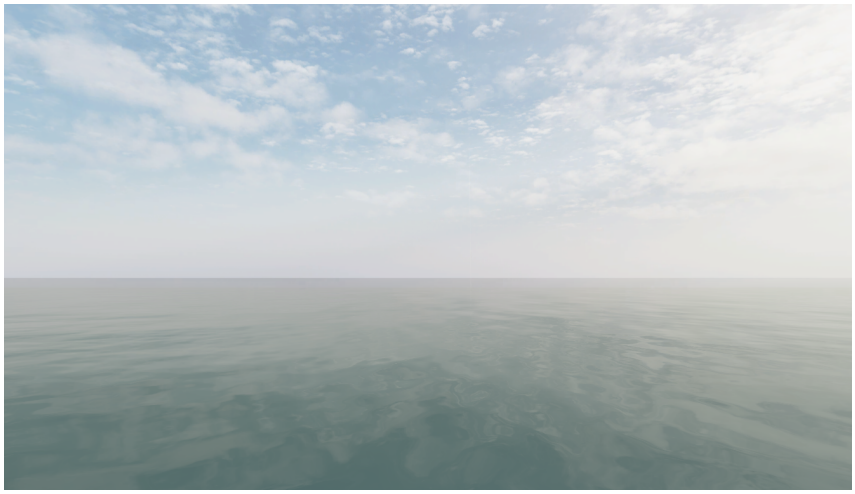


Figure 4.1: View of the simulated world

The vehicles operate at sea, where wind, waves, and currents act on them. The world allows these disturbances to be included, so that the controllers can be tested under conditions that their models do not predict. Each disturbance acts through a different physical path. Wind applies a force on the bodies exposed to the air, waves change the height of the water surface and therefore the buoyancy of the hull, and currents move the water itself, which changes the drag on the submerged bodies. These disturbances can be enabled, disabled, or adjusted for each test.

4.1.2 Wave Adaptive Modular Vessel

The WAM-V is a catamaran with two inflatable hulls connected by a central payload deck, used in maritime robotics competitions. In this work it is the USV of the tethered system. It was chosen mainly because its payload deck is large enough to serve as a platform for the takeoff and landing of the UAV. In addition, a complete and publicly available model of the USV exists in the Virtual RobotX (VRX) simulation framework [26], so that the work could focus on the control and not on building and validating a USV model. Figure 4.2 shows the vehicle in the simulated world.

In the simulator, a vehicle is described by a model that lists its bodies, the joints that connect them, and for each body its mass, its inertia, and its geometry. The WAM-V is represented by a main rigid body,

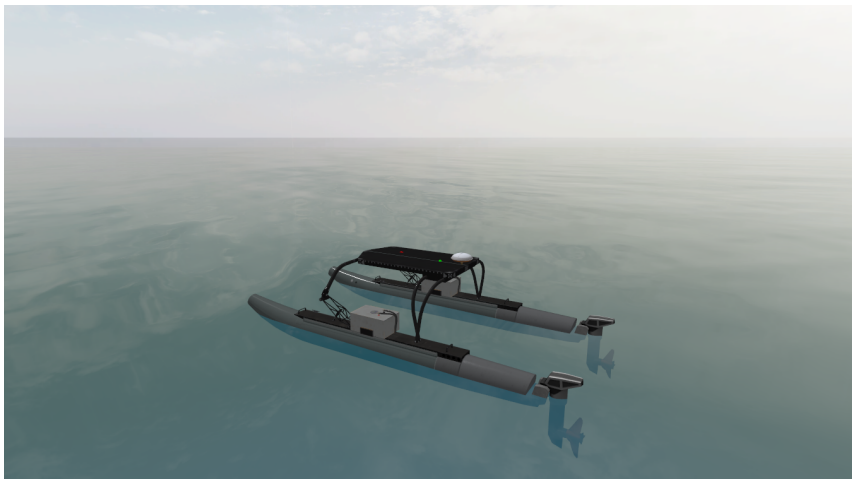
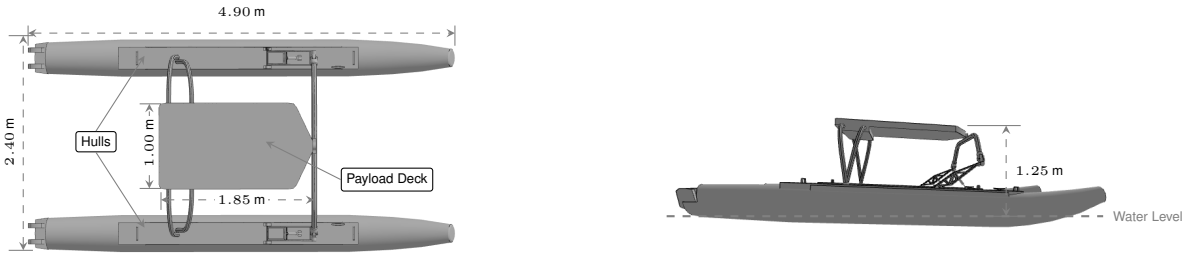


Figure 4.2: The WAM-V catamaran in the simulated world.

which includes the hulls, the deck, the batteries, and the mass of the thrusters, propellers, and sensors, with a total mass of 259.10 kg and moments of inertia of 219, 568, and $683\,\text{kg} \cdot \text{m}^2$ about the longitudinal, lateral, and vertical axes. The thrusters, propellers, and sensors are also modelled as separate bodies attached to it through joints, with negligible mass. The thrusters can rotate to steer the USV, while the joints of the sensors are fixed. As the thrusters are mounted far aft, the centre of gravity of the whole vehicle lies 0.29 m behind the centre of the hull geometry. The origin and the centre of mass of the main body are placed at this point, which is where the hydrodynamic forces described next are applied and which the odometry of the simulator refers to.

The catamaran has an overall length of 4.90 m and a beam of 2.40 m. The payload deck, with 1.85 m in length and 1.00 m in width, is supported by the two hulls at about 1.25 m above the water surface, as shown in Figure 4.3.



(a) Top view.

(b) Side view.

Figure 4.3: WAM-V catamaran layout.

The behaviour of the USV on the water is determined by how the simulator reproduces the forces that the water exerts on it. The hulls are kept afloat by a buoyancy force, which is calculated from the volume of each hull below the water surface. When waves are present, this volume changes along the hulls and the catamaran pitches and rolls accordingly. As the USV moves, the water also resists its motion through drag forces and moments applied in the six degree of freedoms (DOFs). For a velocity

ν_i along or about each degree of freedom, the drag is

$$D_i = -\left(d_i^{\text{lin}} + d_i^{\text{quad}}\,|\nu_i|\right)\nu_i, \tag{4.1}$$

where d_i^{lin} and d_i^{quad} are the linear and quadratic coefficients. The negative sign makes the drag oppose the motion, and the linear term dominates at low speeds while the quadratic term dominates at higher speeds. This makes the vehicle slow down naturally when the propulsion is cut off. The coefficients of each degree of freedom, given in Table 4.1, are those provided by the VRX model.

Table 4.1: Hydrodynamic damping coefficients of the simulated WAM-V.

Damping Mode	Linear Coefficient	Quadratic Coefficient	Unit
Surge	100.00	150.00	$\text{N}/(\text{m/s})$ or $\text{N}/(\text{m/s})^2$
Sway	100.00	100.00	$\text{N}/(\text{m/s})$ or $\text{N}/(\text{m/s})^2$
Heave	500.00	–	$\text{N}/(\text{m/s})$
Roll	300.00	600.00	$\text{N}\cdot\text{m}/(\text{rad/s})$ or $\text{N}\cdot\text{m}/(\text{rad/s})^2$
Pitch	900.00	900.00	$\text{N}\cdot\text{m}/(\text{rad/s})$ or $\text{N}\cdot\text{m}/(\text{rad/s})^2$
Yaw	800.00	800.00	$\text{N}\cdot\text{m}/(\text{rad/s})$ or $\text{N}\cdot\text{m}/(\text{rad/s})^2$

The vehicle is propelled by two azimuth thrusters, one at the stern of each hull, and it steers by rotating them instead of using a rudder. To move, the model needs to receive, for each thruster, the thrust force that it must produce, in newtons, and its steering angle. The angle is limited to ± 30 degrees, to avoid twisting the power cables, and its rate of change to 10 degrees per second. The thrust is limited to $510.0\,\text{N}$ in the forward direction, a value representative of the electric outboard motors used on the WAM-V in robotics competitions, such as the Torqeedo Cruise 2.0 [27]. In reverse, the propeller is less efficient, which is modelled with a multiplier of 0.746 that limits the thrust to $-380.0\,\text{N}$.

4.1.3 x500 UAV

The Holybro X500 v2 [28] is a UAV, and in this work it is the aerial vehicle of the tethered system. It was chosen mainly because it is heavier and more stable than small UAVs, which makes it better suited to fly with a tether attached to it. Its size is also compatible with the payload deck of the WAM-V, leaving enough room for takeoff and landing, and a complete model of the vehicle is available in the simulation ecosystem of the PX4 autopilot [29], so that the work could focus on the control and not on building and validating a vehicle model. Figure 4.4 shows the vehicle in the simulated world.

The x500 is described in the simulator in the same way as the WAM-V, by a model that lists its bodies, the joints that connect them, and the mass and inertia of each one. It consists of a main rigid body, which includes the frame and the sensors, and of four rotors, which are separate bodies connected to it through rotating joints. Their masses, inertias, and positions are given in Table 4.2. The rotors are $0.348\,\text{m}$ apart along the sides of the frame and about $0.49\,\text{m}$ apart along the diagonal. The landing gear is $0.264\,\text{m}$ wide, and the motor mounts are $0.26\,\text{m}$ above the ground, as shown in Figure 4.5.

The vehicle is lifted by the thrust that its rotors produce by pushing air downwards. The model receives, for each rotor i, a command with its angular speed ω_i, in rad/s, and each rotor produces a

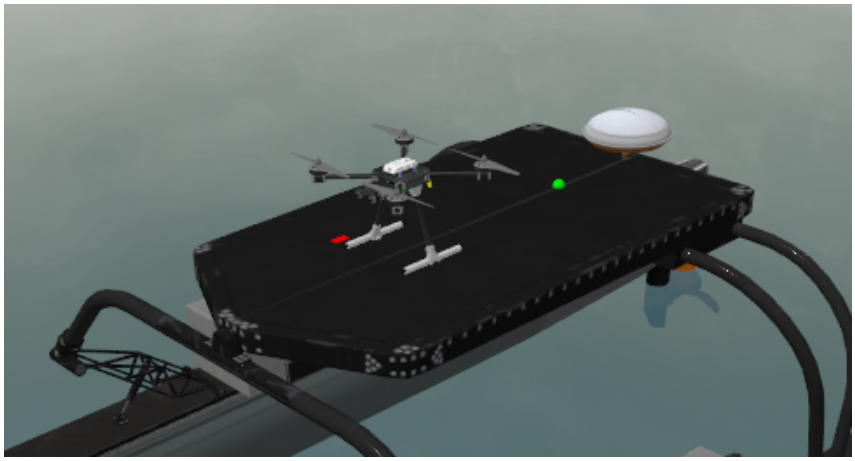
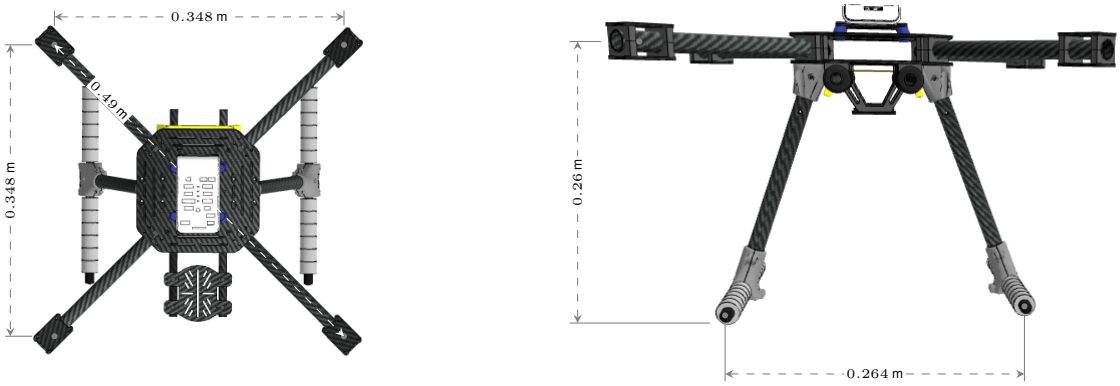


Figure 4.4: The x500 UAV in the simulated world.



(a) Top view.

(b) Front view.

Figure 4.5: x500 UAV layout.

Table 4.2: Parameters of the simulated x500, based on the PX4 model [30].

Parameter	**Value**	**Unit**
Main body mass	2.0	kg
Rotor mass (each)	0.016	kg
Total mass	2.06	kg
Moment of inertia, longitudinal and lateral axes	0.0217	$\text{kg} \cdot \text{m}^2$
Moment of inertia, vertical axis	0.040	$\text{kg} \cdot \text{m}^2$
Rotor position, along each horizontal axis	± 0.174	m
Thrust coefficient k_F	1.11×10^{-5}	$\text{N}/(\text{rad/s})^2$
Moment coefficient k_M	0.016	m
Rotor drag coefficient c_{drag}	8.06×10^{-5}	$\text{N} \cdot \text{s}/\text{rad} \cdot \text{m}$
Minimum rotor speed	150	rad/s
Maximum rotor speed	1000	rad/s
Rotor time constant, accelerating	12.5	ms
Rotor time constant, decelerating	25	ms

thrust force F_i and a reaction moment M_i that depend on it [31],

$$F_i = k_F\,\omega_i^2, \qquad M_i = s_i\,k_M\,F_i, \tag{4.2}$$

where k_F is the thrust coefficient, k_M is the moment coefficient, and $s_i = \pm 1$ gives the sign of the moment, which depends on the direction of rotation. Rotors on the same diagonal rotate in the same direction and adjacent rotors in opposite directions, so that the reaction moments balance each other when the vehicle hovers. The model also includes the aerodynamic drag of the rotors.

The rotors have limits that the model reproduces. The speed of each rotor does not change instantly, and it follows the command with the time constants given in Table 4.2, up to the maximum speed given in the same table. At this speed, each rotor produces about 11.1 N, and the four together produce 44.5 N. As the weight of the vehicle is 20.2 N, the thrust-to-weight ratio is about 2.2. In hovering, the rotors produce about 45% of their maximum thrust, at a speed of about 675 rad/s, which leaves about 24 N available to manoeuvre and to counter the force of the tether.

4.1.4 Tether

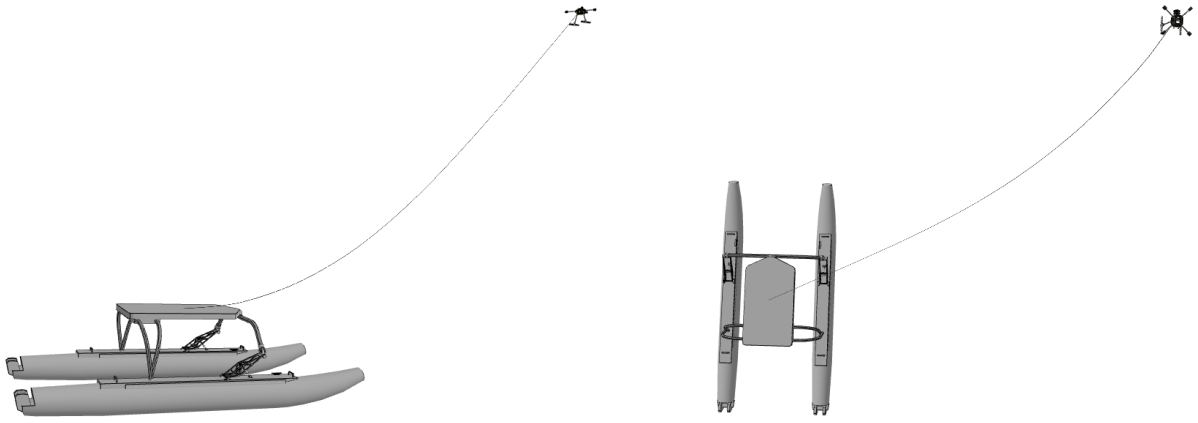
The tether couples the two vehicles, so the behaviour of the system depends on how it is represented. To evaluate the controllers as close as possible to real operation, they should be tested against a tether model that is more detailed than the one used in their design. The model used by the controllers, described in Section 3.1, approximates the tether by a catenary linearised around a constant slack. This simplification keeps the computational cost low, since the controllers evaluate the model at every step of the prediction horizon within each sampling interval. The simulated tether does not have this constraint, so a more detailed model can be used. Since this type of tether is not included in the vehicle models, it is simulated separately and coupled to them.

The tether is simulated with MoorDyn [32], a tool that represents a tether as a chain of lumped masses connected by segments that behave as springs and dampers. Each segment resists stretching and bending, while the weight and the drag of the tether are calculated at the nodes between segments. The slack and the tension of the tether result from the motion of these nodes, without assuming a shape for the tether. MoorDyn runs separately from Gazebo, which does not represent the tether itself but receives its effect on the vehicles through the forces that MoorDyn calculates. The tether is therefore displayed in a separate visualisation tool, Foxglove [33], as shown in Figure 4.6.

In this type of model, the mass of the tether is concentrated at the nodes, and the segments between them have no mass, as shown in Figure 4.7a. The motion of each free node i follows from Newton's second law, with the forces shown in Figure 4.7b,

$$m_i\,\ddot{\mathbf{p}}_i = \mathbf{T}_{i-1} + \mathbf{T}_i + \mathbf{F}_i^{\text{bend}} + m_i\mathbf{g} + \mathbf{F}_i^{\text{drag}}, \tag{4.3}$$

where m_i and $\mathbf{p}_i$ are the mass and the position of the node, $\mathbf{T}_{i-1}$ and $\mathbf{T}_i$ are the tensions of the two adjacent segments, $\mathbf{F}_i^{\text{bend}}$ is the force due to the bending of the tether at the node, and $\mathbf{F}_i^{\text{drag}}$ is the drag,



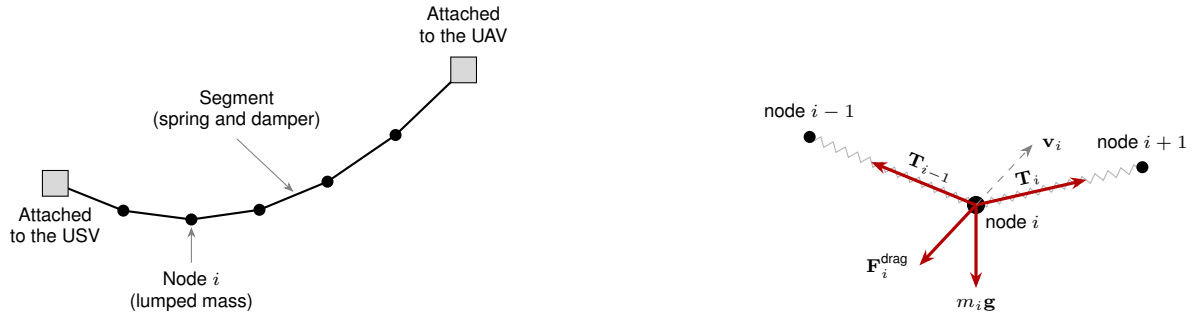
(a) Side view. (b) Top view.

Figure 4.6: The tether between the USV and the UAV, displayed in Foxglove.

which opposes the velocity of the node. The magnitude of the tension of a segment depends on its elongation and on the rate at which it changes,

$$T = \begin{cases} EA\,\eta + BA\,\dot{\eta}, & \eta > 0, \\ 0, & \eta \leq 0, \end{cases} \qquad \eta = \frac{\ell - \ell_0}{\ell_0}, \tag{4.4}$$

where ℓ and ℓ_0 are the current and the unstretched lengths of the segment, EA is the axial stiffness, and BA is the damping coefficient, which is defined through the damping ratio given in Table 4.3. A segment that is shorter than its unstretched length carries no tension, so the tether can go slack. The accelerations are then integrated in time to obtain the new position and velocity of every node. The end nodes are not integrated, since their motion is imposed by the vehicles, and the forces at these nodes are the forces that the tether exerts on them.



(a) Tether divided into nodes and segments. (b) Forces on a node.

Figure 4.7: Lumped-mass model of the tether.

The parameters of the model are the mass per unit length, which gives the mass m_i of each node in (4.3), the axial stiffness EA and the damping ratio, which define the tension in (4.4), the bending stiffness, and the drag coefficients. Their values are given in Table 4.3, together with the number of segments and the time step used to integrate the equations. The mass per unit length and the diameter are based on commercial micro-tethers for UAVs that provide power and data [34, 35], and the axial

stiffness is estimated from the tensile strength of these tethers. The length of the tether is not fixed, and it is changed by the winch described in Section 4.1.5.

Table 4.3: Parameters of the simulated tether.

Parameter	**Value**	**Unit**
Diameter	3	mm
Mass per unit length	0.020	kg/m
Axial stiffness, EA	5×10^4	N
Damping ratio (relative to critical)	0.8	–
Bending stiffness	2×10^{-5}	$\mathrm{N \cdot m^2}$
Transverse drag coefficient	1.2	–
Axial drag coefficient	0.005	–
Density of the surrounding air	1.225	$\mathrm{kg/m^3}$
Number of segments	30	–
Integration time step	0.02	ms

The tether is coupled to the vehicles by a ROS node, as shown in Figure 4.8. At each step, the node reads the position and the velocity of the two attachment points, which are the tether anchor on the deck of the USV and the centre of the body of the UAV, and gives them to MoorDyn. MoorDyn imposes them as the motion of the ends of the tether, integrates the tether over the step, and returns the force vector at each attachment point, which is applied to the vehicles in Gazebo.

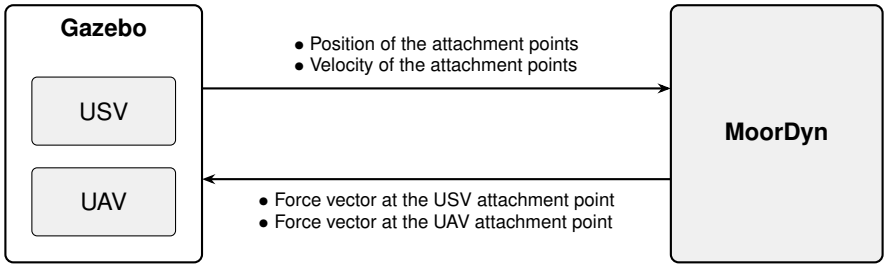


Figure 4.8: Data exchange between the simulation and the tether.

4.1.5 Tether Winch

The model of the tether used by the controllers in Section 3.1 assumes that its slack is kept close to a constant value ε_0, which requires a mechanism that adjusts the length of the tether as the vehicles move. In a real system, this can be achieved by a winch system on the USV, which spools the tether in and out. In simulation, this mechanism is represented by a virtual winch that varies the length of the tether.

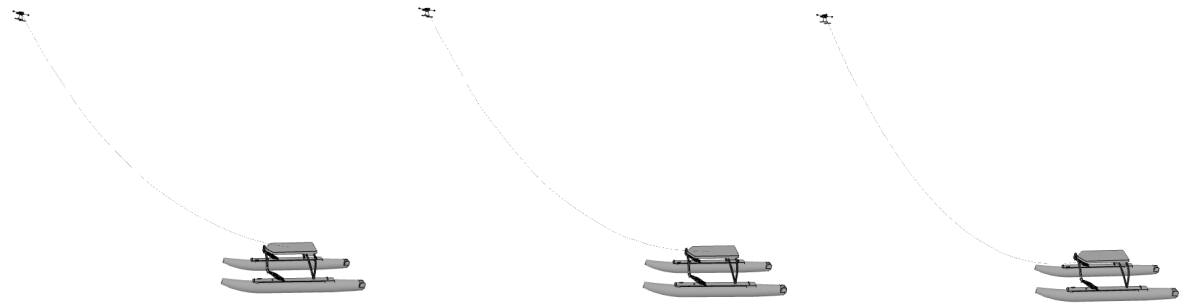
At each step, the virtual winch reads the distance d between the attachment points and computes the tether length L_{ref} that gives the desired slack,

$$L_{\text{ref}} = \frac{d}{1 - \varepsilon_0}, \tag{4.5}$$

which it then applies as the length of the tether in MoorDyn. A real system cannot follow the variations of the length instantly, because of the speed of the winch and the delay in the measurement of the distance.

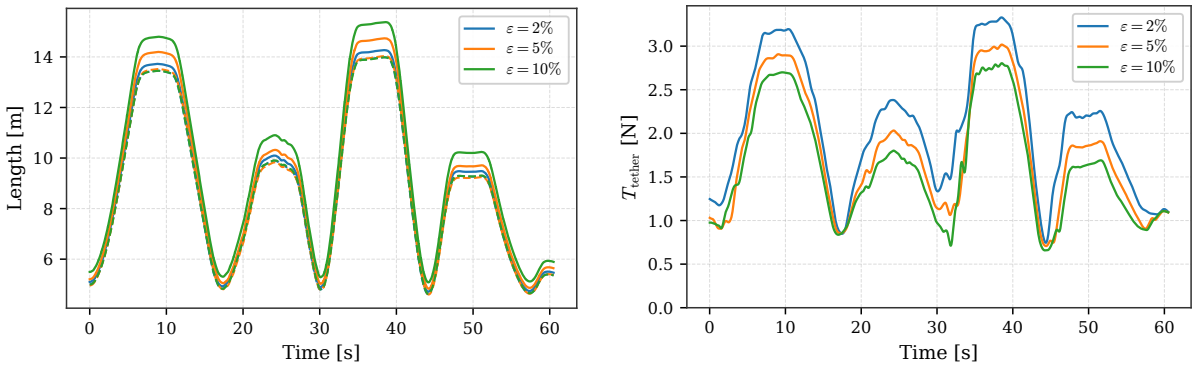
To account for this limitation, the rate of change of the length is limited to $v_{\text{winch}} = 2\,\text{m/s}$, which makes the slack deviate from ε_0 when the distance changes faster than the system can follow.

To evaluate the effect of the slack ratio, the same $60\,\text{s}$ trajectory of the UAV was flown with $\varepsilon_0 = 2\%$, 5%, and 10%. Figure 4.9 shows the shape of the tether for each value, and a larger slack naturally makes the tether sag more between the vehicles. Figure 4.10a shows the distance between the attachment points (dashed) and the length of the tether (solid) along the trajectory. The length follows the distance, staying above it by a margin that grows with ε_0. Figure 4.10b shows the tension at the UAV along the same trajectory. The tension follows the distance, with its peaks coinciding with the largest distances, since the weight of the tether that the UAV supports increases with it. The tension also decreases as the slack increases. Table 4.4 quantifies these results. From $\varepsilon_0 = 2\%$ to 10%, the mean tension decreases from $2.07\,\text{N}$ to $1.62\,\text{N}$, a reduction of 22%, and the maximum from $3.38\,\text{N}$ to $2.88\,\text{N}$.



(a) $\varepsilon_0 = 2\%$. (b) $\varepsilon_0 = 5\%$. (c) $\varepsilon_0 = 10\%$.

Figure 4.9: Shape of the tether for different slack ratios.



(a) Distance and tether length. (b) Tension at the UAV.

Figure 4.10: Tether length and tension for different slack ratios.

Table 4.4: Tension at the UAV for different slack ratios.

ε_0	**Mean** (N)	**Maximum** (N)
2%	2.07	3.38
5%	1.78	3.08
10%	1.62	2.88

These tensions are small compared with the weight of the UAV, which is $20.2\,\text{N}$, so the tension alone

does not determine the choice of ε_0. The choice is instead a compromise on the amount of slack. Some slack is needed to absorb disturbances and avoid sudden tension spikes at the UAV, but too much makes the tether harder to manage and more likely to tangle. A value of $\varepsilon_0 = 5\%$ is used in the remainder of this work.

4.2 Models Validation

Before incorporating the vehicle and tether models into the real-time controllers developed in Chapter 3, their accuracy must be verified against the high-fidelity simulation environment described in Section 4.1 to ensure they capture the dominant dynamics while remaining computationally tractable. Each model is evaluated using the nominal simulator parameters, without fitting them to the validation data, driven by recorded inputs from representative manoeuvres. Model fidelity is quantified by the root-mean-square error (RMSE) in physical units and by the normalised root-mean-square error (NRMSE), defined as the RMSE normalised by the signal range.

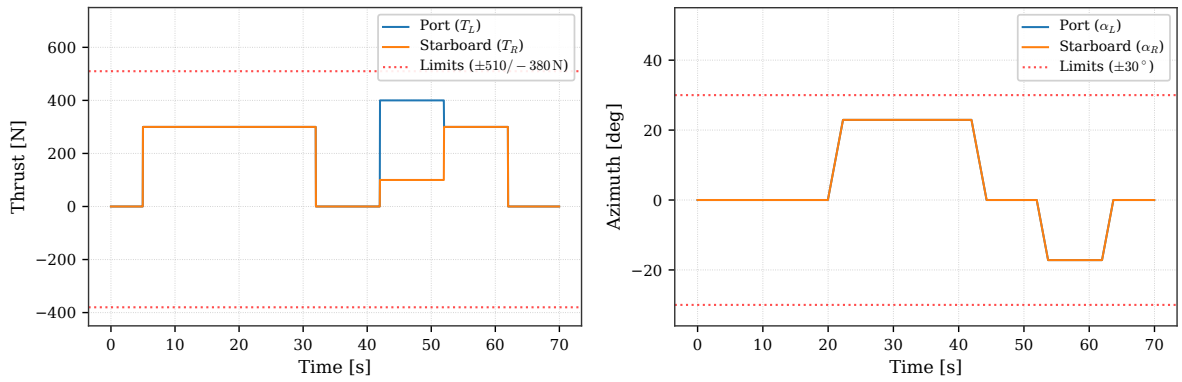
4.2.1 USV Model

The first model to be validated is the one used by the USV NMPC, presented in Section 3.2.3. It predicts how the vessel moves from the commands sent to its thrusters, so its accuracy determines how well the controller can anticipate the motion of the vessel. To compare it with the simulator, the parameters of the model are set to the values of the simulated vessel, described in Section 4.1. These values are listed in Table 4.5.

Table 4.5: Parameters of the USV model.

Parameter	**Symbol**	**Value**
Mass	m	$259.10\,\mathrm{kg}$
Yaw moment of inertia	I_z	$683.22\,\mathrm{kg\cdot m^2}$
Thruster longitudinal arm	x_{arm}	$2.0855\,\mathrm{m}$
Thruster lateral arm	y_{arm}	$1.0271\,\mathrm{m}$
Linear surge damping	X_u	$100.0\,\mathrm{N/(m/s)}$
Quadratic surge damping (forward)	X_{uu}^{+}	$150.0\,\mathrm{N/(m/s)^2}$
Quadratic surge damping (backward)	X_{uu}^{-}	$150.0\,\mathrm{N/(m/s)^2}$
Linear sway damping	Y_v	$100.0\,\mathrm{N/(m/s)}$
Quadratic sway damping	Y_{vv}	$100.0\,\mathrm{N/(m/s)^2}$
Linear yaw damping	N_r	$800.0\,\mathrm{N\cdot m/(rad/s)}$
Quadratic yaw damping	N_{rr}	$800.0\,\mathrm{N\cdot m/(rad/s)^2}$

To compare the model with the simulator, the USV was driven in open loop for $69.96\,\mathrm{s}$ with a sequence of thrust and azimuth commands chosen to excite the three velocities of the model, surge, sway, and yaw rate. Figure 4.11 shows the commands sent to the thrusters, which were recorded together with the resulting motion of the vessel. The model is then given the same commands, starting from the initial state of the vessel, and computes the velocities over the whole run without any correction from the simulator, to be compared with the recorded ones.

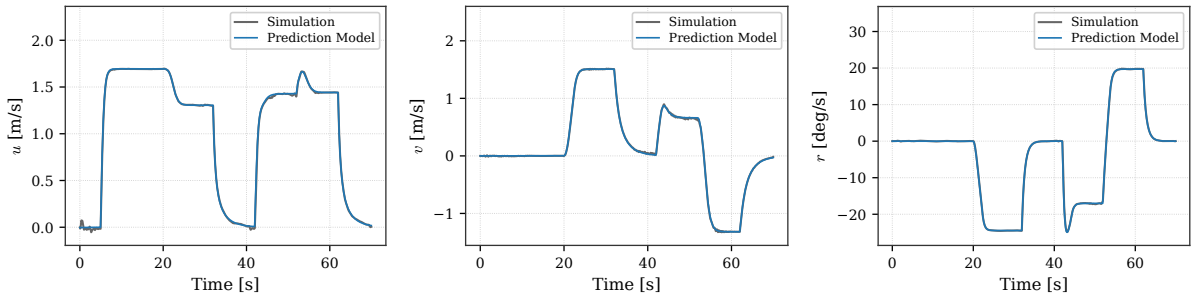


(a) Thrust commands.

(b) Azimuth angle commands.

Figure 4.11: USV thruster inputs.

The dynamics of the model are first tested over the whole run, in a single continuous test that covers all the conditions of the manoeuvre and in which the model never receives the state of the simulator. Figure 4.12 compares the surge velocity u, the sway velocity v, and the yaw rate r of the model with those of the simulator. Only the velocities are compared, because the damping force of the water grows with them and limits their error. The position and the heading are only the integral of the velocities, and would accumulate their small errors over the run. The errors of the three velocities, listed in Table 4.6, are below 1% of their range.



(a) Surge velocity u.

(b) Sway velocity v.

(c) Yaw rate r.

Figure 4.12: USV body-fixed velocities validation.

The NMPC does not use the model over the whole run, but over a prediction horizon of $T_h = 2.0\,\text{s}$ that always starts from the current state of the USV. To evaluate the model in this use, predictions of $2.0\,\text{s}$ were computed along the trajectory of the run, each one starting from the state of the simulator and driven by the recorded commands. Figure 4.13 shows these predictions over the trajectory of the vessel. Since each prediction starts from the state of the simulator, the errors of the position and the heading accumulate only over the horizon, which is the time span of the NMPC prediction.

The error of these predictions grows with how far ahead they are made. Figure 4.14 presents the error of the position, the heading, the linear velocity, and the yaw rate as a function of the prediction horizon, up to $T_h = 2.0\,\text{s}$. The errors grow with the horizon, and at its end they are below 1% of the range of each state, as listed in Table 4.6, and the largest position error represents less than 2% of the length of the vessel, which makes the model suitable for the NMPC.

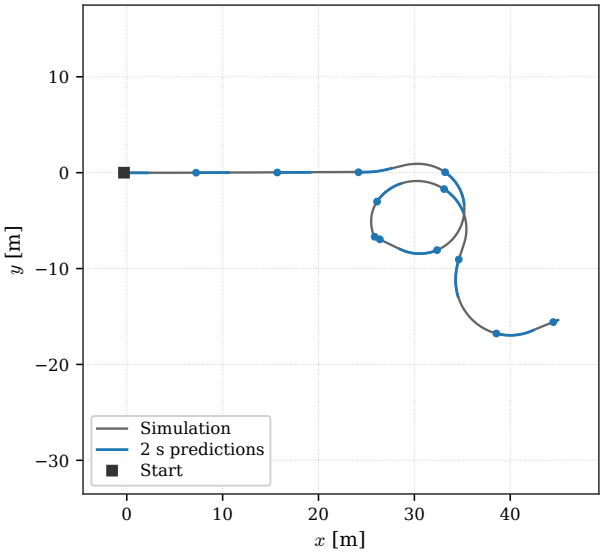
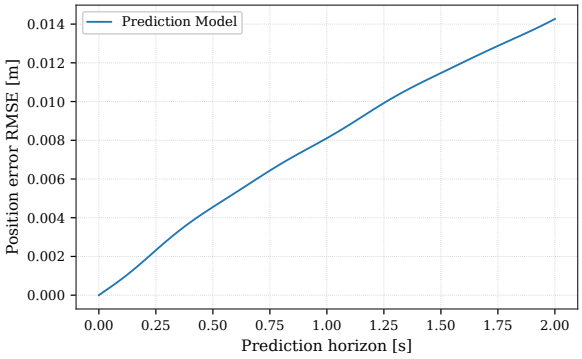
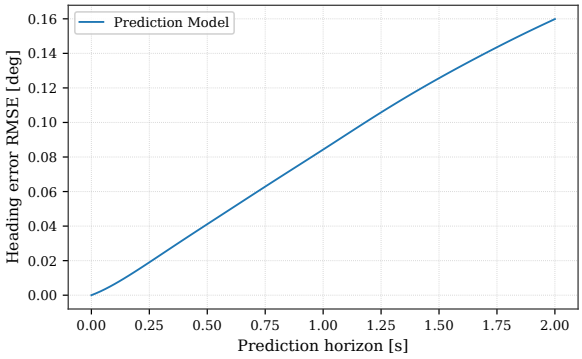


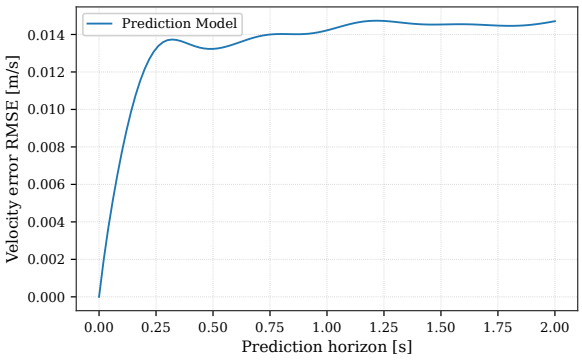
Figure 4.13: USV trajectory and 2.0 s predictions.



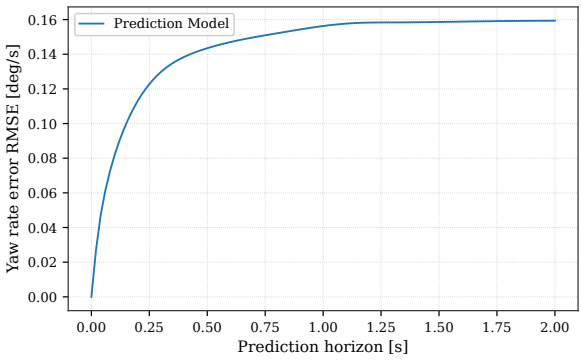
(a) Position error e_p.



(b) Heading error e_ψ.



(c) Linear velocity error e_v.



(d) Yaw rate error e_r.

Figure 4.14: USV prediction errors as a function of the prediction horizon.

Table 4.6: USV model validation errors.

State	Unit	RMSE	NRMSE (%)	Max Error
Velocities over the whole run (69.96 s)				
Surge velocity u	m/s	0.0129	0.74	0.0760
Sway velocity v	m/s	0.0092	0.32	0.0470
Yaw rate r	deg/s	0.1579	0.35	1.8787
Predictions over the horizon ($T_h = 2.0$ s)				
Position error e_p	m	0.0143	0.37	0.0933
Heading error e_ψ	deg	0.1599	0.33	0.8887
Linear velocity error e_v	m/s	0.0147	0.73	0.0756
Yaw rate error e_r	deg/s	0.1594	0.36	1.8723

4.2.2 UAV Model

The second model to be validated is the one used by the UAV NMPC, presented in Section 3.2.4. It predicts how the vehicle moves from the attitude and the thrust that are commanded, so its accuracy determines how well the controller can anticipate that motion. The tether force is left out of the model, so that the motion of the vehicle is validated on its own, and the tether is validated separately in Section 4.2.3. The parameters of the model are obtained from the simulated vehicle, described in Section 4.1.3, and are listed in Table 4.7.

Table 4.7: Parameters of the UAV model.

Parameter	Symbol	Value
Mass	m	$2.06\,\text{kg}$
Gravitational acceleration	g	$9.81\,\text{m/s}^2$
Rotor drag coefficient	d_r	$0.218\,\text{N}\cdot\text{s/m}$
Attitude time constant (roll and pitch)	τ	$0.154\,\text{s}$
Attitude time constant (yaw)	τ_ψ	$0.357\,\text{s}$
Transport delay (roll and pitch)	τ_d	$0.04\,\text{s}$
Transport delay (yaw)	$\tau_{d,\psi}$	$0.06\,\text{s}$

The mass and the gravitational acceleration are taken directly from the simulator, but the other parameters need to be calculated. The simulator applies to each rotor a drag force proportional to both its angular speed and the velocity of the vehicle, through the coefficient $c_{drag} = 8.06 \times 10^{-5}$ of Table 4.2. Around the hovering speed ω_0, at which the thrust of the four rotors balances the weight, the four rotors together give a linear drag on the horizontal velocity,

$$\omega_0 = \sqrt{\frac{mg}{4\,k_F}} \approx 675\,\text{rad/s}, \qquad d_r = 4\,c_{drag}\,\omega_0 \approx 0.218\,\text{N}\cdot\text{s/m}, \tag{4.6}$$

where k_F is the thrust coefficient of the rotors. The nominal attitude time constants are not parameters of the simulator, since they result from the attitude controller of the vehicle. This controller converts the angle error into an angular rate setpoint through a proportional gain, $P_\phi = 6.5\,\text{s}^{-1}$ for roll and pitch and $P_\psi = 2.8\,\text{s}^{-1}$ for yaw. Assuming that the inner loop that tracks the angular rate does so instantly, the

angular rate of the vehicle is equal to the setpoint, and the roll angle ϕ evolves as

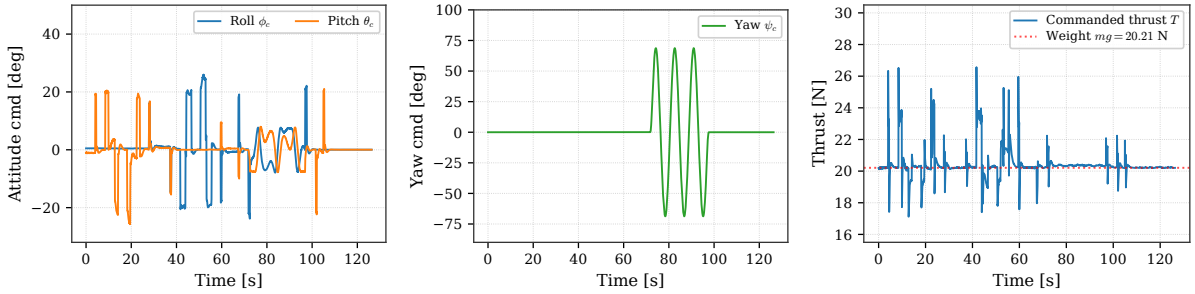
$$\dot{\phi} = P_\phi \, (\phi_c - \phi), \tag{4.7}$$

where ϕ_c is the commanded roll angle, and the same relation holds for the pitch and, with the gain P_ψ, for the yaw. This is a first-order response, in which the attitude changes faster the further it is from the command, and its time constant is the inverse of the gain,

$$\tau = \frac{1}{P_\phi} \approx 0.154\,\text{s}, \qquad \tau_\psi = \frac{1}{P_\psi} \approx 0.357\,\text{s}. \tag{4.8}$$

The attitude controller does not, however, act on the roll and pitch angles separately. It rotates the vehicle so that the direction of the thrust approaches the commanded direction, and it controls the yaw independently, so the time constant τ applies to the direction of the thrust, as in the model of Section 3.2.4. The commands also reach the attitude controller only after a small transport delay. The delays are not parameters of the simulator either, and they were obtained from flights with the NMPC, different from the validation flight, as the values that minimise the error of the attitude predicted over $1\,\text{s}$, giving $\tau_d = 0.04\,\text{s}$ for roll and pitch and $\tau_{d,\psi} = 0.06\,\text{s}$ for yaw.

To compare the model with the simulator, the UAV flew for $126\,\text{s}$ following a reference that excites its motion in all directions, with inclined straight lines followed by a circle in which the yaw oscillates independently of the motion. The reference was tracked by a controller other than the NMPC, so that the commands do not depend on the model being validated. Figure 4.15 shows the attitude and thrust commands generated by this controller, which were recorded together with the resulting motion of the vehicle. The model is given the same commands, and its response is compared with the recorded motion.

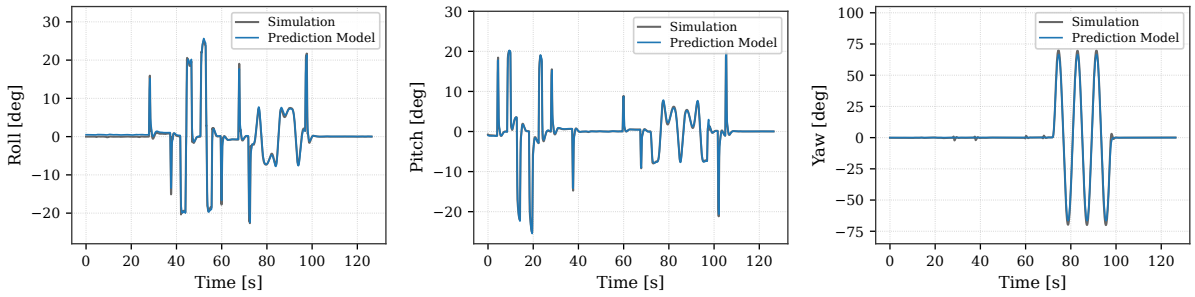


(a) Roll and pitch commands. (b) Yaw command. (c) Thrust command.

Figure 4.15: UAV attitude and thrust commands during the validation flight.

The attitude of the vehicle does not reach the commanded attitude instantly, so the model includes the response of the attitude controller, with the nominal time constants τ and τ_ψ and the transport delays τ_d and $\tau_{d,\psi}$. To test whether it predicts this response correctly, the recorded attitude commands were given to the response of Equation (3.83), and the resulting attitude was compared with the one recorded in the simulator. Figure 4.16 shows, for roll, pitch, and yaw, the attitude of the simulator and the attitude predicted by the model. The errors of the three angles, listed in Table 4.8, are below 2% of their range

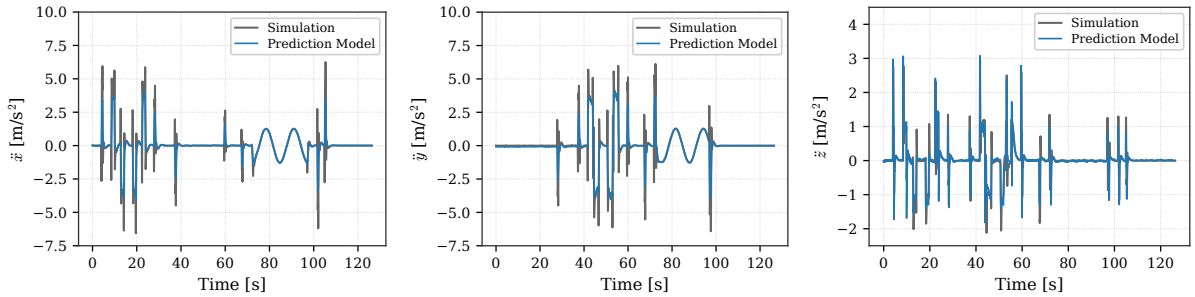
with the delays, against more than 4% if the commanded attitude were taken directly as the attitude of the vehicle.



(a) Roll. (b) Pitch. (c) Yaw.

Figure 4.16: UAV attitude tracking. Simulation and model with the nominal attitude response.

The model is first compared with the simulator on the inertial acceleration of the vehicle, which it computes directly from the attitude, the thrust, and the drag. Since this does not integrate any previous state, the error does not accumulate and can be evaluated over the whole flight. The model is given the recorded thrust and the attitude that it predicts with the delay, together with the velocity recorded in the simulator, which determines the drag. Figure 4.17 compares the acceleration along x, y, and z of the model with that of the simulator. The errors of the three accelerations, listed in Table 4.8, are about 4% of their range.



(a) Along x. (b) Along y. (c) Along z.

Figure 4.17: UAV inertial acceleration.

The NMPC does not use the model over the whole flight, but over a prediction horizon of $T_h = 1.0\,\text{s}$ that always starts from the current state of the vehicle. To evaluate the model in this use, predictions of $1.0\,\text{s}$ were computed along the flight, each one starting from the state of the simulator and driven by the recorded commands. Figure 4.18 shows these predictions over the trajectory of the vehicle. Since the position and the velocity are obtained by integrating the acceleration, their errors accumulate only over the horizon, and they can be compared together with the acceleration.

The error of these predictions grows with how far ahead they are made. Figure 4.19 presents the error of the position and of the velocity as a function of the prediction horizon, up to $T_h = 1.0\,\text{s}$. At the end of the horizon, the errors are below 2.5% of the range of each state, as listed in Table 4.8. For the position, this corresponds to an RMSE of $11\,\text{cm}$ over the up to $8\,\text{m}$ that the vehicle travels in the horizon.

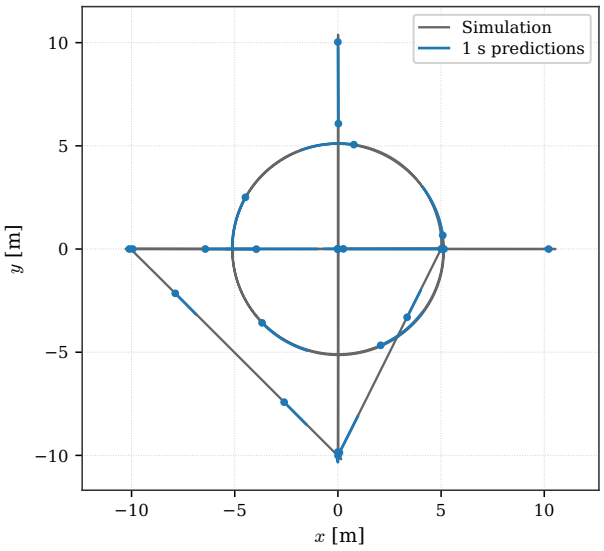
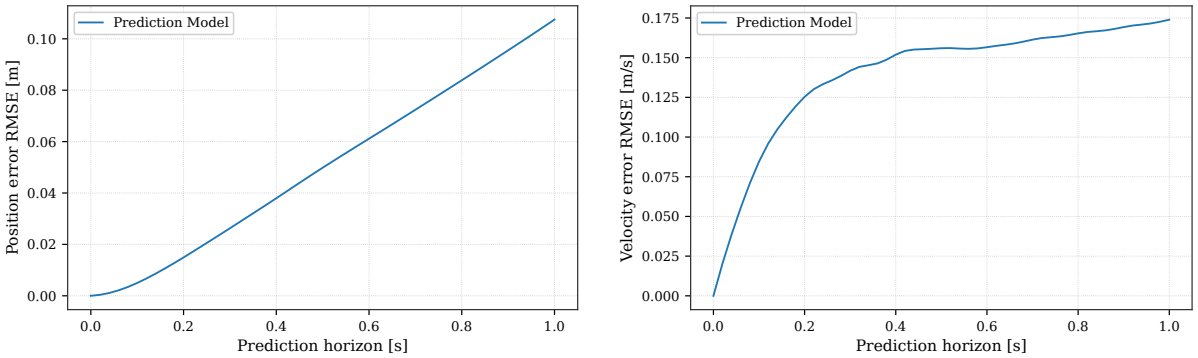


Figure 4.18: UAV trajectory and 1.0 s predictions.

The model is therefore suitable for the NMPC.



(a) Position error.

(b) Velocity error.

Figure 4.19: UAV prediction errors as a function of the prediction horizon.

Table 4.8: UAV model validation errors.

State	Unit	RMSE	NRMSE (%)	Max Error
Attitude over the whole flight (126.32 *s)*				
Roll ϕ, with lag	deg	0.6316	1.32	3.8352
Roll ϕ, without lag	deg	2.4220	5.08	20.1176
Pitch θ, with lag	deg	0.4022	0.89	2.4640
Pitch θ, without lag	deg	2.2981	5.07	19.9585
Yaw ψ, with lag	deg	1.3520	0.96	6.3935
Yaw ψ, without lag	deg	5.7076	4.07	18.7711
Acceleration over the whole flight (126.32 *s)*				
Acceleration $\ddot{x}$	m/s^2	0.4551	3.55	3.3270
Acceleration $\ddot{y}$	m/s^2	0.4969	3.97	3.2636
Acceleration $\ddot{z}$	m/s^2	0.1913	3.77	1.5908
Predictions over the horizon ($T_h = 1.0$ *s)*				
Position error e_p	m	0.1075	1.34	0.7177
Horizontal position error $e_{p,xy}$	m	0.1024	1.31	0.7078
Vertical position error $e_{p,z}$	m	0.0326	1.34	0.2380
Velocity error e_v	m/s	0.1739	2.02	0.8724

4.2.3 Tether Model

The last model to be validated is the tether force model of Section 3.1.3, which gives the force that the tether applies on the UAV from the positions of the two vehicles. The force has a tension, proportional to the distance d between the vehicles, and a direction. The chord, the straight line between the vehicles, would be the direction of the force if the tether were weightless, but the weight of the tether bends it and rotates the force from the chord towards the vertical. The tension is approximately the weight of the tether hanging between the two vehicles, $T_{\text{tether}} = \mu g L$, where μ is the linear mass and L the length of the tether. Since the winch keeps $L = d/(1-\varepsilon_0)$, the tension is proportional to the distance between the vehicles, $T_{\text{tether}} = c_d\, d$, with

The tension is approximately the weight of the tether hanging between the two vehicles, $T = \mu g L$, where μ is the linear mass and L the length of the tether. Since the winch keeps $L = d/(1-\varepsilon_0)$, the tension is proportional to the distance between the vehicles, $T = c_d\, d$, with

$$c_d = \frac{\mu g}{1-\varepsilon_0} = \frac{0.020 \times 9.81}{1-0.05} = 0.2065\,\text{N/m}. \tag{4.9}$$

The deflection of the force is largest when the chord is horizontal and decreases as it approaches the vertical. For a horizontal chord, the weight bends the tether into a catenary [36], and the deflection γ_0 is the angle that the tether makes with the chord at its ends. For a tether of length L between two points at distance d, it is given by

$$\gamma_0 = \arctan\left(\sinh u\right), \qquad \frac{\sinh u}{u} = \frac{L}{d} = \frac{1}{1-\varepsilon_0}, \tag{4.10}$$

where u is the positive solution of the second equation, obtained iteratively. For $\varepsilon_0 = 5\%$, this gives $u = 0.558$ and $\gamma_0 = 30.4°$. The parameters of the model are listed in Table 4.9.

Table 4.9: Parameters of the tether model.

Parameter	Symbol	Value
Linear mass	μ	0.020 kg/m
Slack ratio	ε_0	5%
Tension per unit of distance	c_d	0.2065 N/m
Deflection for a horizontal chord	γ_0	$30.4°$

The tether force was recorded in the simulator during a flight of 60.5 s, in which the UAV moved along x, and the distance between the two vehicles varied between 4.6 m and 14.0 m. The winch regulated the slack to ε_0. The reference is the force computed by MoorDyn and applied on the UAV in Gazebo described in Section 4.1. At each instant, the model receives the positions of the two vehicles and computes the force, which is compared with the reference in tension and in its components in the inertial frame. The comparison is also made for two simpler descriptions of the force, which differ from the model only in its direction: the tether hangs from the UAV as a vertical load, or it is taut along the chord. Since the models give the force directly from the positions, there is no initial state and no error accumulates, so the whole flight is compared instant by instant.

Figure 4.20 compares the tension of the model with that of the simulator along the flight, and Figure 4.21 compares the components of the force on the UAV along x and z. The model follows the tension and the components, with errors between 4% and 11% of their range, as listed in Table 4.10.

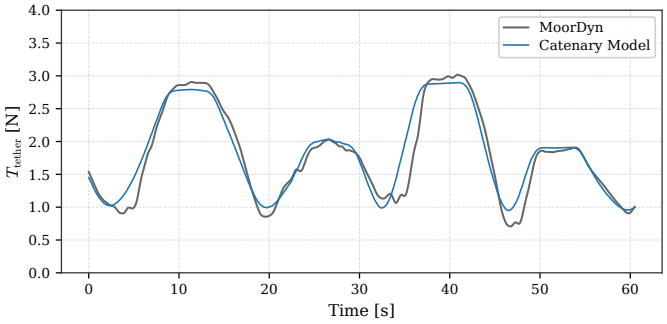
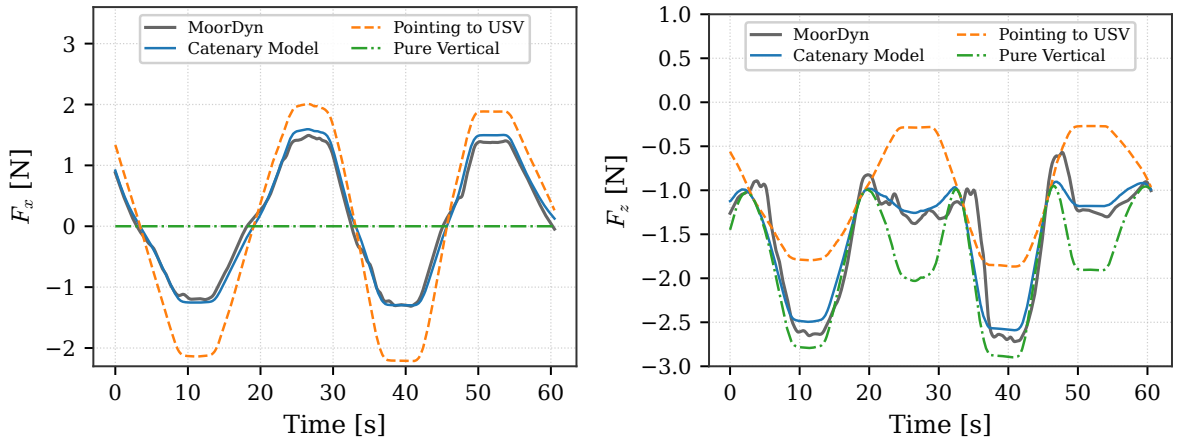


Figure 4.20: Tether tension of the model and of the simulator.

Figure 4.22 shows, for each model, the error of the force vector along the flight, combining its components, and Table 4.10 lists the corresponding errors. Since the three models use the same tension, their errors differ only through the direction of the force. The model reduces the error of the force by 72% relative to the taut tether and by 76% relative to the vertical load, and its error of 0.25 N represents 1.2% of the weight of the UAV, which is 20.2 N.



(a) Force along x.

(b) Force along z.

Figure 4.21: Tether force components at the UAV.

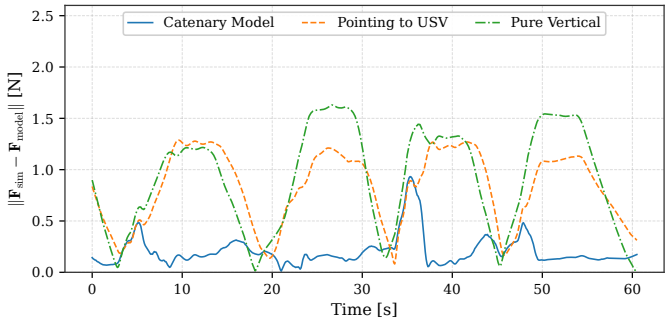


Figure 4.22: Error of the force vector of the tether models.

Table 4.10: Tether force model errors.

Quantity	**Unit**	**RMSE**	**NRMSE (%)**
Model			
Tension T_{tether}	N	0.2031	8.79
Force F_x	N	0.1080	3.85
Force F_z	N	0.2259	10.50
Error of the force vector			
Model	N	0.2505	10.84
Taut tether ($\gamma_0 = 0°$)	N	0.8991	38.91
Vertical load	N	1.0332	44.72

4.3 Control Implementation

With the simulation environment of Section 4.1 developed and the models validated against it in Section 4.2, the next step is to implement the controllers of Chapter 3, which use these models to drive both vehicles.

4.3.1 Optimization Framework

The controllers of Chapter 3 must solve an optimisation problem at every sampling instant, within the time available before the next one. To make this possible, they are implemented with two complementary tools. The problem is first written in CasADi [37], which describes the models, the cost, and the constraints as symbolic expressions and obtains their derivatives automatically. From this description, acados [38] generates a solver in C dedicated to that specific problem, which is compiled once, before operation. During operation, each controller only runs this compiled solver, which is much faster than a general-purpose optimiser.

The prediction model requires integrating the vehicle models over each sampling interval, which is done with the fourth-order Runge–Kutta method [39]. Since the models are nonlinear, the resulting problem cannot be solved directly, and acados approaches the solution iteratively, solving at each iteration a simpler, quadratic approximation of the problem [40]. More iterations bring the result closer to the solution, but take more time.

4.3.2 USV Control

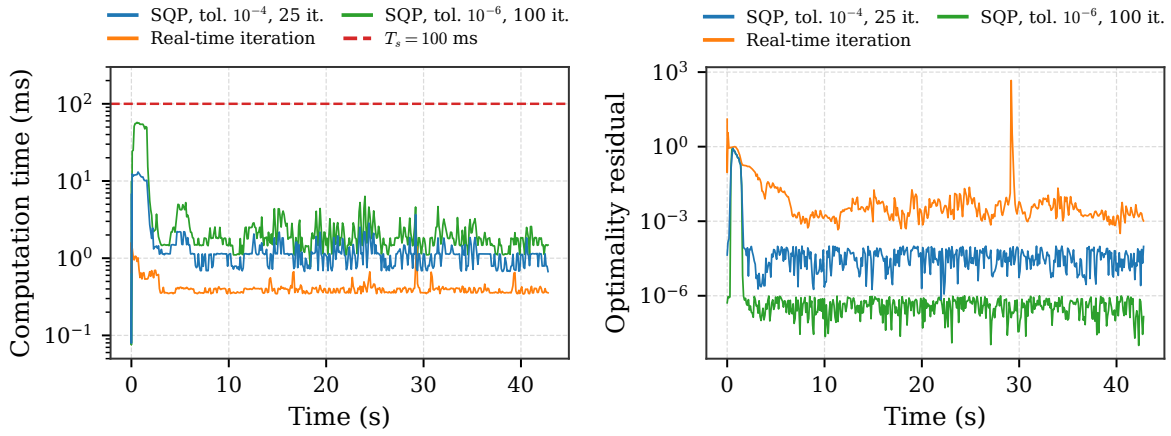
Most limits of the controller are not design choices, but the limits of the thrusters of the simulated WAM-V, described in Section 4.1.2, on the thrust, on the azimuth angle, and on how fast each can change. The remaining two come from how the system is meant to operate. The minimum surge speed keeps the vessel sailing bow first, while still allowing the slow backing needed to hold a position or to brake, and the maximum separation defines the working radius of the UAV around the USV. All parameters of the controller are listed in Table 4.12.

The sampling time, which sets how often the controller computes a new command, and the prediction horizon, which sets how far ahead it plans, are both chosen from how fast the USV responds to its commands. At the operating speeds, the hull takes between half a second and a second to react to a change of force. The controller runs at $10\,\text{Hz}$, so it computes a new command every $0.1\,\text{s}$, five to ten while the vessel responds to a single one, and can correct this response as it develops. The horizon must be long enough for the controller to see most of this response, and is set to $2\,\text{s}$, or 20 steps, which covers two to four times the response time and is the horizon over which the model was validated in Section 4.2.

The weights of the cost set what the controller values most, and were tuned in simulation until the vessel followed its references smoothly. They form a baseline for all the missions of this work, which could be adapted to each manoeuvre, favouring precision when holding a station or smoothness on a

long transit. The position and the heading dominate the cost, since following the reference is the purpose of the controller. The sway velocity is also penalised, so that the vessel does not drift sideways, and the thrust and its rate of change receive a light penalty that avoids unnecessary effort without compromising the tracking. The steering needs little weight to remain smooth, since the azimuth rate limit already prevents abrupt changes of direction.

Since the solver approaches the solution iteratively, its settings trade the quality of the solution for computation time. To choose them, three settings were compared on a recorded run, on a laptop with an Intel i5-1135G7 processor (4 cores, 2.4 GHz), through their iterations, computation time, and optimality residual, which measures how far the command is from satisfying the conditions that define a solution. The results are shown in Figure 4.23, and summarised in Table 4.11 after the first 2 s, once the solver has stabilised. A strict tolerance gives the smallest residual but needs the most iterations, a single iteration per instant, known as RTI [41], is the fastest but leaves the command furthest from the solution, and a moderate tolerance lies between the two. The controller uses this moderate tolerance, of 10^{-4}, since a stricter one changes the applied thrust by less than 0.1 N and the azimuth by less than $0.01°$, with a limit of 25 iterations that keeps even the first instants of the run well within the 100 ms available.



(a) Computation time.

(b) Optimality residual.

Figure 4.23: USV NMPC computation time and optimality residual for different solver settings.

Table 4.11: USV NMPC solver statistics for different solver settings.

	Iterations		**Time (ms)**		**Residual**	
Settings	Mean	Max	Mean	Max	Mean	Max
Tolerance 10^{-6}, 100 iterations	5.3	14	2.0	6.3	4×10^{-7}	1×10^{-6}
Tolerance 10^{-4}, 25 iterations	3.0	7	1.1	3.7	4×10^{-5}	1×10^{-4}
Single iteration	1	1	0.4	1.4	1.1	5×10^{2}

The solver may also fail to return a usable command, for instance when the solution of the previous instant is a poor starting point for the current one. In that case, the solver is restarted from the current state of the vessel and solves the problem once more. If it fails again, the thrusters keep the last command, and the UAV keeps the last predicted trajectory of the USV, until a new solution is found. Reaching the iteration limit is not treated as a failure, and the command reached is still used. In the

recorded run, the solver never failed, and the limit was reached only in the first instants of the run.

Table 4.12: Parameters of the USV NMPC.

Parameter	Symbol	Value
Limits		
Thrust limits	$T_{L,max} = T_{R,max}$	$510.0\,\mathrm{N}$
	$T_{L,min} = T_{R,min}$	$-380.0\,\mathrm{N}$
Azimuth limits	$\alpha_{L,max} = \alpha_{R,max}$	$30°$
	$\alpha_{L,min} = \alpha_{R,min}$	$-30°$
Thrust rate limit	$\dot{T}_{L,max} = \dot{T}_{R,max}$	$500\,\mathrm{N/s}$
Azimuth rate limit	$\dot{\alpha}_{L,max} = \dot{\alpha}_{R,max}$	$10°/\mathrm{s}$
Minimum surge speed	u_{min}	$-0.3\,\mathrm{m/s}$
Surge slack penalty	$\mu_{u,1},\ \mu_{u,2}$	$200,\ 2000$
Maximum tether separation	d_{max}	$15.0\,\mathrm{m}$
Tether slack penalty	$\mu_1,\ \mu_2$	$500,\ 5000$
Sampling and horizon		
Sampling time	T_s	$0.1\,\mathrm{s}$
Prediction horizon	N	20
Nodes covered by the UAV trajectory	N_c	10
Cost weights		
Position weight	Q_p	$\mathrm{diag}(150,\ 150)$
Heading weight	q_ψ	150
Velocity weight	Q_ν	$\mathrm{diag}(2,\ 40,\ 1)$
Thruster command weight	Q_τ	$\mathrm{diag}(10^{-5},\ 10^{-5},\ 0.1,\ 0.1)$
Thruster rate weight	R	$\mathrm{diag}(10^{-5},\ 10^{-5},\ 10,\ 10)$

4.3.3 UAV Control

Unlike the USV, the UAV is not driven directly by its controller, but through the PX4 autopilot [29]. PX4 can follow commands given at different levels, from positions to the speeds of the rotors, and the controller uses it at the attitude level. At every sampling instant, it sends the commanded attitude and thrust, and PX4 turns them into rotor speeds through its own attitude controller. The NMPC does not need to know how this is done, only how quickly the attitude follows the command, and these dynamics are already included in its prediction model, as described in Section 3.2.4.

The NMPC computes the thrust in newtons, while PX4 expects a throttle δ_T between zero and one, which sets the speed of the rotors. This conversion is usually based on the hover throttle that PX4 estimates in flight, but here that estimate also absorbs the pull of the tether, which the NMPC already accounts for. Since the vehicle is powered through the tether, its supply voltage does not drop during the flight as that of a battery would, which is one of the main causes of variation of this relation, and any remaining deviation is compensated by the disturbance observer. The conversion therefore uses the theoretical relation of the rotors. The total thrust of the four rotors grows with the square of their speed ω, and the throttle sets this speed linearly between ω_{min} and ω_{max},

$$T = 4\,k_F\,\omega^2, \qquad \delta_T = \frac{\omega - \omega_{min}}{\omega_{max} - \omega_{min}}, \tag{4.11}$$

where k_F, ω_{min}, and ω_{max} are given in Table 4.2. Solving the first relation for the speed and replacing it

in the second gives the throttle sent to PX4 for the thrust T computed by the NMPC,

$$\delta_T = \frac{\sqrt{T/(4\,k_F)} - \omega_{min}}{\omega_{max} - \omega_{min}}, \tag{4.12}$$

which is limited to the interval $[0, 1]$.

As for the USV, some limits of the controller come from the vehicle itself. The thrust is kept above a small minimum, so that the rotors do not stop during descents, and its maximum is set slightly below what the rotors produce at full speed, since rotating the vehicle requires some rotors to spachas quin faster than others, and this margin leaves them room to do so. When the margin is not enough, the mixer of PX4, which distributes the commands among the rotors, gives priority to the attitude and reduces the total thrust instead. Representing each rotor in the model would let the NMPC account for this directly, but it would have to command the rotors itself, bypassing the attitude controller of PX4. The rate at which the separation between the UAV and the USV can grow is limited to the speed of the winch, described in Section 4.1.5, and the maximum separation is the same as in the USV controller. The remaining bounds, on the altitude, the speed, the tilt, and the rates of the commands, define the operating envelope of the vehicle, and were chosen for the intended operation, within the capabilities of the vehicle. All parameters of the controller are listed in Table 4.14.

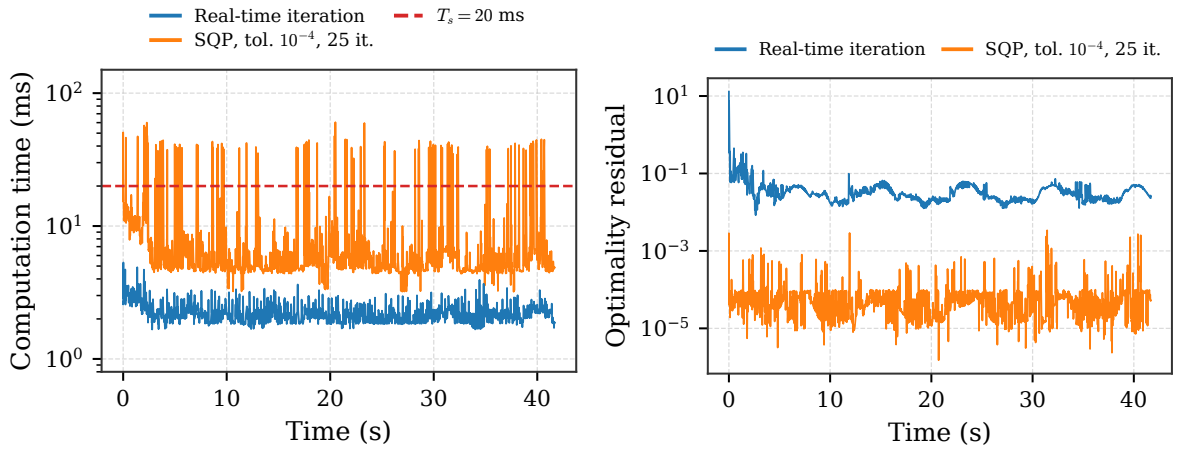
The sampling time and the prediction horizon of the UAV controller follow from how fast the vehicle responds, which is much faster than the USV. Its roll and pitch respond to a new command within about $0.2\,$s, the delay and the time constant of Table 4.7 together. The controller runs at $50\,$Hz, the rate at which the state estimate of the vehicle is received, so it computes a new command every $0.02\,$s, about ten while the vehicle responds to a single one, and can correct this response as it develops. The horizon is set to $1\,$s, or 50 steps, which covers about five times this response and more than twice the slower response of the yaw, and is the horizon over which the model was validated.

The weights of the cost were also tuned in simulation, until the vehicle followed its references closely with smooth commands, and are likewise a baseline that could favour precision when landing or smoothness when following a target. The position dominates the cost, with a larger weight on the height, since the thrust corrects it directly, whereas a horizontal correction requires the vehicle to tilt first and a lighter weight keeps this tilting gradual. The yaw is tracked as well, independently of the position, since the vehicle can turn without moving. Among the inputs, the rates of the roll and pitch commands are penalised the most, while the thrust may change quickly within its rate limit. The tilt and the deviation of the thrust from the hover value are lightly penalised to discourage values larger than the tracking requires.

The disturbance observer of Chapter 3.2 is set by its gain and its bound. With $\beta = 0.03$, the estimate converges with a time constant of about $T_s/\beta \approx 0.7\,$s, slower than the response of the attitude, so that it does not react to the short prediction errors of a manoeuvre, but fast enough to follow the gusts of the wind, which vary over several seconds. The bound of $8\,$N is above the force of a strong wind on the vehicle, of about $5\,$N at $12\,$m/s.

The same comparison was made for the UAV, close to its limits, between iterating until a tolerance of 10^{-4}, with at most 25 iterations, as in the USV, and the RTI, which relies on the solution changing

little between consecutive instants, so that refining the previous one once is enough to follow it. Iterating towards convergence leaves a residual several hundred times smaller, but it exceeded the $20\,\text{ms}$ available in about 6% of the instants, most of them ending at the iteration limit without converging, as shown in Figure 4.24 and Table 4.13. The single iteration never took more than $5\,\text{ms}$, and its commands differed from the converged ones by less than $0.7°$ in roll and pitch and $0.6\,\text{N}$ in thrust, making the RTI the strategy used by the controller.



(a) Computation time.

(b) Optimality residual.

Figure 4.24: UAV NMPC computation time and optimality residual for different solver settings.

Table 4.13: UAV NMPC solver statistics for different solver settings.

	Iterations		**Time (ms)**		**Residual**	
Settings	Mean	Max	Mean	Max	Mean	Max
Tolerance 10^{-4}, 25 iterations	4.6	25	7.7	60.4	8×10^{-5}	3×10^{-3}
RTI	1	1	2.2	4.7	3×10^{-2}	2×10^{-1}

If the solver fails to return a usable command, the vehicle keeps the last attitude and thrust commanded until a new solution is found, while the disturbance estimate is held, since there is no prediction to compare with the measured velocity. With a single iteration there is no iteration limit to reach, and in the recorded run the RTI never failed.

Table 4.14: Parameters of the UAV NMPC.

Parameter	Symbol	Value
Limits		
Thrust limits	$T_{min},\ T_{max}$	$2.02\,\text{N},\ 39.96\,\text{N}$
Thrust rate limit	$\dot{T}_{max}$	$30\,\text{N/s}$
Tilt limits	$\phi_{max},\ \theta_{max}$	$0.5\,\text{rad}$
Attitude command rate limits	$\dot{\phi}_{max},\ \dot{\theta}_{max},\ \dot{\psi}_{max}$	$0.8,\ 0.8,\ 1.0\,\text{rad/s}$
Horizontal and vertical speed limits	$v_{xy,max},\ v_{z,max}$	$10.0\,\text{m/s}$
Minimum altitude	z_{min}	$2.0\,\text{m}$
Slack penalty, bounds h_1–h_5	$\mu_{1,j},\ \mu_{2,j}$	$100,\ 500$
Maximum tether separation	d_{max}	$15.0\,\text{m}$
Maximum tether pay-out rate	$\dot{d}_{max}$	$2.0\,\text{m/s}$
Slack penalty, tether h_6, h_7	$\mu_{1,j},\ \mu_{2,j}$	$500,\ 2000$
Sampling and horizon		
Sampling time	T_s	$0.02\,\text{s}$
Prediction horizon	N	50
Cost weights		
Position weight	Q_p	$\text{diag}(60,\ 60,\ 150)$
Velocity weight	Q_v	$\text{diag}(5,\ 5,\ 7.5)$
Tilt weight	Q_{tilt}	$\text{diag}(5,\ 5)$
Yaw weight	q_ψ	20
Thrust weight	Q_T	0.118
Input weight	R	$\text{diag}(30,\ 30,\ 5,\ 5 \times 10^{-4})$
Disturbance observer		
Observer gain	β	0.03
Disturbance bound	$\bar{d}$	$8\,\text{N}$

4.4 Manoeuvres and Missions

The controllers of Section 4.3 follow the references they are given, but do not decide where each vehicle should go. This is the role of the manoeuvres, which turn a simple motion, such as an orbit around the USV or a landing on its deck, into the reference of each controller at every instant. The missions then chain these manoeuvres, for both vehicles, into a complete operation.

4.4.1 Manoeuvres

A manoeuvre gives the reference of one vehicle as a function of time, sampled over the horizon of the controller at every instant. Five manoeuvres were defined for the UAV, takeoff, formation, orbit, chase and landing, and four for the USV, which holds its position, follows a route of waypoints, pursues a target, keeping a safe distance from it, or supports the UAV. Each is set by a few parameters, such as a destination or a speed. The references are simple geometric curves in time, such as lines and circles, since the controllers keep the vehicles within their limits whatever reference they are given, even a single point.

Most manoeuvres of the UAV are given relative to the winch, since the vehicle works around the USV and must return to it. Their references move with the USV along the trajectory predicted by its controller, either in fixed axes or in the axes of the USV, so that they turn with it, as when the UAV scouts ahead

of the bow. The height is taken above the mean height of the winch, so that the UAV does not follow the heave of the deck. The point of the formation and the centre of the orbit can also be fixed in the world, to reach or circle a structure, and the chase follows the target.

When the UAV works around a structure, the tether may not reach far enough for it to go all the way around. The USV then supports it, taking as its reference the point on the water below the reference of the UAV, pushed away from the structure to a safe distance, so that it follows the UAV and gives it the reach of the tether. This is only possible when the reference of the UAV is fixed in the world, since a reference given relative to the winch would move with the USV, and each vehicle would then chase the other.

Each manoeuvre also sets the heading of the UAV, through a point that the vehicle looks at, or away from, such as the winch, a structure or a target, so that a camera on the vehicle keeps it in view. For the takeoff and the landing it follows instead the heading of the USV, which aligns the vehicle with the deck.

Figure 4.25 shows the manoeuvres of the UAV as performed in the missions of Section 4.4.2, seen from above in fixed axes, with the two vehicles drawn at the same instants, the older ones more transparent, so that their motion can be seen together. The formation (b) keeps the UAV ahead of the bow, in the axes of the USV, while it follows a route, and the orbit (c) circles the USV as it sails, shown in its axes, where the orbit is a circle, while in (d) the UAV orbits the USV on patrol until it detects a target, which it then chases (e) as the USV pursues it. The takeoff and the landing (a, f) are shown instead through the height of the UAV above the mean height of the winch, which shows the heave of the deck.

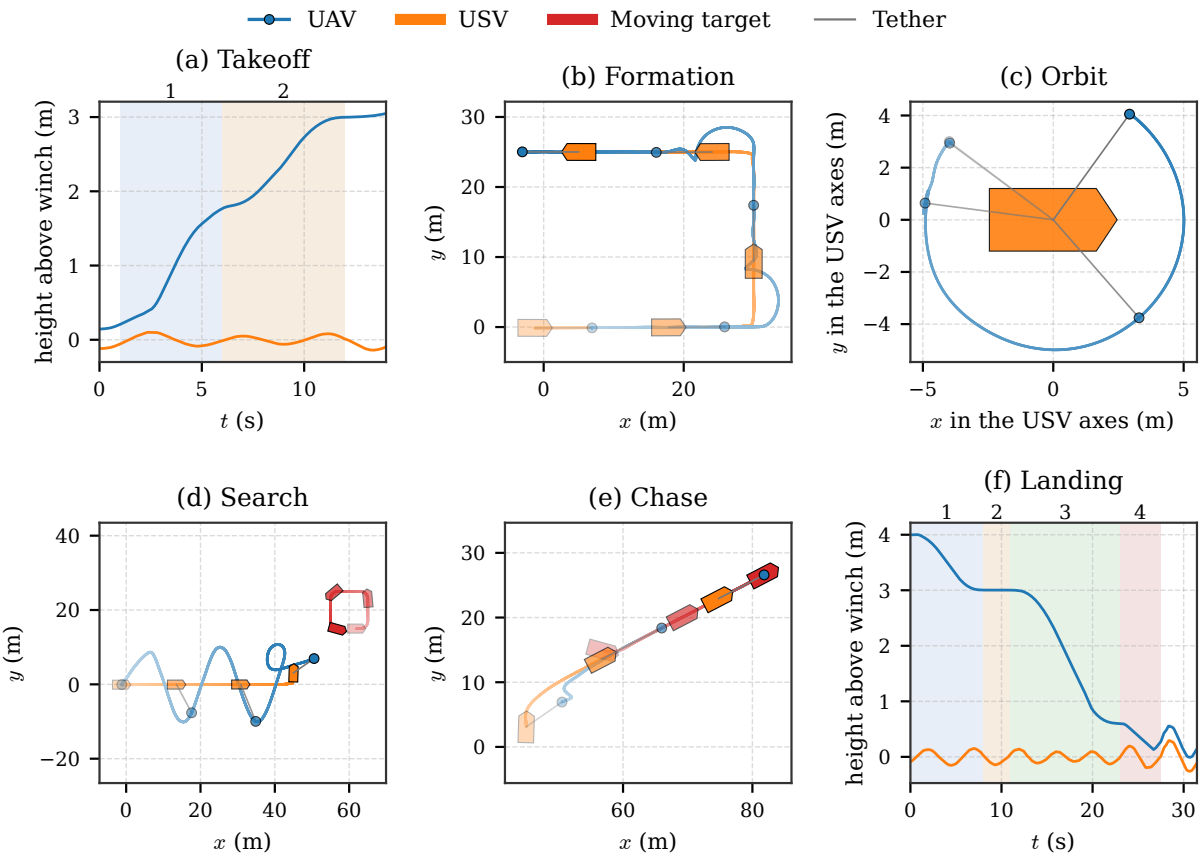


Figure 4.25: Manoeuvres performed in simulation.

The takeoff and the landing are the manoeuvres in which the UAV is closest to the deck and moves with it. To hold the vehicle against the deck, so that the waves do not tip it over and throw it into the sea, the winch can pull the tether with a constant force, which it applies at the start of the takeoff and keeps until the UAV is clear of the deck. The takeoff therefore begins with a vertical climb off the pad, phase 1 in Figure 4.25a, and only then moves the vehicle to its first position, phase 2, so that it does not drift over the edge of the deck at low height.

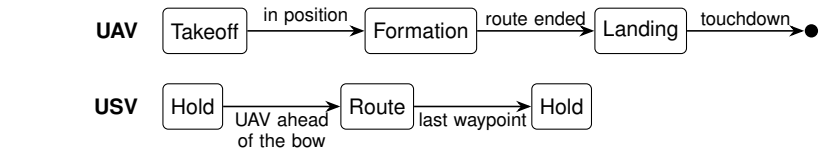
The landing is divided into four phases, shown in Figure 4.25f. The UAV first moves above the winch (1) and waits until it is centred above the pad and moving with it (2). It then descends with the winch pulling it towards the deck (3), and ends with a slow final approach towards a point below the deck (4), so that it touches down even if the deck falls away, and the speed of the contact is set by the heave rather than by the descent. Once it rests on the pad, the vehicle is disarmed, and the winch keeps it against the deck.

4.4.2 Missions

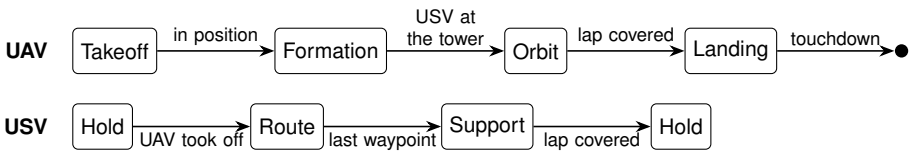
A mission is a file with two sequences of manoeuvres, one for each vehicle. Each manoeuvre ends when a condition is met, either an event, such as the detection of a target, or the end of a manoeuvre of the other vehicle.

A mission is aborted when a manoeuvre fails, when it takes too long, or when the operator requests it. The USV then holds its position and the UAV lands.

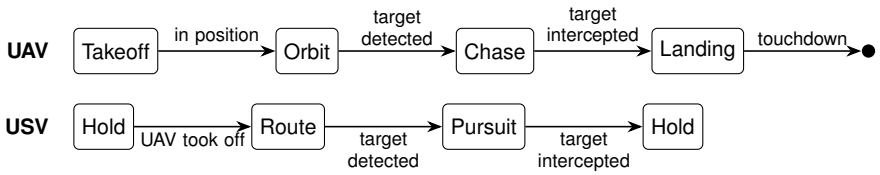
Three missions were tested in simulation, shown in Figure 4.26. In the scout (a), the UAV flies ahead of the bow. In the inspection (b), it circles a tower while the USV supports it. In the patrol (c), it orbits the USV until it detects a target, which it then chases while the USV pursues it.



(a) Scout.



(b) Inspection.



(c) Patrol.

Figure 4.26: Missions tested in simulation.

4.5 Software Architecture and Mission Control

4.5.1 Architecture

The software is built on ROS 2 [42], in which each component runs as an independent node, and the nodes exchange messages through named topics. The controllers therefore do not depend on whether their data comes from the simulation or from the real vehicles.

Figure 4.27 shows the components and the main data exchanged between them. Gazebo simulates the vehicles and the waves, while PX4 runs as the autopilot of the UAV, in lockstep with the simulation. Each is connected to ROS 2 by a bridge, the ROS–Gazebo bridge [43] and the Micro XRCE-DDS agent [44]. The tether node applies in Gazebo the force of the tether, computed by MoorDyn [32] for the length set by the winch. The two NMPC controllers send their commands to the thrusters of the USV and to the autopilot of the UAV while exchanging their predicted trajectories, and the mission executor produces the references of both. A web server connects ROS 2 to a Mission Control Centre.

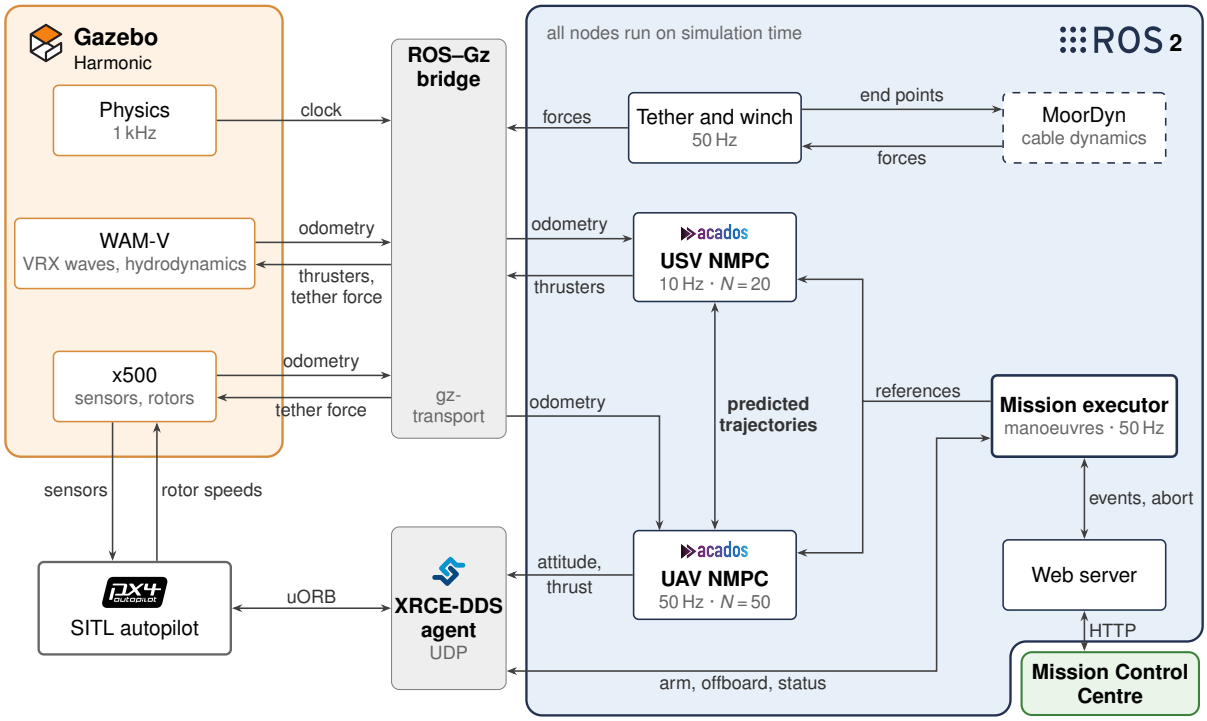


Figure 4.27: Software architecture.

4.5.2 Mission Control Centre

The Mission Control Centre is a web page, shown in Figure 4.28, from which an operator runs the whole system without the command line. Since every mission is launched and recorded in the same way, it also made the tests in simulation repeatable. It is organised in four parts.

- **Simulation**, at the top, starts and stops the simulation, with or without the graphical interface of Gazebo.

- **Missions**, on the left, lists the missions with their manoeuvres, starts one, optionally recording its data, and aborts it.
- **Map**, in the centre, shows the two vehicles, the tether and their references from above.
- **Telemetry and progress**, on the right, show the state of each vehicle and of the tether, and the manoeuvres of each vehicle already done, running and still to come.

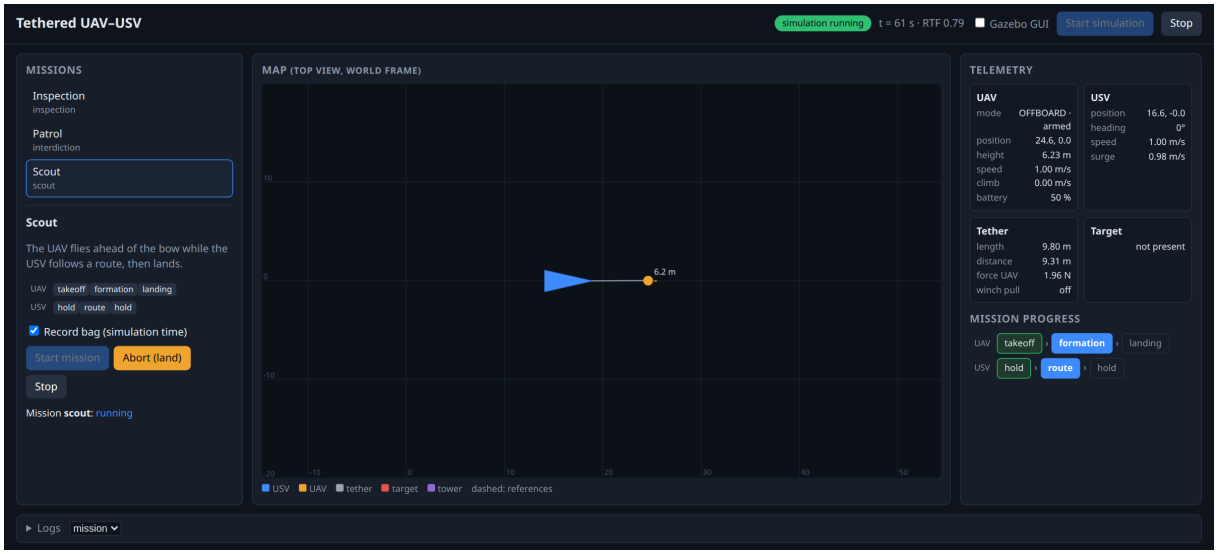


Figure 4.28: Mission Control Centre during the scout mission.

Chapter 5

Results

5.1 Trajectory Tracking

5.1.1 Nominal Tracking

5.1.2 Tether Constraint

5.1.3 Actuator Limits

5.2 Missions

5.2.1 Scout

5.2.2 Inspection

5.2.3 Patrol

5.3 Disturbances

5.3.1 Waves

5.3.2 Wind

Chapter 6

Conclusions

6.1 Achievements

6.2 Future Work

Bibliography

[1] E. M. S. Agency. EMSA Outlook 2025_v3.0, 2025.

[2] G. Briguglio and V. Crupi. Review on Sensors for Sustainable and Safe Maritime Mobility. *Journal of Marine Science and Engineering*, 12(2):353, Feb. 2024. ISSN 2077-1312. doi: 10.3390/jmse12020353.

[3] R. Dunley. Uncrewed naval vessels and the span of maritime tasks. *Marine Policy*, 149:105481, Mar. 2023. ISSN 0308-597X. doi: 10.1016/j.marpol.2023.105481.

[4] B. Şengül, F. Yılmaz, and Ö. Uğurlu. Safety–Security Analysis of Maritime Surveillance Systems in Critical Marine Areas. *Sustainability*, 15(23):16381, Jan. 2023. ISSN 2071-1050. doi: 10.3390/su152316381.

[5] N. Nomikos, P. K. Gkonis, P. S. Bithas, and P. Trakadas. A Survey on UAV-Aided Maritime Communications: Deployment Considerations, Applications, and Future Challenges. *IEEE Open Journal of the Communications Society*, 4:56–78, 2023. ISSN 2644-125X. doi: 10.1109/OJCOMS.2022.3225590.

[6] S. Bauk, N. Kapidani, L. Sousa, Ž. Lukšić, and A. Spuža. Advantages and disadvantages of some unmanned aerial vehicles deployed in maritime surveillance. *Maritime Transport Conference*, (8), 2020. ISSN 2938-8813.

[7] F. Fattori and S. Cocuzza. Tethered Drones: A Comprehensive Review of Technologies, Challenges, and Applications. *Drones*, 9(6):425, June 2025. ISSN 2504-446X. doi: 10.3390/drones9060425.

[8] Exail Technologies and Elistair. Exail and Elistair partner to offer tethered UAV capability on DriX USV. Exail Press Release, 2024. URL `https://www.exail.com/news/exail-partners-with-elistair-to-provide-tethered-uav-for-drix-usv/`. Accessed: 2026-08-05.

[9] ACUA Ocean and Teledyne FLIR Defense. ACUA Ocean and Teledyne FLIR Defense demonstrate integrated tethered UAS and USV for UK Ministry of Defence. Defense Advancement News, 2024. URL `https://www.defenseadvancement.com/news/acua-ocean-teledyne-flir-demonstrate-integrated-uav-usv/`. Accessed: 2026-08-05.

[10] C. W. de Silva. *Modeling of Dynamic Systems with Engineering Applications*. CRC Press, July 2022. ISBN 978-1-000-61439-8.

[11] A. K. Tangirala. *Principles of System Identification: Theory and Practice*. CRC Press, 2013. ISBN 978-1-040-08117-4.

[12] A. Kourani and N. Daher. Marine locomotion: A tethered UAV-Buoy system with surge velocity control. *Robotics and Autonomous Systems*, 145:103858, Nov. 2021. ISSN 09218890. doi: 10.1016/j.robot.2021.103858.

[13] G. Yu, J. Reis, D. Cabecinhas, R. Cunha, and C. Silvestre. Reduced-Complexity Active Disturbance Rejection Controller for Quadrotor-Slung-Load Transportation. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, 53(8):5248–5259, 2023. ISSN 2168-2216. doi: 10.1109/TSMC.2023.3263881.

[14] F. Novák, T. Báča, and M. Saska. Collaborative Object Manipulation on the Water Surface by a UAV-USV Team Using Tethers. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 3425–3432, Oct. 2024. doi: 10.1109/IROS58592.2024.10802469.

[15] Z. Zhang, S. Liu, H. Wang, and H. Wu. A cooperative control framework for path following of UAV-USV cooperative system in maritime scenarios. *Ocean Engineering*, 342:122796, Dec. 2025. ISSN 00298018. doi: 10.1016/j.oceaneng.2025.122796.

[16] B. D. O. Anderson and J. B. Moore. *Optimal Control: Linear Quadratic Methods*. Prentice Hall Information and System Sciences Series. Prentice Hall, Englewood Cliffs, NJ, 1. [print.] edition, 1990. ISBN 978-0-13-638560-8.

[17] K. Zhou, J. C. Doyle, K. Glover, and J. C. Doyle. *Robust and Optimal Control*. Prentice Hall, Upper Saddle River, NJ, 1996. ISBN 978-0-13-456567-5.

[18] E. F. Camacho and C. Bordons. *Model Predictive Control*. Advanced Textbooks in Control and Signal Processing. Springer, London, second edition edition, 2007. ISBN 978-1-85233-694-3 978-0-85729-398-5. doi: 10.1007/978-0-85729-398-5.

[19] L. Grüne and J. Pannek. *Nonlinear Model Predictive Control: Theory and Algorithms*. Communications and Control Engineering. Springer-Verlag London Limited, London, 2011. ISBN 978-0-85729-500-2 978-0-85729-501-9. doi: 10.1007/978-0-85729-501-9.

[20] J. B. Rawlings, D. Q. Mayne, and M. Diehl. *Model Predictive Control: Theory, Computation and Design*. Nob Hill Publishing, LLC, Santa Barbara, 2nd edition edition, 2020. ISBN 978-0-9759377-5-4.

[21] W. Li, Y. Ge, Z. Guan, H. Gao, and H. Feng. NMPC-based UAV-USV cooperative tracking and landing. *Journal of the Franklin Institute*, 360(11):7481–7500, July 2023. ISSN 00160032. doi: 10.1016/j.jfranklin.2023.06.023.

[22] T. I. Fossen. *Handbook of Marine Craft Hydrodynamics and Motion Control*. John Wiley & Sons, Chichester, UK, 2011. doi: 10.1002/9781119974499.

[23] G. Pannocchia and J. B. Rawlings. Disturbance models for offset-free model-predictive control. *AIChE Journal*, 49(2):426–437, 2003.

[24] U. Maeder, F. Borrelli, and M. Morari. Linear offset-free model predictive control. *Automatica*, 45 (10):2214–2222, 2009.

[25] N. Koenig and A. Howard. Design and algorithms shared by an open source robotics simulator. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, volume 3, pages 2154–2159, 2004.

[26] B. Bingham, C. Agüero, M. McCarrin, J. Klamo, J. Malia, K. Allen, T. Lum, M. Rawson, and R. Waqar. Toward maritime robotic simulation in gazebo. In *MTS/IEEE OCEANS Conference*, pages 1–10, Seattle, WA, USA, Oct. 2019. doi: 10.23919/OCEANS40490.2019.8962724.

[27] Torqeedo GmbH. Cruise 2.0/4.0 R/T operating instructions, 2020. URL `https://www.torqeedo.com/en/customer-downloads.html`. Accessed: 2026-10-02.

[28] Holybro. Holybro X500 V2 development kit, 2023. URL `https://docs.holybro.com/drone-development-kit/px4-development-kit-x500v2`. Accessed: 2026-10-02.

[29] L. Meier, D. Honegger, and M. Pollefeys. PX4: A node-based multithreaded open source robotics framework for deeply embedded platforms. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 6235–6240, 2015. doi: 10.1109/ICRA.2015.7140074.

[30] PX4 Autopilot. PX4 autopilot simulation models: x500 quadrotor, 2025. URL `https://github.com/PX4/PX4-Autopilot`. Release v1.16.1, Tools/simulation/gz/models/x500, accessed: 2026-10-02.

[31] R. Mahony, V. Kumar, and P. Corke. Multirotor aerial vehicles: Modeling, estimation, and control of quadrotor. *IEEE Robotics & Automation Magazine*, 19(3):20–32, 2012. doi: 10.1109/MRA.2012.2206474.

[32] M. Hall and A. Goupee. Validation of a lumped-mass mooring line model with DeepCwind semisubmersible model test data. *Ocean Engineering*, 104:590–603, 2015. doi: 10.1016/j.oceaneng.2015.05.035.

[33] Foxglove Technologies. Foxglove: Visualization and debugging platform for robotics, 2024. URL `https://foxglove.dev`. Accessed: 2026-10-02.

[34] Elistair. Safe-T 2 advanced tethered drone station: Technical specifications, 2024. URL `https://elistair.com/solutions/tethering-station-safe-t/`. Accessed: 2026-10-02.

[35] Linden Photonics. MicroTethers: Thin & lightweight, flexible, power & optics, 2024. URL `https://lindenphotonics.com/documents/Linden%20-%20MicroTethers.pdf`. Accessed: 2026-10-02.

[36] H. M. Irvine. *Cable Structures*. MIT Press, Cambridge, MA, 1981.

[37] J. A. E. Andersson, J. Gillis, G. Horn, J. B. Rawlings, and M. Diehl. CasADi: A software framework for nonlinear optimization and optimal control. *Mathematical Programming Computation*, 11(1): 1–36, Mar. 2019. ISSN 1867-2949, 1867-2957. doi: 10.1007/s12532-018-0139-4.

[38] R. Verschueren, G. Frison, D. Kouzoupis, J. Frey, N. van Duijkeren, A. Zanelli, B. Novoselnik, T. Albin, R. Quirynen, and M. Diehl. acados—a modular open-source framework for fast embedded optimal control. *Mathematical Programming Computation*, 14(1):147–183, 2022. doi: 10.1007/s12532-021-00208-8.

[39] W. Kutta. Beitrag zur näherungsweisen Integration totaler Differentialgleichungen. *Zeitschrift für Mathematik und Physik*, 46:435–453, 1901.

[40] G. Frison and M. Diehl. HPIPM: a high-performance quadratic programming framework for model predictive control. *IFAC-PapersOnLine*, 53(2):6563–6569, 2020. doi: 10.1016/j.ifacol.2020.12.073.

[41] M. Diehl, H. G. Bock, and J. P. Schlöder. A real-time iteration scheme for nonlinear optimization in optimal feedback control. *SIAM Journal on Control and Optimization*, 43(5):1714–1736, 2005. doi: 10.1137/S0363012902400713.

[42] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall. Robot Operating System 2: Design, architecture, and uses in the wild. *Science Robotics*, 7(66):eabm6074, 2022. doi: 10.1126/scirobotics.abm6074.

[43] Open Robotics. ros_gz: Integration between ROS and Gazebo, 2024. URL `https://github.com/gazebosim/ros_gz`. Accessed: 2026-10-21.

[44] eProsima. Micro XRCE-DDS: DDS for extremely resource constrained environments, 2024. URL `https://github.com/eProsima/Micro-XRCE-DDS-Agent`. Accessed: 2026-10-21.